2025-2026.1

CÔNG NGHỆ THÔNG TIN

TRẦN DUY VIỆT

ĐẠI HỌC KINH TẾ QUỐC DÂN

TRƯỜNG CÔNG NGHỆ



KHÓA LUẬN TỐT NGHIỆP

Thiết kế và phát triển nền tảng chấm bài và tổ chức thi lập trình trực tuyến NEU OJ

Tên sinh viên : Trần Duy Việt

Giảng viên hướng dẫn : TS. Phạm Xuân Lâm

Hà Nội, 12/2025

ĐẠI HỌC KINH TẾ QUỐC DÂN

TRƯỜNG CÔNG NGHỆ



KHÓA LUẬN TỐT NGHIỆP

Thiết kế và phát triển nền tảng chấm bài và tổ chức thi lập trình trực tuyến NEU OJ

Tên sinh viên : **Trần Duy Việt**
Mã sinh viên : **11226912**
Lớp : **Công nghệ thông tin 64B**
Ngành : **Công nghệ thông tin**
Khoa : **Công nghệ thông tin**
Giảng viên hướng dẫn : **TS. Phạm Xuân Lâm**

(Chữ ký GVHD)

Hà Nội, 12/2025

MỤC LỤC

MỤC LỤC ..... **i**

LỜI CAM ĐOAN ..... **v**

LỜI CẢM ƠN ..... **vi**

DANH MỤC CÁC TỪ VIẾT TẮT VÀ THUẬT NGỮ ..... **vii**

DANH MỤC HÌNH ..... **ix**

DANH MỤC BẢNG BIỂU **1**

MỞ ĐẦU **2**

CHƯƠNG 1. TỔNG QUAN VỀ ĐỀ TÀI **3**

1.1. Đặt vấn đề **3**

1.2. Mục tiêu nghiên cứu **4**

1.2.1. Mục tiêu tổng quát 4

1.2.2. Mục tiêu cụ thể 4

1.3. Đối tượng nghiên cứu **4**

1.4. Phạm vi nghiên cứu **5**

1.4.1. Phạm vi về đối tượng sử dụng và phạm vi triển khai 5

1.4.2. Phạm vi về chức năng 5

1.4.3. Phạm vi về phi chức năng 6

1.4.4. Các vấn đề ngoài phạm vi nghiên cứu 6

1.5. Đóng góp của đề tài **7**

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ TỔNG QUAN HỆ THỐNG LIÊN QUAN **8**

2.1. Lịch sử phát triển của phương pháp giảng dạy, đánh giá lập trình **8**

2.1.1. Các phương pháp giảng dạy và đánh giá truyền thống 8

2.1.2. Sự ra đời của các hệ thống chấm bài tự động 9

2.1.2.1. Giai đoạn 1: Các hệ thống quản lý kỳ thi cục bộ 9

2.1.2.2. Giai đoạn 2: Sự bùng nổ của các hệ thống chấm bài tự động dựa trên nền tảng website 9
2.1.2.3. Giai đoạn 3: Kỷ nguyên đánh giá thông minh và phản hồi ngữ nghĩa dựa trên AI 10
2.1.3. Xu hướng giáo dục lập trình hiện đại 11
2.1.3.1. Phản hồi tức thì 11
2.1.3.2. Học tập trên nền web và IDE trực tuyến 11
2.1.3.3. Xu hướng ứng dụng trí tuệ nhân tạo trong giảng dạy 11
2.2. Cơ sở lý thuyết về kỹ thuật xây dựng hệ thống 13
2.2.1. Nguyên lý hoạt động chung của quá trình thực thi mã nguồn 13
2.2.1.1. Giai đoạn biên dịch 13
2.2.1.2. Giai đoạn thực thi 13
2.2.2. Mô hình phân lớp của hệ sinh thái OJ 13
2.2.2.1. Lớp nền tảng ứng dụng 13
2.2.2.2. Lớp hệ thống thực thi mã trực tuyến 13
2.2.2.3. Lớp lõi thực thi 14
2.2.2.4. Lớp Sandbox 14
2.2.3. Kỹ thuật ảo hóa và cô lập môi trường 14
2.2.3.1. So sánh các kỹ thuật sandboxing 14
2.2.3.2. Cơ chế hoạt động của isolate 15
2.2.3.3. Các nguy cơ bảo mật và phương án phòng chống 16
2.3. Tổng quan và đánh giá các hệ thống hiện có 16
2.3.1. Các hệ thống thương mại và thi đấu 16
2.3.2. Các hệ thống OJ mã nguồn mở khác 17
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 18
3.1. Phân tích hệ thống 18
3.1.1. Phân tích chức năng 18
3.1.1.1. Biểu đồ Use Case tổng quát hệ thống 18

3.1.1.2. Biểu đồ hoạt động 19
3.1.2. Phân tích cấu trúc 41
3.1.3. Phân tích hành vi 42
3.2. Thiết kế hệ thống 66
3.2.1. Nền tảng công nghệ và môi trường phát triển 66
3.2.1.1. Hệ thống thực thi mã nguồn Judge0 66
3.2.1.2. Framework phát triển phía back-end NestJS 67
3.2.1.3. Framework phát triển phía front-end Next.js 67
3.2.1.4. Hệ quản trị cơ sở dữ liệu MongoDB 68
3.2.1.5. Công nghệ container hoá Docker 69
3.2.1.6. Giao tiếp thời gian thực Socket.IO 69
3.2.1.7. Bộ nhớ đệm và hàng đợi Redis 70
3.2.1.8. Các công nghệ phụ trợ và hạ tầng 70
3.2.2. Biểu đồ gói 71
3.2.3. Biểu đồ triển khai 72
CHƯƠNG 4. KẾT QUẢ TRIỂN KHAI, ĐÁNH GIÁ 75
4.1. Môi trường triển khai và thử nghiệm 75
4.2. Kết quả xây dựng chức năng 75
4.3. Đánh giá hiệu năng hệ thống 85
4.3.1. Phương pháp và công cụ đo lường 85
4.3.2. Đánh giá thời gian xử lý 87
4.3.3. Kiểm thử chịu tải 87
4.4. Kết quả triển khai thực tế 89
4.5. Đánh giá tổng thể, hạn chế và rủi ro 90
4.5.1. So sánh tính năng giữa NEU OJ và các hệ thống phổ biến. 90
4.5.2. Ưu điểm 91
4.5.3. Hạn chế 91

4.5.4. Rủi ro 91
4.5.4.1. Rủi ro về vận hành và tải của hệ thống 91
4.5.4.2. Rủi ro về nguy cơ bị tấn công trong môi trường thực thi mã (sandbox) 92
4.5.4.3. Rủi ro khi tích hợp AI trong tương lai 92
4.6. Đánh giá từ người dùng 92
4.7. Định hướng phát triển 94
4.7.1. Phát triển trợ lý CodeMentor 94
4.7.2. Xây dựng quy trình đánh giá, chấm điểm kết hợp AI 94
KẾT LUẬN 95
TÀI LIỆU THAM KHẢO 96
PHỤ LỤC 99
5.1. Phụ lục A: Xây dựng hệ thống 99
5.1.1. Xây dựng hệ thống back-end 99
5.1.1.1. Thiết lập NodeJS và NestJS 99
5.1.1.2. Thiết lập cơ sở dữ liệu 100
5.1.1.3. Thiết lập dịch vụ biên dịch và thực thi mã nguồn - Judge0 100
5.1.1.4. Thiết lập mở rộng và cân bằng tải cho Judge0 101
5.1.2. Trực quan hoá dữ liệu bằng Grafana và Prometheus 102
5.1.3. Xây dựng hệ thống giao diện NEU OJ 102
5.1.3.1. Thiết lập Next.js Framework 102
5.1.3.2. Kiến trúc thành phần và giao diện người dùng 103
5.1.4. Thực hiện triển khai lên môi trường Production 103
5.1.4.1. Kiến trúc hạ tầng máy chủ 103
5.1.4.2. Khởi chạy hệ thống và thiết lập luồng triển khai tự động 104

LỜI CAM ĐOAN

Tôi xin cam đoan rằng khóa luận tốt nghiệp với đề tài "Thiết kế và phát triển nền tảng chấm bài và tổ chức thi lập trình trực tuyến NEU OJ" là công trình nghiên cứu của cá nhân tôi, được thực hiện dưới sự hướng dẫn một cách khoa học của TS. Phạm Xuân Lâm.

Toàn bộ nội dung trình bày trong khóa luận này là kết quả nghiên cứu, tìm hiểu và triển khai của tôi trong quá trình thực hiện đề tài. Các số liệu, kết quả thí nghiệm và nội dung phân tích được trình bày trong báo cáo là trung thực, không sao chép từ bất kỳ nguồn nào mà không được trích dẫn rõ ràng. Tôi chịu hoàn toàn trách nhiệm về tính trung thực và chính xác của các nội dung trong khóa luận.

Hà Nội, ngày 8 tháng 12 năm 2025

Sinh viên

(ký, ghi rõ họ tên)

LỜI CẢM ƠN

Để hoàn thành khóa luận tốt nghiệp này, bên cạnh sự nỗ lực của bản thân, tôi đã nhận được sự giúp đỡ và động viên rất lớn từ thầy cô và bạn bè. Trước hết, tôi xin gửi lời cảm ơn sâu sắc đến Trường Công nghệ, Đại học Kinh tế Quốc dân và khoa Công nghệ thông tin đã tạo điều kiện thuận lợi cho tôi trong quá trình nghiên cứu.

Tôi xin dành lời cảm ơn đặc biệt nhất đến TS. Phạm Xuân Lâm - người thầy đã luôn nhiệt tình chỉ bảo, sửa chữa những thiếu sót và định hướng tư duy cho tôi từ những ngày đầu thực hiện đề tài.

Xin cảm ơn các thầy cô trong khoa đã truyền lửa đam mê và kiến thức nền tảng vững chắc cho tôi. Cảm ơn những người bạn đã luôn đồng hành và hỗ trợ tôi hết mình.

Dù đã rất cố gắng, song khóa luận chắc chắn vẫn còn những điểm thiếu sót. Tôi kính mong nhận được sự chỉ dẫn và góp ý của quý thầy cô cùng các bạn để bài báo cáo này hoàn chỉnh hơn.

DANH MỤC CÁC TỪ VIẾT TẮT VÀ THUẬT NGỮ

Từ viết tắt	Ý nghĩa
API	Giao diện lập trình ứng dụng (Application Programming Interface)
NEU	Đại học Kinh tế Quốc dân (National Economics University)
OJ	Hệ thống đánh giá trực tuyến (Online Judge)
IDE	Môi trường Phát triển Tích hợp (Integrated Development Environment)
CSDL	Cơ sở dữ liệu
ODM	Object-Document Mapping
OCES	Hệ thống thực thi mã trực tuyến (Online Code Execution System)
CEE	Lõi thực thi mã nguồn (Code Execution Engine)
AI	Trí tuệ nhân tạo
SSO	Single Sign On
RPS	Thông lượng yêu cầu (Request Per Second)
ACM ICPC	International Collegiate Programming Contest
PC^2	Programming Contest Control System
LAN	Mạng cục bộ (Local Area Network)

LLM	Mô hình ngôn ngữ lớn (Large Language Model)
RAG	Retrieval-Augmented Generation
OS	Hệ điều hành (Operating System)
CMS	Contest Management System
CPU	Central Processing Unit
RAM	Random Access Memory
PC	Personal Computer
VPS	Virtual Private Server
S3	Amazon Simple Storage Service
LTS	Long Term Support
CRUD	Create – Read – Update – Delete
SEO	Search Engine Optimization
CI/CD	Continuous Integration/Continuous Delivery

DANH MỤC HÌNH

Hình 3.1. Biểu đồ ca sử dụng tổng quát hệ thống NEU OJ.....18
Hình 3.2. Biểu đồ hoạt động chức năng đăng nhập 19
Hình 3.3. Biểu đồ hoạt động chức năng sử dụng IDE trực tuyến 20
Hình 3.4. Biểu đồ hoạt động chức năng đóng góp bài tập. 21
Hình 3.5. Biểu đồ hoạt động chức năng cập nhật thông tin chung của bài tập. ...22
Hình 3.6. Biểu đồ hoạt động chức năng xoá bài tập. 23
Hình 3.7. Biểu đồ hoạt động chức năng thêm test case.....24
Hình 3.8. Biểu đồ hoạt động chức năng cập nhật test case. 25
Hình 3.9. Biểu đồ hoạt động chức năng xoá test case. 26
Hình 3.10. Biểu đồ hoạt động chức năng cấu hình ngôn ngữ cho bài tập.....27
Hình 3.11. Biểu đồ hoạt động chức năng thêm kỳ thi. 28
Hình 3.12. Biểu đồ hoạt động chức năng cập nhật thông tin chung của kỳ thi....29
Hình 3.13. Biểu đồ hoạt động chức năng xoá kỳ thi. 30
Hình 3.14. Biểu đồ hoạt động chức năng tạo đề cho kỳ thi. 31
Hình 3.15. Biểu đồ hoạt động chức năng xoá bài tập khỏi kỳ thi. 32
Hình 3.16. Biểu đồ hoạt động chức năng thêm quản trị viên cho kỳ thi. 33
Hình 3.17. Biểu đồ hoạt động chức năng xoá quản trị viên khỏi kỳ thi. 34
Hình 3.18. Biểu đồ hoạt động chức năng tham gia kỳ thi. 35
Hình 3.19. Biểu đồ hoạt động chức năng làm bài thi. 36
Hình 3.20. Biểu đồ hoạt động chức năng duyệt kết quả làm bài.....37
Hình 3.21. Biểu đồ hoạt động chức năng giám sát kỳ thi. 38
Hình 3.22. Biểu đồ hoạt động chức năng luyện tập. 39
Hình 3.23. Biểu đồ hoạt động chức năng CodeMentor - Chatbot AI hỏi đáp.....40
Hình 3.24. Biểu đồ lớp hệ thống NEU OJ.....41
Hình 3.25. Biểu đồ tuần tự chức năng đăng nhập bằng SSO. 42
Hình 3.26. Biểu đồ tuần tự chức năng IDE trực tuyến. 43

Hình 3.27. Biểu đồ tuần tự chức năng đóng góp bài tập. 44
Hình 3.28. Biểu đồ tuần tự chức năng cập nhật thông tin chung của bài tập. 45
Hình 3.29. Biểu đồ tuần tự chức năng xoá bài tập. 46
Hình 3.30. Biểu đồ tuần tự chức năng thêm test case vào bài tập. 47
Hình 3.31. Biểu đồ tuần tự chức năng cập nhật test case. 48
Hình 3.32. Biểu đồ tuần tự chức năng xoá test case. 49
Hình 3.33. Biểu đồ tuần tự chức năng cấu hình ngôn ngữ cho bài tập. 50
Hình 3.34. Biểu đồ tuần tự chức năng thêm kỳ thi. 51
Hình 3.35. Biểu đồ tuần tự chức năng cập nhật thông tin chung của kỳ thi. 52
Hình 3.36. Biểu đồ tuần tự chức năng xoá kỳ thi. 53
Hình 3.37. Biểu đồ tuần tự chức năng tạo đề cho kỳ thi. 54
Hình 3.38. Biểu đồ tuần tự chức năng xoá bài tập khỏi kỳ thi. 55
Hình 3.39. Thêm quản trị viên cho kỳ thi. 56
Hình 3.40. Biểu đồ tuần tự chức năng xoá quản trị viên khỏi kỳ thi. 57
Hình 3.41. Biểu đồ tuần tự chức năng tham gia kỳ thi. 59
Hình 3.42. Biểu đồ tuần tự chức năng làm bài thi. 60
Hình 3.43. Biểu đồ tuần tự chức năng duyệt kết quả làm bài. 62
Hình 3.44. Biểu đồ tuần tự chức năng giám sát kỳ thi. 63
Hình 3.45. Biểu đồ tuần tự chức năng luyện tập. 64
Hình 3.46. Biểu đồ tuần tự chức năng CodeMentor. 65
Hình 3.47. Biểu đồ gói hệ thống NEU OJ 71
Hình 3.48. Biểu đồ triển khai hệ thống NEU OJ. 72
Hình 3.49. Sơ đồ kiến trúc tổng quan NEU OJ. 74
Hình 4.1. Giao diện màn hình IDE trực tuyến. 76
Hình 4.2. Giao diện màn hình danh sách kỳ thi. 76
Hình 4.3. Giao diện màn hình đề thi. 77
Hình 4.4. Giao diện màn hình làm bài thi. 77

Hình 4.5. Giao diện cảnh báo gian lận. 78
Hình 4.6. Giao diện chức năng xem kết quả nộp bài. 78
Hình 4.7. Giao diện màn hình quản lý bài tập. 79
Hình 4.8. Giao diện màn hình đóng góp/cập nhật bài tập. 79
Hình 4.9. Giao diện màn hình chức năng thêm/cập nhật test case. 80
Hình 4.10. Giao diện màn hình chức năng cấu hình ngôn ngữ cho bài tập. 80
Hình 4.11. Giao diện màn hình quản lý kỳ thi. 81
Hình 4.12. Giao diện màn hình chức năng thêm/cập nhật kỳ thi. 81
Hình 4.13. Giao diện màn hình chức năng quản lý đề thi. 82
Hình 4.14. Giao diện màn hình chức năng thêm bài tập vào đề thi. 82
Hình 4.15. Giao diện màn hình tổng quan điểm số kỳ thi. 83
Hình 4.16. Giao diện màn hình chi tiết bài thi của người tham gia. 83
Hình 4.17. Giao diện màn hình chức năng giám sát kỳ thi. 84
Hình 4.18. Giao diện chức năng luyện tập. 84
Hình 4.19. Giao diện chức năng báo lỗi. 85
Hình 4.20. Mã Python sử dụng để chạy kiểm thử chịu tải trên NEU OJ. 86
Hình 4.21. Kết quả thực thi đoạn mã kiểm thử chịu tải. 86
Hình 4.22. Cấu hình kịch bản kiểm thử với Loctus. 86
Hình 4.23. Thời gian phản hồi với 1 yêu cầu. 87
Hình 4.24. Kết quả thống kê chịu tải tại 1000 người dùng đồng thời. 88
Hình 4.25. Kết quả thống kê chịu tải tại 1500 người dùng đồng thời. 88
Hình 4.26. Kết quả thống kê chịu tải tại 2000 người dùng đồng thời. 88
Hình 4.27. Các chủ đề ôn luyện hiện có trên hệ thống. 89
Hình 4.28. Leaderboard cuộc thi Olympic Tin học cấp đại học khối chuyên. 89
Hình 4.29. Người dùng báo lỗi font web khó đọc. 93

DANH MỤC BẢNG BIỂU

Bảng 1. So sánh các kỹ thuật sandboxing. 15

Bảng 2. Kết quả kiểm thử chịu tải theo các mức tải.....87

Bảng 3. Bảng so sánh tính nữa giữa NEU OJ với các hệ thống khác. 90

MỞ ĐẦU

Trong kỷ nguyên số hóa, khi công nghệ trở thành động lực cốt lõi thúc đẩy mọi lĩnh vực của đời sống xã hội, việc đào tạo nguồn nhân lực chất lượng cao với tư duy thuật toán sắc bén và kỹ năng lập trình vững chắc là nhiệm vụ trọng tâm đối với sinh viên lĩnh vực Công nghệ thông tin và Khoa học máy tính. Để đáp ứng nhu cầu này, việc xây dựng một môi trường học tập và rèn luyện hiệu quả, nơi sinh viên có thể thực hành, được đánh giá và tham gia các kỳ thi một cách công bằng và minh bạch là vô cùng cần thiết. Tuy nhiên, các phương pháp giảng dạy và kiểm tra truyền thống thường gặp khó khăn trong việc cung cấp nhận xét, phản hồi ngay lập tức và vẫn chưa hoàn toàn có thể tổ chức các cuộc thi quy mô lớn một cách hiệu quả.

Nhận thấy nhu cầu cấp thiết đó, đề tài **"Thiết kế và phát triển nền tảng chấm bài và tổ chức thi lập trình trực tuyến NEU OJ"** được thực hiện nhằm phát triển một nền tảng hiện đại phục vụ quá trình học tập và rèn luyện kỹ năng lập trình cho sinh viên tại Đại học Kinh tế Quốc dân. Hệ thống NEU OJ được thiết kế như một môi trường toàn diện, tập trung vào các tính năng cốt lõi: cung cấp thư viện bài tập đa dạng, tích hợp trình soạn thảo mã nguồn trực tuyến mạnh mẽ và tự động hóa quy trình chấm bài dựa trên bộ dữ liệu kiểm thử. Bên cạnh đó, hệ thống còn cung cấp khả năng tổ chức các cuộc thi lập trình chuyên nghiệp với bảng xếp hạng cập nhật theo thời gian thực và các công cụ giám sát hiệu quả, công bằng.

Mục tiêu của đề tài là xây dựng một hệ thống nhằm nâng cao trải nghiệm học tập và giảng dạy, giúp sinh viên tương tác, thực hành thuật toán và cấu trúc dữ liệu một cách hiệu quả. Đồng thời, cung cấp cho giảng viên các công cụ mạnh mẽ để quản lý và giám sát thi cử một cách minh bạch. Với kiến trúc hệ thống được thiết kế theo hướng mở và hiện đại, đề tài cũng đặt nền móng định hướng cho việc mở rộng và tích hợp các công nghệ hiện đại hơn trong tương lai, như việc ứng dụng AI để hỗ trợ tra cứu kiến thức cơ bản hay chấm bài và đưa ra nhận xét tự động, nhằm không ngừng hoàn thiện môi trường giáo dục số tại trường.

CHƯƠNG 1. TỔNG QUAN VỀ ĐỀ TÀI

1.1. Đặt vấn đề

Trong môi trường đào tạo các lĩnh vực Công nghệ thông tin và Khoa học máy tính hiện đại, việc trang bị cho sinh viên kỹ năng lập trình vững chắc không chỉ dừng lại ở kiến thức lý thuyết mà đòi hỏi một quá trình rèn luyện thực hành liên tục và cường độ cao. Tuy nhiên, quy trình dạy và học lập trình theo phương pháp truyền thống tại nhiều cơ sở giáo dục, bao gồm cả Đại học Kinh tế Quốc dân, đang bộc lộ một số bất cập, trở thành rào cản trong việc nâng cao chất lượng đào tạo.

Về phía người học, thách thức lớn nhất là chu trình phản hồi chậm (slow feedback loop). Khi hoàn thành một bài tập lập trình, sinh viên phải chờ đợi một khoảng thời gian dài để giảng viên hoặc trợ giảng chấm chữa thủ công. Sự chậm trễ này làm gián đoạn dòng tư duy, khiến sinh viên khó nhận ra và sửa chữa lỗi sai một cách kịp thời, từ đó làm giảm hiệu quả học tập [1]. Thêm vào đó, sinh viên thường thiếu một nguồn tài nguyên học tập tập trung, với một thư viện bài tập đủ lớn, đa dạng về độ khó và được hệ thống hóa để có thể tự luyện tập và đánh giá năng lực của bản thân ngoài giờ học.

Về phía giảng viên và nhà trường, công tác tổ chức giảng dạy và đánh giá gặp phải nhiều khó khăn. Việc chấm hàng trăm, thậm chí hàng nghìn bài nộp của sinh viên một cách thủ công là một công việc tốn nhiều thời gian, công sức và dễ xảy ra sai sót, thiếu nhất quán [2]. Hơn nữa, việc tổ chức các kỳ thi lập trình quy mô lớn theo cách truyền thống đặt ra những thách thức không nhỏ về công tác hậu cần, giám sát và đảm bảo tính công bằng, minh bạch. Các vấn đề như gian lận, khó khăn trong việc thiết kế bộ dữ liệu kiểm thử đồng nhất và áp lực chấm thi khổng lồ là những bài toán nan giải.

Những hạn chế nêu trên tạo ra một "khoảng trống" cấp thiết, đòi hỏi một giải pháp công nghệ có khả năng tự động hóa quy trình đánh giá, cung cấp phản hồi tức thì và xây dựng một môi trường học tập, thi cử chuyên nghiệp. Chính vì vậy, đề tài **"Thiết kế và phát triển nền tảng chấm bài và tổ chức thi lập trình trực tuyến NEU OJ"** được thực hiện nhằm giải quyết trực tiếp các vấn đề này, mang đến một công cụ hiện đại để tối ưu hóa trải nghiệm dạy và học, góp phần nâng cao năng lực cạnh tranh cho sinh viên lĩnh vực Công nghệ thông tin và Khoa học máy tính.

1.2. Mục tiêu nghiên cứu

1.2.1. Mục tiêu tổng quát

Xây dựng thành công nền tảng chấm bài và tổ chức thi lập trình trực tuyến NEU Online Judge (NEU OJ), nhằm tạo ra một công cụ học tập, rèn luyện và đánh giá kỹ năng lập trình hiện đại, hiệu quả cho sinh viên và giảng viên tại Đại học Kinh tế Quốc dân.

1.2.2. Mục tiêu cụ thể

Để đạt được mục tiêu tổng quát, đề tài tập trung giải quyết các nhiệm vụ cụ thể sau:

1. Nghiên cứu cơ sở lý thuyết: Tìm hiểu về quy trình chấm bài tự động, các kỹ thuật đảm bảo an toàn trong thực thi mã nguồn và kiến trúc hệ thống Online Judge hiện đại.
2. Phân tích và thiết kế hệ thống: Xây dựng bộ yêu cầu phần mềm, thiết kế kiến trúc tổng thể và cơ sở dữ liệu cho hệ thống NEU OJ dựa trên các công nghệ Next.js, NestJS và MongoDB.
3. Tích hợp lõi chấm bài: Ứng dụng thành công Judge0 để xử lý cốt lõi việc biên dịch và thực thi mã nguồn đa ngôn ngữ, đảm bảo tính chính xác và hiệu năng cao.
4. Xây dựng các phân hệ chức năng: Phát triển hoàn thiện các module: Thư viện bài tập, Quản lý cuộc thi, và Công cụ hỗ trợ giảng viên theo thiết kế đã đề ra.
5. Đảm bảo hiệu năng và độ ổn định: Xây dựng hệ thống có khả năng chịu tải ổn định với số lượng người dùng đồng thời tối thiểu đạt 500 người. Tối ưu hóa quy trình chấm bài để thời gian xử lý hệ thống (không bao gồm thời gian thực thi thuật toán của sinh viên) đạt dưới 5 giây và độ trễ xếp hàng dưới 500ms trong điều kiện tải thông thường.
6. Kiểm thử và đánh giá: Triển khai thử nghiệm hệ thống trong phạm vi nội bộ, thực hiện kiểm thử chức năng và hiệu năng để đảm bảo sự ổn định trước khi đưa vào sử dụng thực tế.

1.3. Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài bao gồm:

- Quy trình nghiệp vụ Online Judge: Các quy trình về quản lý bài tập, tổ chức thi, nộp bài, chấm điểm tự động và xếp hạng thí sinh trong môi trường giáo dục đại học.
- Lịch sử, cách triển khai các phương pháp và quy trình đánh giá mã nguồn tự động.
- Kỹ thuật ảo hoá và cô lập môi trường thực thi mã nguồn: Tập trung nghiên cứu cơ chế cô lập tiến trình (Linux Namespaces) và quản lý tài nguyên (Cgroups) để xây dựng môi trường thực thi mã nguồn an toàn, ngăn chặn các hành vi nguy hiểm (Fork bomb, truy cập trái phép) từ mã nguồn người dùng.
- Kiến trúc hệ thống phân tán và mô hình phân lớp: Nghiên cứu mô hình kiến trúc microservices và cơ chế cân bằng tải để phân tách logic nghiệp vụ khỏi logic chấm bài, đảm bảo khả năng mở rộng theo chiều ngang khi số lượng người dùng tăng cao.
- Kỹ thuật phát triển ứng dụng Web hiệu năng cao: Nghiên cứu các mẫu thiết kế trong xây dựng back-end và kỹ thuật rendering tối ưu phía front-end để đảm bảo trải nghiệm người dùng mượt mà.

1.4. Phạm vi nghiên cứu

1.4.1. Phạm vi về đối tượng sử dụng và phạm vi triển khai

Hệ thống được thiết kế và xây dựng hướng tới đối tượng người dùng chính là sinh viên và giảng viên thuộc khoa Công nghệ thông tin, Trường Công nghệ, Đại học Kinh tế Quốc dân. Các luồng nghiệp vụ, giao diện và tính năng sẽ được tối ưu hóa dựa trên khảo sát và nhu cầu thực tế của nhóm đối tượng này.

Do đó, phạm vi triển khai và thu thập phản hồi ban đầu sẽ được thực hiện trong môi trường nội bộ của khoa. Kết quả đánh giá về tính hiệu quả và sự phù hợp của hệ thống sẽ mang tính cục bộ, đại diện cho một nhóm người dùng cụ thể. Tuy nhiên, nền tảng được xây dựng với kiến trúc mở, hướng tới khả năng nhân rộng và áp dụng cho các đơn vị đào tạo khác trong tương lai.

1.4.2. Phạm vi về chức năng

Đề tài tập trung nghiên cứu và phát triển các nhóm chức năng cốt lõi sau:

- **Luyện tập:** Truy cập thư viện bài tập mở, tìm kiếm, lọc bài tập và nộp bài để hệ thống chấm tự động.

- **Thi đấu/Kiểm tra:** Đăng ký và tham gia các kỳ thi lập trình, xem đề thi, nộp bài trong thời gian quy định và theo dõi bảng xếp hạng thời gian thực.
- **Đóng góp:** Cho phép cộng đồng sinh viên/giảng viên đóng góp đề bài mới vào kho tài nguyên chung.
- **Trải nghiệm:** Sử dụng trình soạn thảo mã nguồn trực tuyến với các tính năng đầy đủ.
- **Quản lý thư viện đề:** Tạo mới, chỉnh sửa, xóa bài tập; cấu hình bộ dữ liệu kiểm thử; phân loại độ khó.
- **Quản lý cuộc thi:** Tạo cuộc thi, thiết lập thời gian, danh sách thí sinh, và cấu hình các ràng buộc (ngôn ngữ, bộ nhớ, thời gian chạy).
- **Giám sát và chấm thi:** Theo dõi tiến trình cuộc thi, hệ thống hỗ trợ phát hiện các dấu hiệu gian lận cơ bản; thực hiện chạy thử hoặc chấm lại các bài nộp khi cần thiết.

1.4.3. Phạm vi về phi chức năng

Hệ thống cần đáp ứng các tiêu chuẩn kỹ thuật sau:

- Hiệu năng: Đảm bảo khả năng chấm bài nhanh chóng, bảng xếp hạng cập nhật với độ trễ thấp ngay cả khi có lượng truy cập đồng thời cao trong giờ thi.
- Tính an toàn và bảo mật: Việc thực thi mã nguồn của người dùng phải được cô lập hoàn toàn để tránh ảnh hưởng đến máy chủ; đảm bảo toàn vẹn dữ liệu điểm số.
- UI/UX: Giao diện trực quan, thân thiện, dễ sử dụng cho cả người không chuyên sâu về kỹ thuật; tương thích tốt trên các trình duyệt phổ biến.
- Khả năng mở rộng: Kiến trúc hệ thống được thiết kế dạng module, dễ dàng nâng cấp, thêm mới ngôn ngữ lập trình hoặc tích hợp các tính năng nâng cao như ứng dụng AI vào hỏi đáp hoặc đánh giá bài tập.

1.4.4. Các vấn đề ngoài phạm vi nghiên cứu

- Về cơ chế giám sát: Đề tài tập trung giải quyết bài toán giám sát hành vi thi cử thời gian thực như phát hiện chuyển tab, rời khỏi màn hình. Việc phân tích sâu mã nguồn để phát hiện sao chép bằng các thuật toán phức tạp hoặc phân tích phong cách lập trình chưa nằm trong phạm vi triển khai của phiên bản này.

- Về ứng dụng trí tuệ nhân tạo: Đề tài giới hạn ở mức đề xuất ý tưởng và thiết kế mô hình lý thuyết, thể hiện qua các biểu đồ phân tích và thiết kế hệ thống, cho các chức năng trợ lý ảo CodeMentor và quy trình nhận xét, chấm điểm kết hợp mô hình ngôn ngữ lớn. Việc triển khai thực tế giao diện người dùng, tích hợp dịch vụ và triển khai tính năng cho các module AI này chưa được thực hiện trong phạm vi khóa luận và được xác định là hướng phát triển trong tương lai.
- Về hạ tầng triển khai: Hệ thống được thiết kế và triển khai tối ưu trên nền tảng máy chủ vật lý sử dụng công nghệ container hóa. Đề tài chưa đi sâu nghiên cứu các giải pháp mở rộng tự động trên nền tảng điện toán đám mây như Kubernetes hay Serverless để phục vụ quy mô người dùng cực lớn.

1.5. Đóng góp của đề tài

Đóng góp quan trọng nhất của đề tài là việc thiết kế và xây dựng thành công nền tảng chấm bài trực tuyến NEU OJ hoàn chỉnh. Hệ thống được phát triển đồng bộ từ các phân hệ giao diện (front-end), hệ thống quản lý (back-end) cho đến lõi chấm bài, mang lại một môi trường thực hành hiện đại giúp khắc phục những hạn chế về giao diện tương tác và trải nghiệm người dùng thường gặp ở các hệ thống chấm bài cũ.

Về mặt kỹ thuật, khóa luận đã đề xuất và hiện thực hóa một kiến trúc hệ thống an toàn, linh hoạt dựa trên mô hình Microservices kết hợp với công nghệ container hóa. Đặc biệt, việc ứng dụng thành công cơ chế cô lập sandbox/isolate đã giúp tách biệt hoàn toàn môi trường thực thi mã nguồn khỏi hệ thống quản lý trung tâm. Kiến trúc này không chỉ đảm bảo tính bảo mật, ngăn chặn các nguy cơ tấn công từ mã nguồn người dùng mà còn tạo tiền đề vững chắc cho việc mở rộng tài nguyên hệ thống theo chiều ngang trong tương lai.

Tính thực tiễn và hiệu quả của đề tài đã được kiểm chứng rõ nét thông qua quá trình triển khai thực tế tại trường. Hệ thống đã vượt qua phạm vi của một mô hình thử nghiệm nội bộ để trở thành nền tảng chính thức phục vụ tổ chức thành công kỳ thi Olympic Tin học cấp đại học và các bài kiểm tra kết thúc học phần. Trong suốt quá trình vận hành thực tế, hệ thống đã chứng minh được sự ổn định, độ tin cậy cao và khả năng xử lý mượt mà các tác vụ biên dịch, chấm điểm trong điều kiện áp lực thời gian thực của kỳ thi, đáp ứng trọn vẹn các yêu cầu khắt khe về nghiệp vụ tổ chức thi cử.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ TỔNG QUAN HỆ THỐNG LIÊN QUAN

2.1. Lịch sử phát triển của phương pháp giảng dạy, đánh giá lập trình

Sự phát triển của các hệ thống đánh giá lập trình tự động không chỉ đơn thuần là sự tiến bộ về mặt công nghệ phần mềm, mà còn phản ánh sự chuyển dịch trong triết lý giáo dục: từ mô hình lấy giảng viên làm trung tâm sang mô hình lấy người học làm trung tâm, nơi phản hồi tức thì và khả năng tự điều chỉnh đóng vai trò then chốt. Lịch sử này trả dài hơn 60 năm, song hành cùng sự bùng nổ của khoa học máy tính.

2.1.1. Các phương pháp giảng dạy và đánh giá truyền thống

Trong giai đoạn sơ khai của giáo dục khoa học máy tính, khoảng những năm 1960, việc đánh giá năng lực lập trình là một quy trình thủ công cực kỳ tốn kém và chậm chạp. Các hệ thống đánh giá đầu tiên xuất hiện tại Đại học Stanford vào năm 1961 để hỗ trợ chấm điểm cho ngôn ngữ lập trình ALGOL, đánh dấu những nỗ lực đầu tiên nhằm giảm tải cho con người [3]. Tuy nhiên, trong phần lớn thế kỷ 20, phương pháp chấm thủ công vẫn chiếm ưu thế.

Phương pháp truyền thống yêu cầu sinh viên nộp mã nguồn thông qua các phương tiện vật lý (như thẻ đục lỗ trong thập kỷ 60-70) hoặc sau này là qua email và các hệ thống quản lý học tập (LMS) tệp tin đơn giản [4]. Giảng viên hoặc trợ giảng phải thực hiện một quy trình lặp lại: thiết lập môi trường cục bộ, biên dịch, thực thi với dữ liệu kiểm thử hạn chế và đọc mã nguồn để kiểm tra cú pháp, cách giải bài toán.

Những phương pháp đánh giá năng lực lập trình thủ công như kể trên được cho là không hề khả thi hoặc gặp khó khăn khi quy mô lớp học lớn [5], điều này đồng nghĩa với việc khả năng mở rộng của các phương pháp này là hạn chế hoặc gần như bằng không. Khi số lượng sinh viên trong các lớp học tăng lên hàng trăm, duy trì việc trả về nhận xét và đánh giá kịp thời cho từng có nhân là bất khả thi. Sự chậm trễ trong phản hồi – có thể kéo dài từ vài ngày đến vài tuần – làm đứt gãy quy trình nhận thức của người học. Sinh viên không thể liên kết được điểm số nhận được với quá trình tư duy khi viết mã nguồn, dẫn đến hiệu quả học tập giảm sút nghiêm trọng [6].

Hơn nữa, tính chủ quan và thiếu nhất quán là điểm yếu cố hữu của con người. Các nghiên cứu so sánh độ tin cậy giữa chấm bằng máy và chấm thủ công

cho thấy con người dễ bị ảnh hưởng bởi các yếu tố ngoại lai như mệt mỏi, thiên kiến cá nhân hoặc sự khác biệt trong tiêu chí đánh giá giữa các giảng viên [2]. Một vấn đề khác là việc kiểm thử thủ công thường chỉ bao phủ được các trường hợp thông thường mà bỏ qua các trường hợp biên hoặc các vấn đề về hiệu năng như thời gian thực thi, bộ nhớ… dẫn đến việc đánh giá không toàn diện độ chính xác và hiệu quả của thuật toán [7].

2.1.2. Sự ra đời của các hệ thống chấm bài tự động

Nhu cầu khắc phục các hạn chế trên đã thúc đẩy sự ra đời của các hệ thống chấm bài tự động (Online Judges – OJ). Quá trình này có thể được phân chia thành các giai đoạn phát triển dựa trên kiến trúc hệ thống.

2.1.2.1. Giai đoạn 1: Các hệ thống quản lý kỳ thi cục bộ

Đại diện tiêu biểu nhất cho giai đoạn này là PC^2 – Programming Contest Control System, được phát triển tại Đại học California State, Sacramento [8]. Trong suốt thập niên 90 và đầu những năm 2000, PC^2 là tiêu chuẩn vàng cho các kỳ thi Lập trình Sinh viên Quốc tế (ACM ICPC).

PC^2 hoạt động dựa trên mô hình máy trạm – máy chủ trong mạng nội bộ (LAN). Thí sinh sử dụng một phần mềm chuyên dụng tại máy khách để nộp bài và máy chủ trung tâm sẽ phân phối bài nộp đến các máy chấm. Mặc dù PC^2 giải quyết tốt bài toán tự động hoá và tính điểm thời gian thực, kiến trúc của nó bộc lộ một vài điểm bất lợi, cứng nhắc:

- Phụ thuộc vào môi trường: Yêu cầu cài đặt phần mềm phức tạp trên từng máy trạm của thi sinh tham gia kỳ thi, cấu hình tường lửa và mạng LAN nghiêm ngặt.
- Thiếu tính linh hoạt: Khó khăn trong việc hỗ trợ truy cập từ xa qua internet hoặc triển khai cho các lớp học phân tán.
- Giao diện người dùng: Giao diện cũ kỹ, thiếu tính tương tác và trực quan so với các tiêu chuẩn phần mềm hiện tại.

2.1.2.2. Giai đoạn 2: Sự bùng nổ của các hệ thống chấm bài tự động dựa trên nền tảng website

Sự phổ biến của Internet đã mở ra kỷ nguyên của các OJ dựa trên nền tảng web. Một trong những đại diện đầu tiên của xu thế này là ***Uva Online Judge*** (Đại học Valladolid, Tây Ban Nha, 1995) [9]. Hệ thống này cho phép người dùng toàn

cầu nộp bài qua giao diện web và nhận kết quả qua email, tạo một kho lưu trữ bài toán khổng lồ đầu tiên trên thế giới.

Tiếp nối là các hệ thống như SPOJ (Sphere Online Judge), POJ (Peking University) và thế hệ hiện đại như Codeforces, LeetCode, HackerRank… Đặc điểm chung của thế hệ này là khả năng tiếp cận mọi lúc mọi nơi, cộng đồng lớn mạnh và kho bài tập phong phú. Về mặt kỹ thuật, các hệ thống này chuyển dịch từ việc chạy các kịch bản thực thi mã nguồn đơn giản, dễ bị tấn công sang sử dụng các kỹ thuật cô lập tiên tiến dựa trên cơ chế của hệ điều hành để đảm bảo an toàn khi chạy mã nguồn không tin cậy [10].

2.1.2.3. Giai đoạn 3: Kỷ nguyên đánh giá thông minh và phản hồi ngữ nghĩa dựa trên AI

Từ năm 2023, sự bùng nổ của các mô hình ngôn ngữ lớn như GPT-4, Claude và Gemini đã mở ra giai đoạn thứ ba. Giai đoạn này đánh dấu sự chuyển dịch từ đánh giá dựa trên kết quả sang đánh giá dựa trên quá trình và ngữ nghĩa.

Về độ chính xác và tương quan với con người, các nghiên cứu thực nghiệm năm 2025 đã chứng minh AI có khả năng đánh giá với độ chính xác cao tương đương với giảng viên. Nghiên cứu của Bernik et al. (2025) trên 315 bài nộp Python cho thấy mô hình GPT-4 Turbo đạt hệ thống tương quan Pearson 0.91 so với điểm số của giảng viên, với sai số tuyệt đối trung bình là 0.68 điểm trên thang 10 [11]. Điều này khẳng định các mô hình ngôn ngữ lớn có thể đóng vai trò "người chấm sơ loại" đáng tin cậy. Tuy nhiên, Jukiewicz (2025) lưu ý rằng các mô hình có "triết lý" chấm điểm khác nhau: GPT-4o có xu hướng "phóng khoáng" trong khi các mô hình nhỏ hơn như GPT-4.1-nano thường khắt khe hơn con người [12].

Về khả năng phản hồi và giải thích, khác với thông báo lỗi khô khan của các máy chấm bài tự động dựa trên đầu vào – đầu ra, hệ thống ở giai đoạn 3 cung cấp phản hồi mang tính định hướng và xây dựng. Piscitelli et al. (2025) chỉ ra rằng khi sử dụng chiến lược Few-shot Prompting – cung cấp ví dụ mẫu, các mô hình như Claude-Sonnet có thể sinh ra các nhận xét chi tiết về cấu trúc mã nguồn, phong cách lập trình và gợi ý sửa lỗi cụ thể mà không tiết lộ đáp án [13]. Bernik et al. cũng ghi nhận AI có khả năng phát hiện các lỗi cú pháp và logic tinh vi mà đôi khi giảng viên bỏ sót do mệt mỏi [11].

Mặc dù mạnh mẽ, việc áp dụng các mô hình ngôn ngữ lớn trong việc đánh giá kết quả lập trình của sinh viên vẫn còn nhiều thách thức, điển hình là sự thiếu nhất quá về định dạng và ảo giác của các mô hình này. Pitscitelli et al. phát hiện

rằng trong chế độ Zero-shot – không có ví dụ, nhiều mô hình thất bại trong việc tuân thủ định dạng điểm số yêu cầu [13]. Do đó, xu hướng hiện tại là mô hình lai (hybrid), nghĩa là sử dụng kết quả thực thi mã nguồn của các trình thực thi ở các giai đoạn trước, kết hợp với AI để phân tích ngữ nghĩa và tạo phản hồi, dưới sự giám sát cuối cùng của con người.

2.1.3. Xu hướng giáo dục lập trình hiện đại

2.1.3.1. Phản hồi tức thì

Nghiên cứu của Kehrer et al. (2013) và Attali & Powers (2009) khẳng định rằng sinh viên nhận được phản hồi tức thì có hiệu suất học tập vượt trội so với nhóm nhận phản hồi chậm [14], [15]. Cơ chế phản hồi tức thì tạo ra một vòng lặp học tập chặt chẽ: thử nghiệm, thất bại, phân tích lỗi, sửa chữa, thành công. Các thông báo lỗi cụ thể đến từ hệ thống như “Wrong answer on test 5”, “Segmentation Fault” giúp sinh viên nhanh chóng định vị vấn đề và phát triển tư duy gỡ lỗi [16]. Một khảo sát định lượng trên 151 sinh viên cũng cho thấy điểm số bài kiểm tra sau thử nghiệm (post-test scores) của nhóm nhận phản hồi ngay lập tức cao hơn đáng kể so với nhóm đối chứng [1].

2.1.3.2. Học tập trên nền web và IDE trực tuyến

Xu hướng chuyển dịch công cụ soạn thảo và biên dịch lên trực tuyến đang loại bỏ rào cản nhập môn. Thay vì mất hàng giờ cài đặt môi trường như trình biên dịch, IDE cục bộ và dễ gặp lỗi cấu hình, sinh viên có thể viết mã nguồn, biên dịch và chạy ngay trên trình duyệt thông qua các trình soạn thảo tích hợp [10]. Điều này đặc biệt quan trọng trong các kỳ thi tập trung, nơi môi trường lập trình tại máy trạm cần được chuẩn hoá, đồng bộ và kiểm soát chặt chẽ.

2.1.3.3. Xu hướng ứng dụng trí tuệ nhân tạo trong giảng dạy

Phương pháp Socratic trong Khanmigo: Từ "Trả lời" đến "Dẫn dắt"

Một trong những ứng dụng sư phạm tiên tiến nhất là việc chuyển đổi AI từ vai trò "người giải bài tập" sang "người hướng dẫn Socratic". Khanmigo, được phát triển bởi Khan Academy trên nền tảng GPT-4 [17], là ví dụ điển hình cho cơ chế này. Thay vì cung cấp đáp án ngay lập tức khi sinh viên đặt câu hỏi, Khanmigo được lập trình (thông qua *system prompts*) để đặt ngược lại các câu hỏi gợi mở [17]. Phương pháp này ngăn chặn việc "gian lận thụ động" và buộc sinh viên phải tham gia vào quá trình tư duy phản biện.

Hệ thống Phản hồi tự động

Việc chấm điểm và phản hồi mã nguồn thủ công là gánh nặng lớn đối với giảng viên. Các hệ thống mới sử dụng LLM để tự động hóa quy trình này với độ chính xác cao. Quy trình 2 lớp: một nghiên cứu về hệ thống chấm điểm Python sử dụng GPT-4 Turbo đã đề xuất quy trình sau [18]:

1. Lớp 1 - Kiểm thử đơn vị: Mã của sinh viên được chạy qua các test case tiêu chuẩn. Nếu thất bại, nó được chuyển sang lớp 2.
2. Lớp 2 - Sửa lỗi và phân tích: LLM được yêu cầu sửa mã của sinh viên với thay đổi tối thiểu.
3. So sánh & phản hồi: Hệ thống so sánh mã gốc và mã đã sửa để xác định loại lỗi (logic, cú pháp, hay thừa mã) và tạo ra gợi ý thay vì lời giải.

Kết quả là hệ thống này đạt độ chính xác 63% trong việc đưa ra gợi ý hoàn toàn chính xác và đầy đủ, giúp giảm tải đáng kể cho giảng viên. Tuy nhiên, vẫn cần sự giám sát của con người do 37% gợi ý vẫn chứa lỗi hoặc ảo giác [18].

Kiến trúc RAG

Để giải quyết vấn đề "ảo giác" của AI, các hệ thống gia sư thông minh hiện đại như LPITutor sử dụng kiến trúc RAG. Vấn đề là các LLM tiêu chuẩn (như GPT-3.5) có thể giải thích sai các khái niệm lập trình chuyên sâu hoặc cung cấp các thư viện không tồn tại [19].

Giải pháp RAG cho vấn đề này:

- Truy xuất: Khi sinh viên hỏi một câu hỏi, hệ thống trước tiên tìm kiếm trong một kho tri thức chuẩn (ví dụ: tài liệu giáo trình, tài liệu Python chính thức) để tìm các đoạn văn bản liên quan.
- Tăng cường: Các đoạn văn bản này được đưa vào prompt gửi cho AI làm "bối cảnh”.
- Tạo sinh: AI trả lời câu hỏi dựa chỉ trên bối cảnh được cung cấp.

Prompt Engineering hai lớp: LPITutor sử dụng một lớp prompt tĩnh (khuôn mẫu sư phạm) và một lớp prompt động (nội dung bài học cụ thể) để đảm bảo câu trả lời vừa chính xác về kiến thức, vừa phù hợp với trình độ của sinh viên [19].

2.2. Cơ sở lý thuyết về kỹ thuật xây dựng hệ thống

2.2.1. Nguyên lý hoạt động chung của quá trình thực thi mã nguồn

Một hệ thống OJ thực chất là một hệ thống đánh giá dựa trên đầu vào – đầu ra. Quy trình này dựa trên nguyên lý hoạt động của trình biên dịch và trình thông dịch.

2.2.1.1. Giai đoạn biên dịch

Đối với các ngôn ngữ biên dịch như C, C++, Go,… mã nguồn được chuyển đổi thành tệp tin nhị phân. Quá trình này bao gồm các bước: Phân tích từ vựng, phân tích cú pháp, phân tích ngữ nghĩa và sinh mã [20]. Hệ thống OJ cần bắt giữ được các luồng lỗi chuẩn (stderr) từ trình biên dịch để trả về thông báo biên dịch lỗi – "Compilation Error" cho người dùng nếu có lỗi cú pháp.

2.2.1.2. Giai đoạn thực thi

Mã máy được nạp vào bộ nhớ và thực thi bởi CPU. Đây là giai đoạn rủi ro nhất, nơi hệ thống phải đối mặt với các hành vi không xác định của mã nguồn người dùng như vòng lặp vô hạn, truy cập bộ nhớ trái phép,… Hệ thống phải áp dụng các giới hạn tài nguyên cứng để đảm bảo chương trình luôn kết thúc, giải quyết vấn đề dừng trong thực tế bằng cách đặt một ngưỡng giới hạn thời gian [10].

2.2.2. Mô hình phân lớp của hệ sinh thái OJ

Nghiên cứu của Došilović và Mekterović (2020) trong bài báo giới thiệu Judge0 [10] đã đề xuất một mô hình kiến trúc 4 lớp, tách biệt rõ ràng trách nhiệm của từng thành phần. Đây là cơ sở lý thuyết quan trọng để thiết kế NEU OJ theo hướng microservices hiện đại thay vì nguyên khối.

2.2.2.1. Lớp nền tảng ứng dụng

- Đây là lớp giao diện người dùng và logic nghiệp vụ giáo dục.
- Nhiệm vụ của lớp này là cung cấp các chức năng như quản lý bài tập, kỳ thi và hiển thị các kết quả thi…

2.2.2.2. Lớp hệ thống thực thi mã trực tuyến

- Lớp hệ thống thực thi mã trực tuyến (OCES - Online Code Execution System) là lớp trung gian, đóng vai trò điều phối các yêu cầu thực thi mã nguồn, các máy chủ API có thể được kể đến như thành phần chính thuộc lớp này.

- Nhiệm vụ chính của lớp này là tiếp nhận yêu cầu qua API, xác thực, đưa yêu cầu vào hàng đợi và cân bằng tải tới các máy chấm.

2.2.2.3. Lớp lõi thực thi

- Lớp lõi thực thi (CEE - Code Execution Engine) là các "công nhân" (workers) thực hiện chức năng cốt lõi của hệ thống.
- Nhiệm vụ của lõi thực thi là nhận các yêu cầu từ hàng đợi, chuẩn bị môi trường, gọi đến trình biên dịch, thực thi mã nguồn và thu thập số liệu liên quan như thời gian, bộ nhớ sử dụng.

2.2.2.4. Lớp Sandbox

- Lớp sandbox hay còn gọi là "hộp cô lập" là lớp thấp nhất và quan trọng nhất về an toàn bảo mật cho máy chủ.
- Nhiệm vụ của các "hộp" này là cung cấp môi trường bị cô lập hoàn toàn với môi trường máy chủ vật lý để thực thi các đoạn mã không đáng tin cậy từ người dùng.

⇨ Sự phân rã này cho phép tối ưu hoá bảo mật và hiệu năng. Việc tách biệt OCES ra khỏi nền tảng ứng dụng giúp hệ thống có thể mở rộng theo chiều ngang bằng cách thêm nhiều các worker node mà không ảnh hưởng đến ứng dụng chính [10]. Đồng thời, nếu sandbox bị tấn công, kẻ tấn công chỉ bị giới hạn trong một "hộp kín" dùng một lần, không thể truy cập vào máy chủ vật lý thực sự hay gây bất cứ ảnh hưởng gì khác [21].

2.2.3. Kỹ thuật ảo hóa và cô lập môi trường

Sandboxing – cô lập là kỹ thuật tạo ra một môi trường kiểm soát chặt chẽ, hạn chế quyền truy cập của tiến trình đối với tài nguyên hệ thống máy chủ vật lý. Đối với một OJ, sandbox là tuyến phòng thủ quan trọng nhất.

2.2.3.1. So sánh các kỹ thuật sandboxing

Hiện nay, việc xây dựng môi trường thực thi an toàn thường được tiếp cận theo ba hướng chính: ảo hóa phần cứng, container hóa bằng Docker và cô lập trực tiếp trên hệ điều hành (isolate).

Bảng 1. So sánh các kỹ thuật sandboxing.

Đặc điểm	**Máy ảo**	**Docker containers**	**Linux Containers/ Isolate**
Kiến trúc	Ảo hoá phần cứng, chạy Guest OS riêng biệt.	Ảo hoá cấp OS, chia sẻ kernel với máy chủ, dùng Docker daemon.	Sử dụng trực tiếp tính năng Kernel, không qua Daemon.
Mức độ cô lập	Rất cao với kernel riêng biệt.	Trung bình, bởi chia sẻ kernel gây rủi ro leo thang đặc quyền.	Cao.
Thời gian khởi động	Rất chậm, bởi cần khởi động OS.	Nhanh (giây).	Rất nhanh (mili giây).
Tài nguyên sử dụng	Cao.	Thấp.	Rất thấp.
Đánh giá cho OJ	Không phù hợp bởi số lượng yêu cầu thực thi lớn, tốn thời gian và không đủ tài nguyên.	Khả thi nhưng vẫn sẽ gặp tình trạng thiếu tài nguyên phần cứng khi số lượng yêu cầu tăng.	Tối ưu nhất.

2.2.3.2. Cơ chế hoạt động của isolate

Isolate là một công cụ sandbox chuyên dụng được phát triển bởi Martin Mareš và Bernard Blackham, được sử dụng trong CMS cho kỳ thi Olympic tin học quốc tế [22]. Isolate không sử dụng Docker engine mà tương tác trực tiếp với Linux kernel để tạo ra các sandbox siêu nhẹ thông qua hai cơ chế chính:

- Linux Namespaces (Không gian tên): Cơ chế này cung cấp sự cô lập về mặt "tầm nhìn" cho tiến trình.
    - PID Namespace: Tiến trình trong sandbox thấy mình có PID = 1,

không thể nhìn thấy hay tác động đến các tiến trình khác của hệ thống.

- Mount Namespace: Tạo ra một hệ thống tệp riêng biệt. Isolate thường mount thư mục gốc ở chế độ chỉ đọc và chỉ cho phép ghi vào thư mục tạm thời, ngăn chặn việc xoá hay thay đổi file hệ thống.
- Network Namespace: Cô lập ngăn xếp mạng. Trong các kỳ thi, kết nối mạng thường bị vô hiệu hoá hoàn toàn để ngăn chặn gian lận hoặc tấn công ra bên ngoài.

- Control Groups (cgroups): Cơ chế này quản lý và giới hạn tài nguyên.
  - Memory Cgroup: Đặt giới hạn cứng cho RAM. Nếu tiến trình vượt quá, Kernel sẽ kích hoạt Out-Of-Memory Killer để dừng tiến trình ngay lập tức.
  - CPU Cgroup: Giới hạn thời gian sử dụng CPU và gán tiến trình vào các lõi cụ thể để đảm bảo kết quả đo đạc thời gian là chính xác và nhất quán, tránh bị ảnh hưởng bởi các tác vụ nền khác của hệ thống.
  - PIDs Cgroups: Giới hạn số lượng tiến trình con tối đa được tạo ra.

2.2.3.3. Các nguy cơ bảo mật và phương án phòng chống

- Fork Bomb: Một chương trình đệ quy tự nhân bảo, có thể làm cạn kiệt bảng tiến trình của hệ điều hành, gây treo máy chủ. Isolate ngăn chặn điều này bằng PIDs Cgroups bằng cách đặt ra số lượng tiến trình tối đa cho sandbox.
- Vòng lặp vô hạn: Gây tiêu tốn CPU. Isolate sử dụng bộ đếm thời gian của hệ điều hành để gửi tín hiệu SIGKILL khi hết thời gian cho phép.
- Truy cập trái phép: Đọc hoặc gây tác động lên các tệp tin hệ thống nhạy cảm. Cơ chế Mount Namespace ngăn chặn điều này bằng cách "giam lỏng" tiến trình trong một thư mục giả lập, không thể truy cập ra ngoài.

2.3. Tổng quan và đánh giá các hệ thống hiện có

2.3.1. Các hệ thống thương mại và thi đấu

Các nền tảng như Codeforces, LeetCode và HackerRank đại diện cho chuẩn mực hiện tại của OJ.

Điểm mạnh của các hệ thống kể trên hoặc tương tự là khả năng chịu tải khổng lồ lên tới hàng chục nghìn người thi đồng thời, kho bài tập phong phú và

trải nghiệm người dùng xuất sắc. Hệ thống xếp hạng và cộng động mạnh cũng là những yếu tố thu hút không thể bỏ qua.

Đặt trong bối cảnh giáo dục đại học, những hệ thống này có một vài điểm không phù hợp có thể kể đến như:

- Mã nguồn đóng: Nhà trường không thể tuỳ biến hệ thống hoặc kiểm soát dữ liệu sinh viên.
- Chi phí: Các giải pháp dành cho doanh nghiệp, trường học có chi phí rất cao.
- Thiếu tính năng quản lý đào tạo: Các hệ thống này tập trung vào thi đấu và tuyển dụng. Chúng thiếu đi các tính năng quản lý lớp học, tích hợp danh sách sinh viên từ hệ thống đào tạo của trường, chấm điểm quá trình hay phát hiện sao chép trong nội bộ lớp học.
- Hạn chế trong kiểm soát gian lận: Mặc dù các nền tảng này có thuật toán phát hiện sao chép mã nguồn mạnh mẽ dựa trên cấu trúc logic, nhưng các tính năng giám sát thi cử nghiêm ngặt như: khóa trình duyệt, theo dõi chuyển tab, hay giám sát webcam thường chỉ có ở các gói doanh nghiệp với chi phí cao kể trên. Các phiên bản miễn phí hoặc cộng đồng không thể ngăn chặn sinh viên sử dụng tài liệu, AI, hoặc công cụ hỗ trợ thứ 3 trong quá trình làm bài thi thời gian thực.

2.3.2. Các hệ thống OJ mã nguồn mở khác

Hai hệ thống mã nguồn mở nổi bật nhất là CMS sử dụng cho Olympic tin học quốc tế và DOMjudge sử dụng cho kỳ thi ICPC.

Với CMS, ưu điểm của hệ thống này là rất mạnh mẽ, ổn định và bảo mật cao. Đi cùng với đó là những điểm yếu như kiến trúc nguyên khối và phức tạp. CMS được thiết kế như một giải pháp "trọn gói" với giao diện và cơ sở dữ liệu riêng, rất khó để có thể tách riêng phần engine chấm bài ra để tích hợp hoặc mở rộng. Đồng tời việc cài đặt và cấu hình đòi hỏi nhiều công sức quản trị hệ thống.

Với DOMjudge, ổn định, giao diện quản lý và độ phổ biến trong cộng đồng thi ICPC là những điểm vượt trội của hệ thống này. Nhưng cũng tương tự như CMS, hệ thống này cũng theo hướng nguyên khối với việc được xây dựng bằng công nghệ cũ là PHP/Symfony. Mặc dù phiên bản mới đây đã hỗ trợ API, nhưng việc tuỳ biến sâu logic chấm hoặc giao diện người dùng tách biệt gặp nhiều hạn chế do sự ràng buộc chặt chẽ với các thành phần [23].

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Phân tích hệ thống

3.1.1. Phân tích chức năng

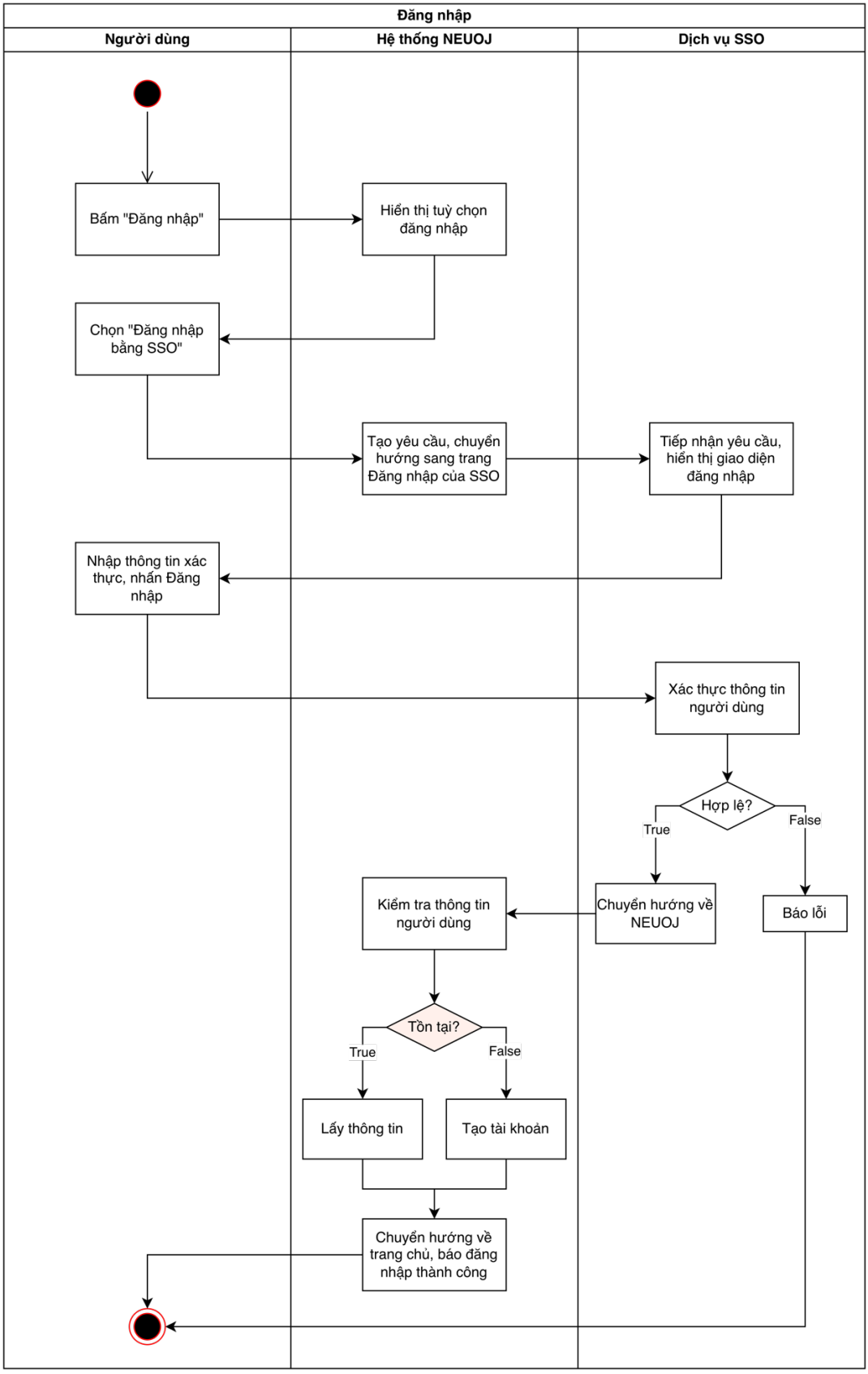
3.1.1.1. Biểu đồ Use Case tổng quát hệ thống

Trong giai đoạn thử nghiệm, hệ thống sẽ tạm thời không triển khai đăng ký cho người dùng vãng lai mà chỉ cho phép xác thực qua nền tảng SSO của đại học.

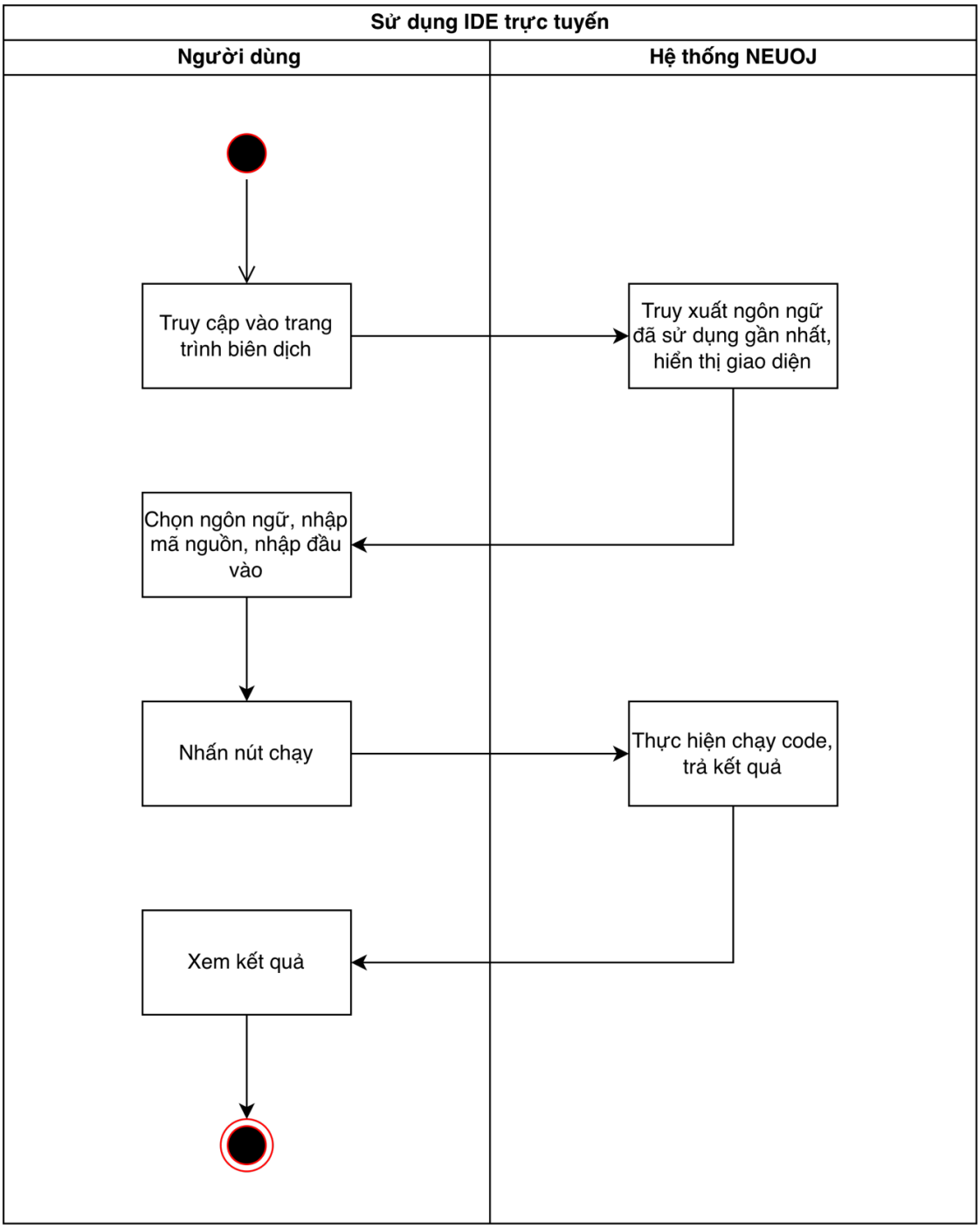


Hình 3.1. Biểu đồ ca sử dụng tổng quát hệ thống NEU OJ

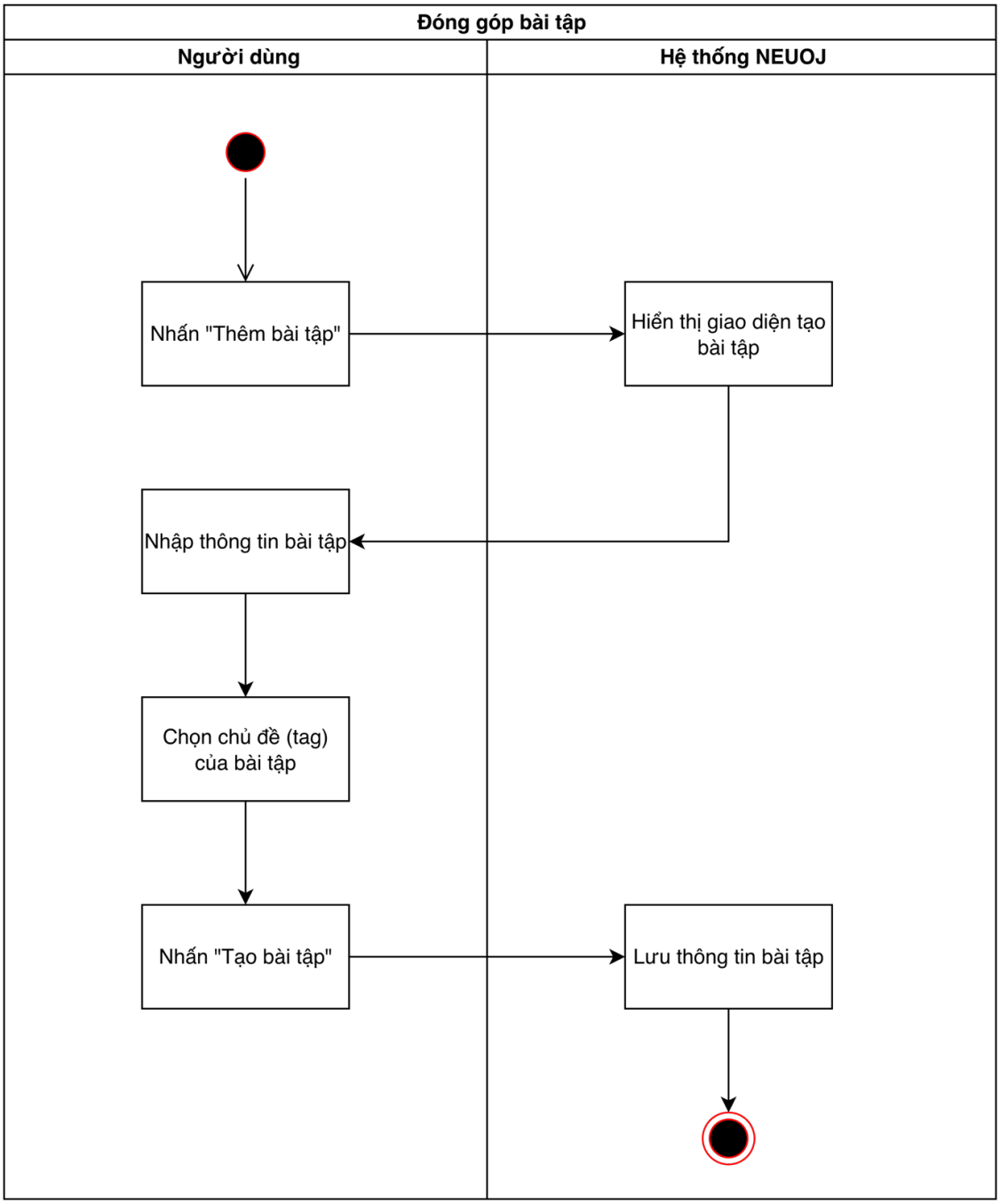
3.1.1.2. Biểu đồ hoạt động



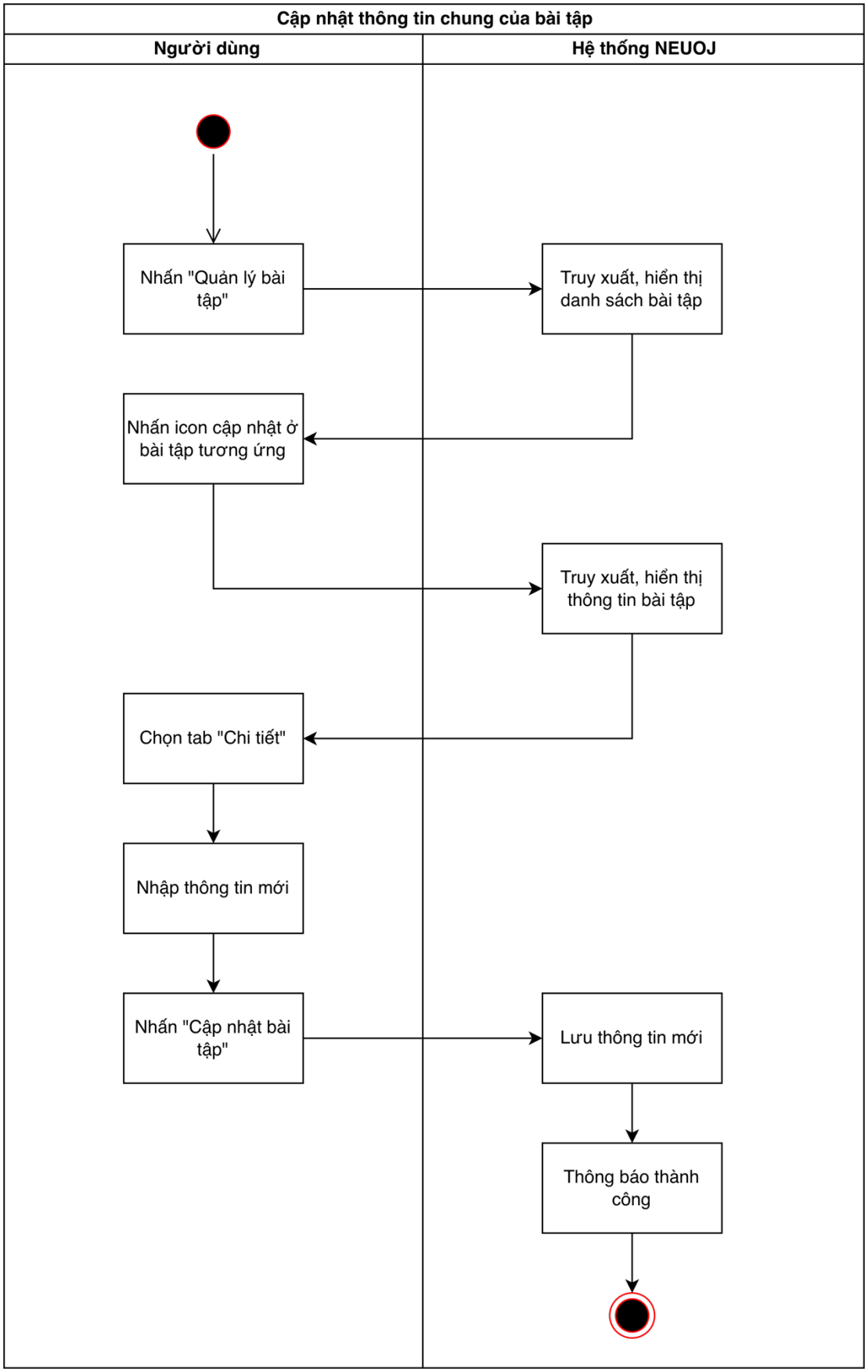
Hình 3.2. Biểu đồ hoạt động chức năng đăng nhập



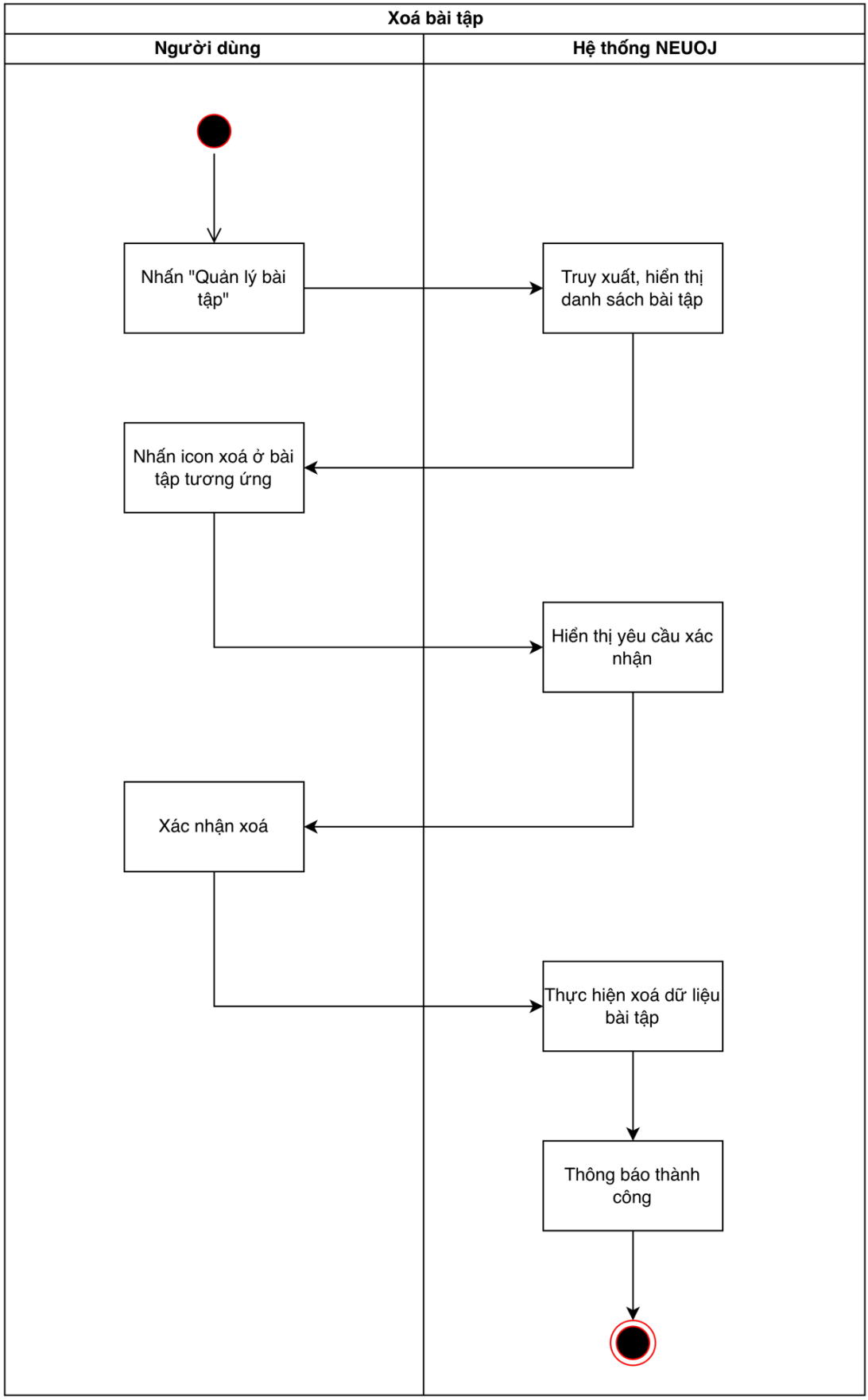
Hình 3.3. Biểu đồ hoạt động chức năng sử dụng IDE trực tuyến



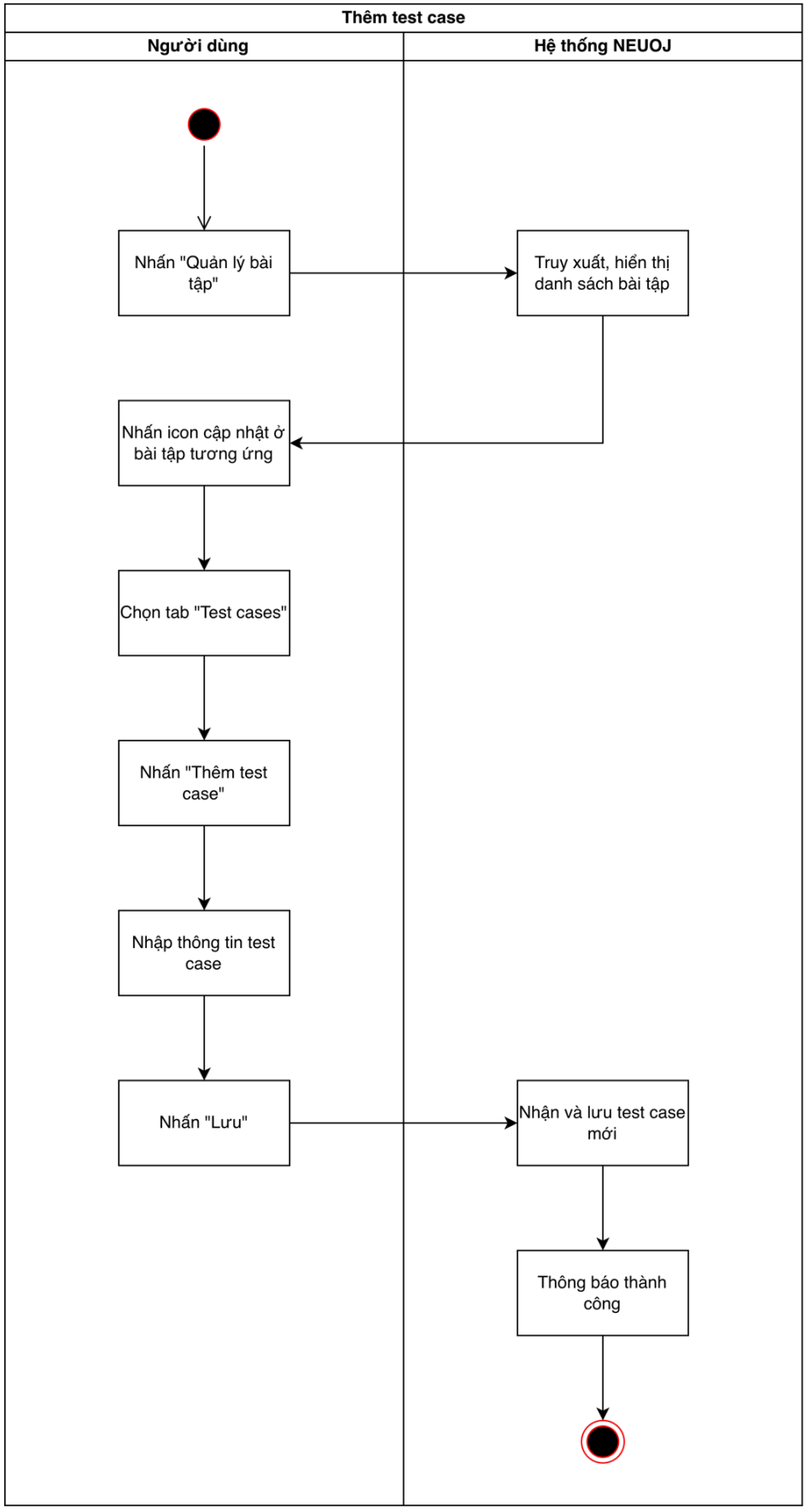
Hình 3.4. Biểu đồ hoạt động chức năng đóng góp bài tập.



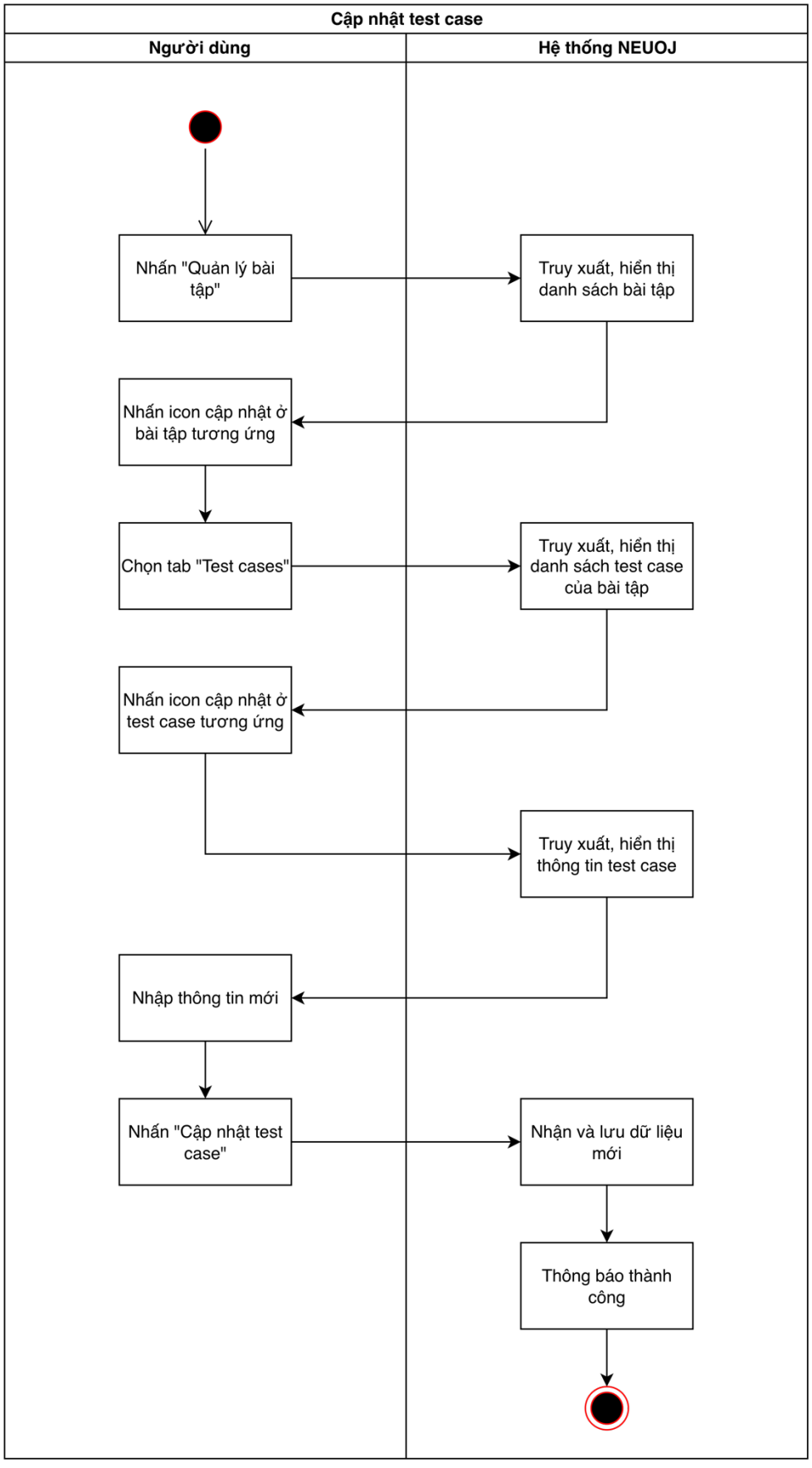
Hình 3.5. Biểu đồ hoạt động chức năng cập nhật thông tin chung của bài tập.



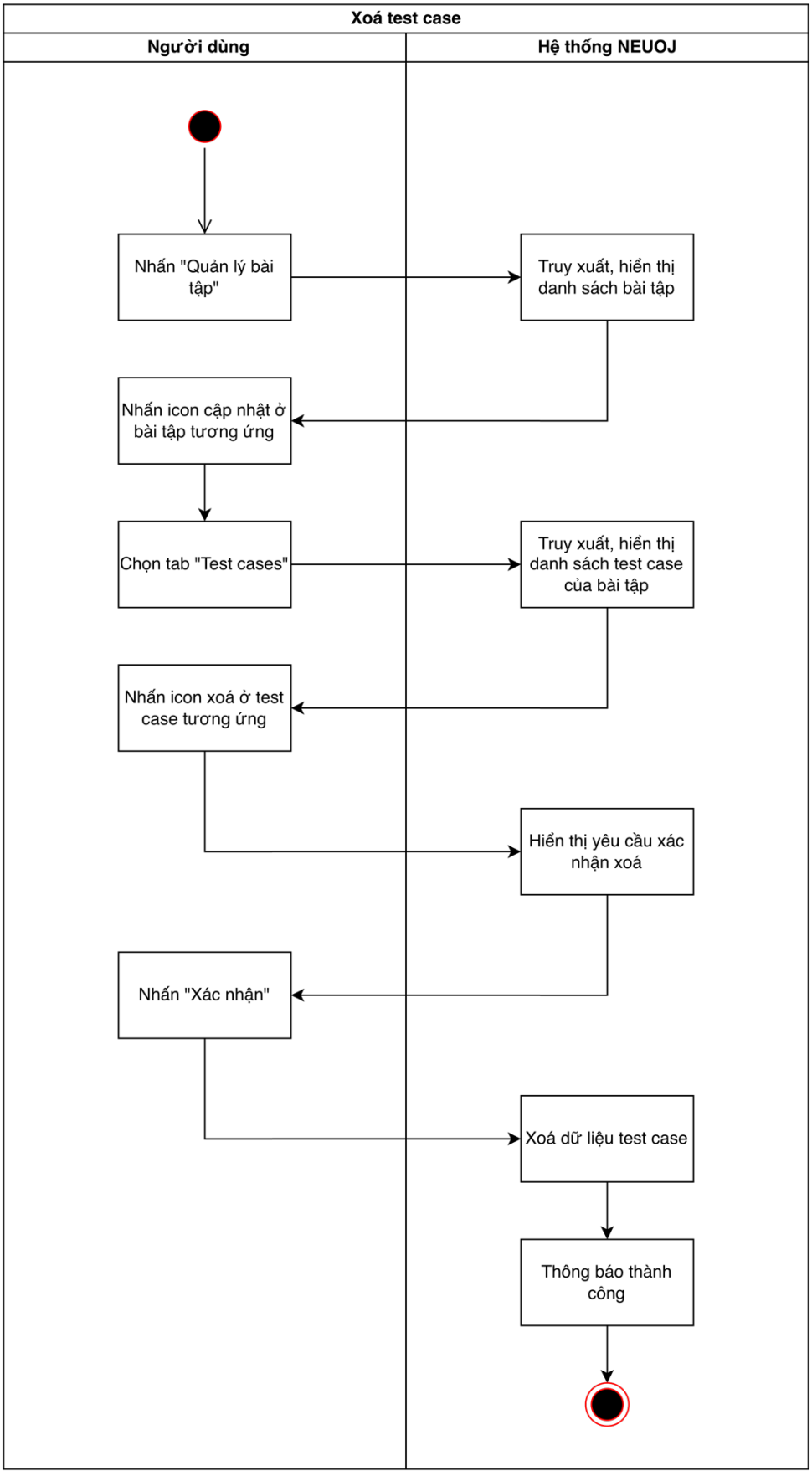
Hình 3.6. Biểu đồ hoạt động chức năng xoá bài tập.



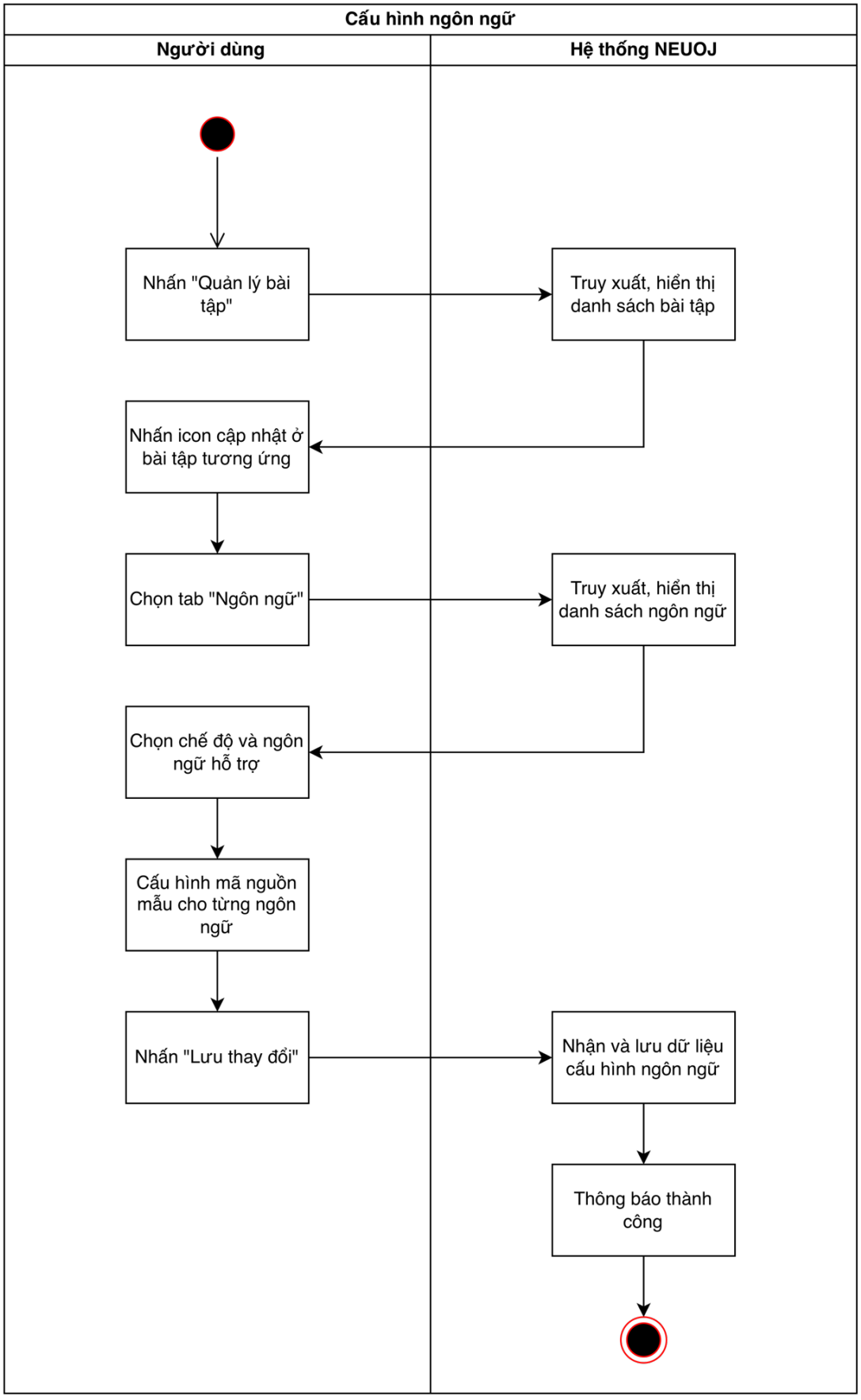
Hình 3.7. Biểu đồ hoạt động chức năng thêm test case.



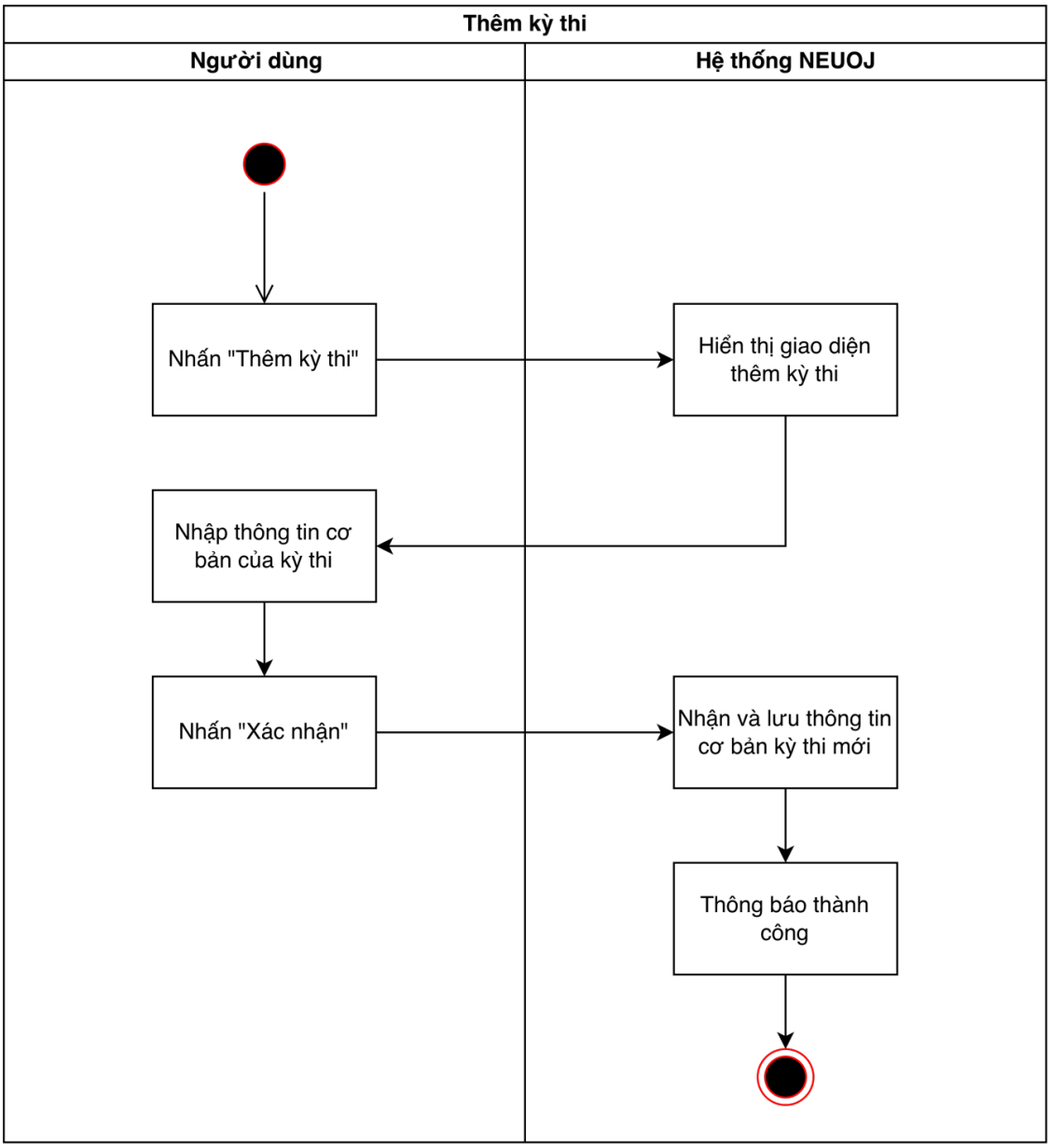
Hình 3.8. Biểu đồ hoạt động chức năng cập nhật test case.



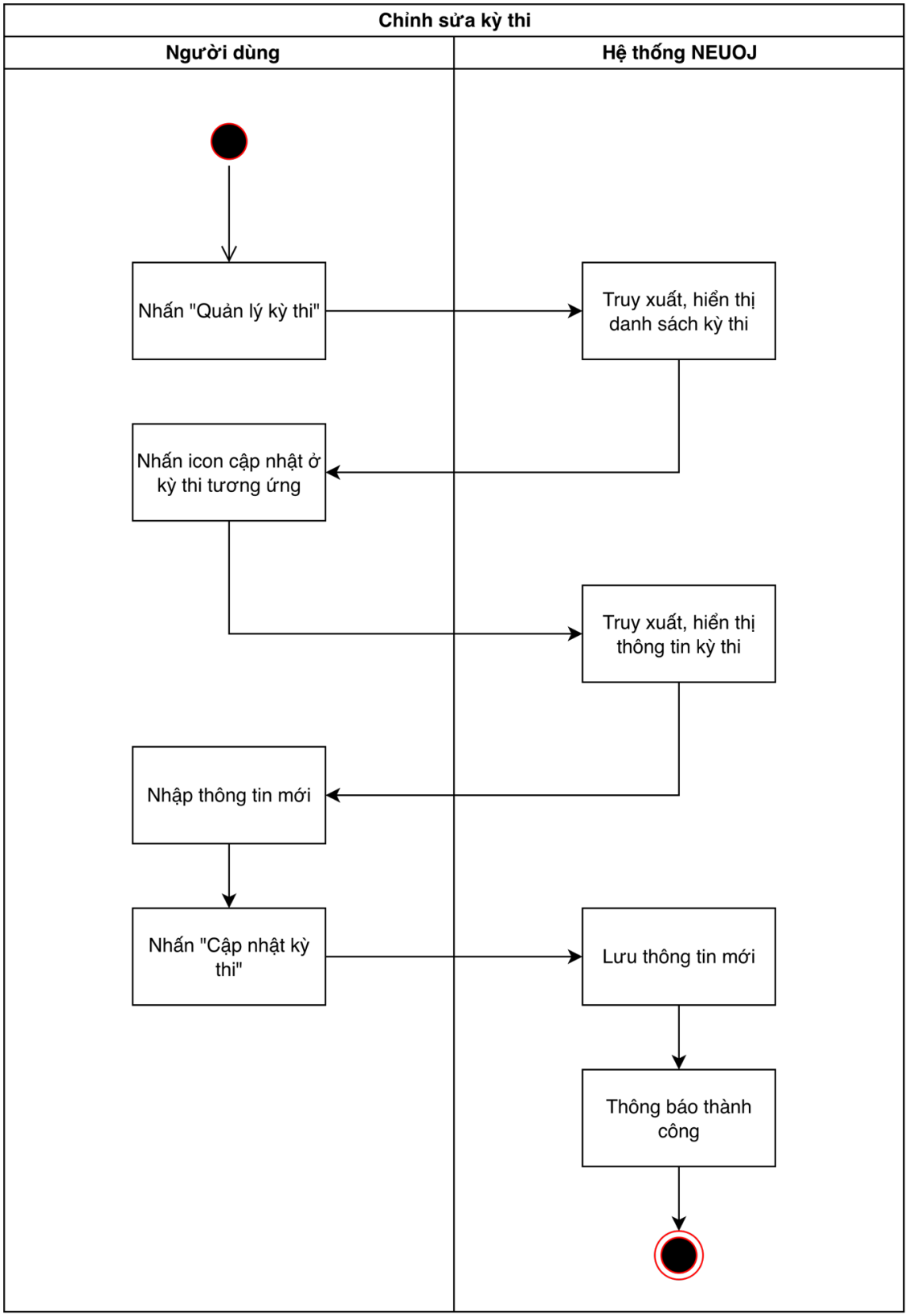
Hình 3.9. Biểu đồ hoạt động chức năng xoá test case.



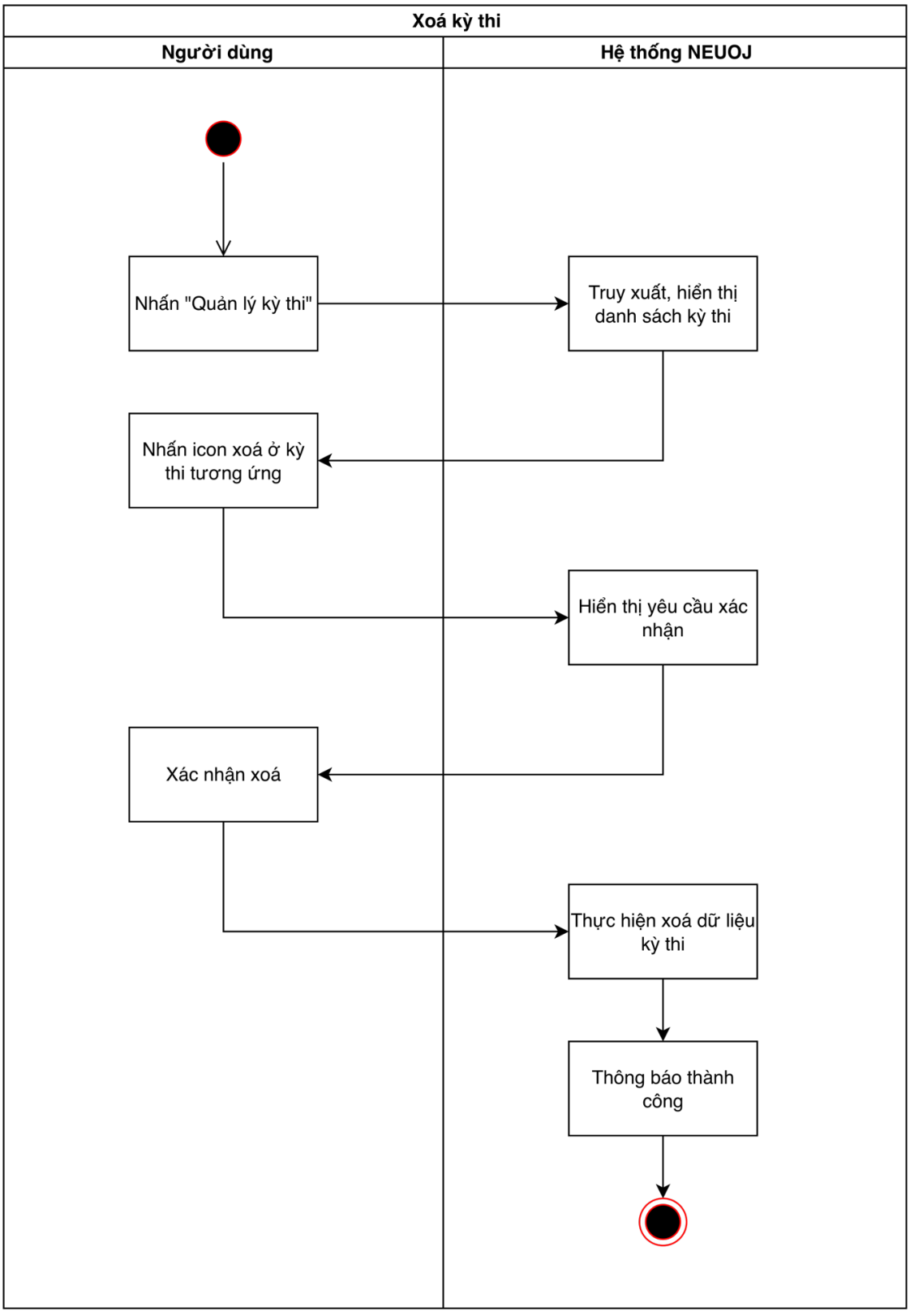
Hình 3.10. Biểu đồ hoạt động chức năng cấu hình ngôn ngữ cho bài tập.



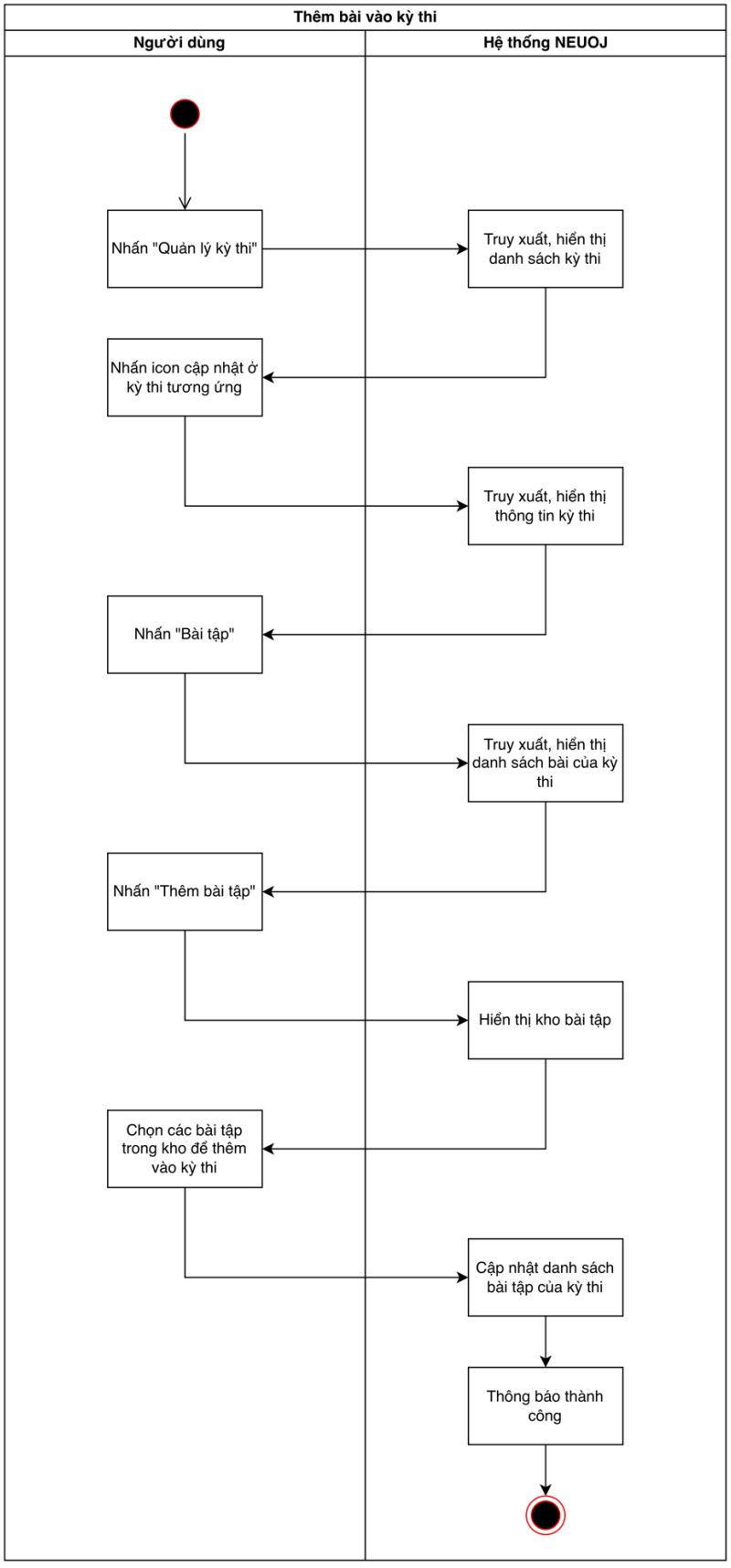
Hình 3.11. Biểu đồ hoạt động chức năng thêm kỳ thi.



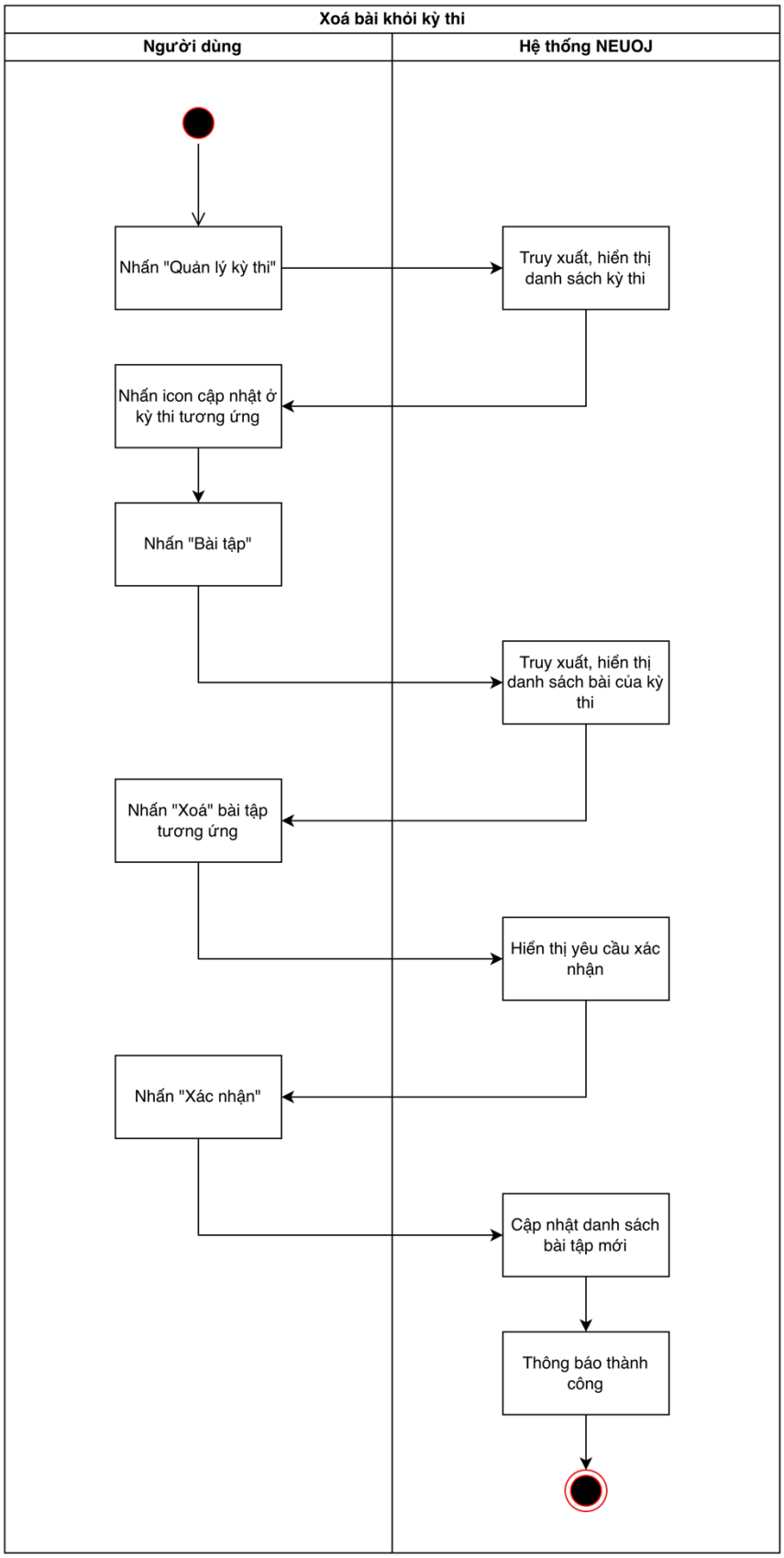
Hình 3.12. Biểu đồ hoạt động chức năng cập nhật thông tin chung của kỳ thi.



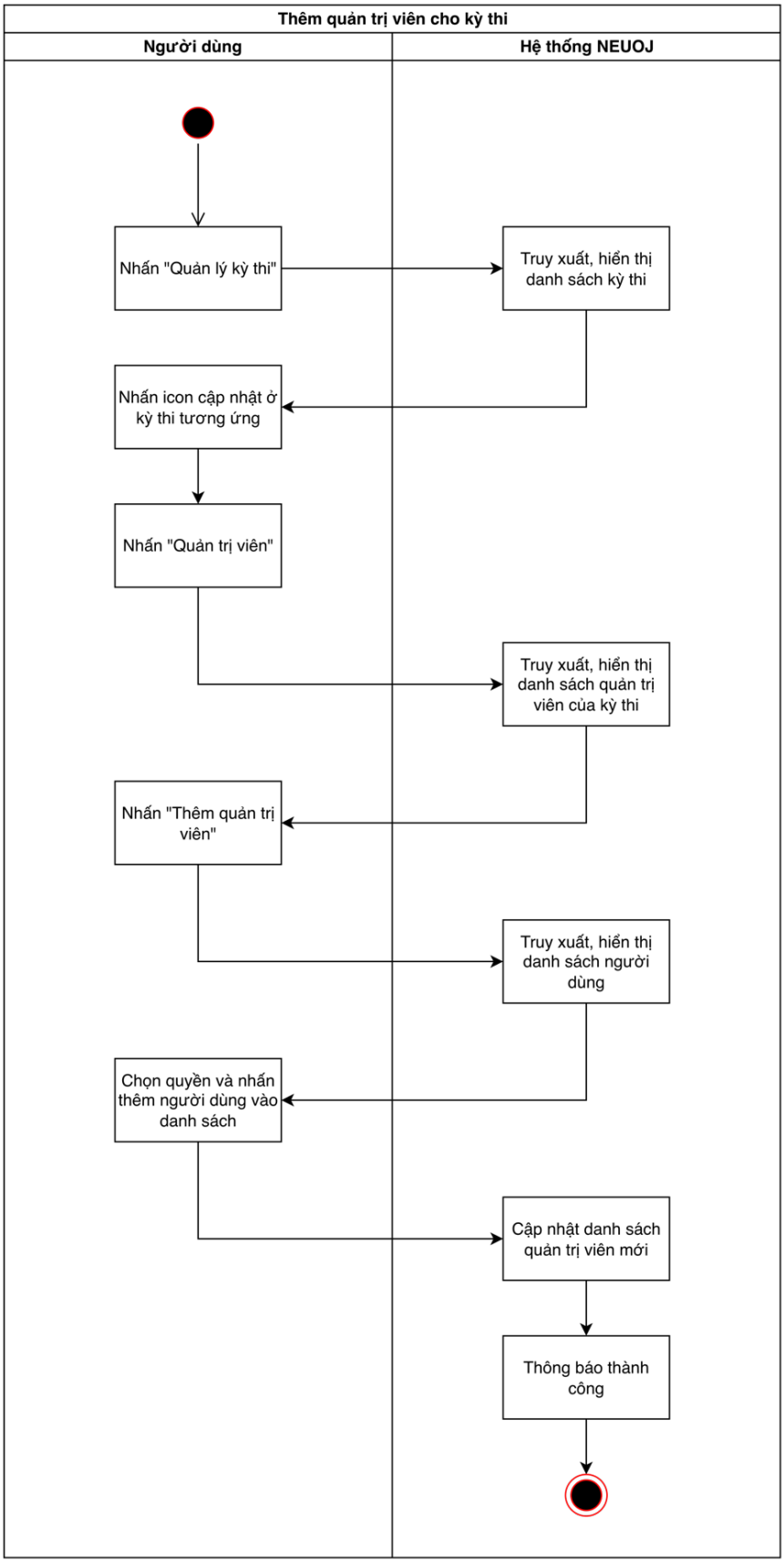
Hình 3.13. Biểu đồ hoạt động chức năng xoá kỳ thi.



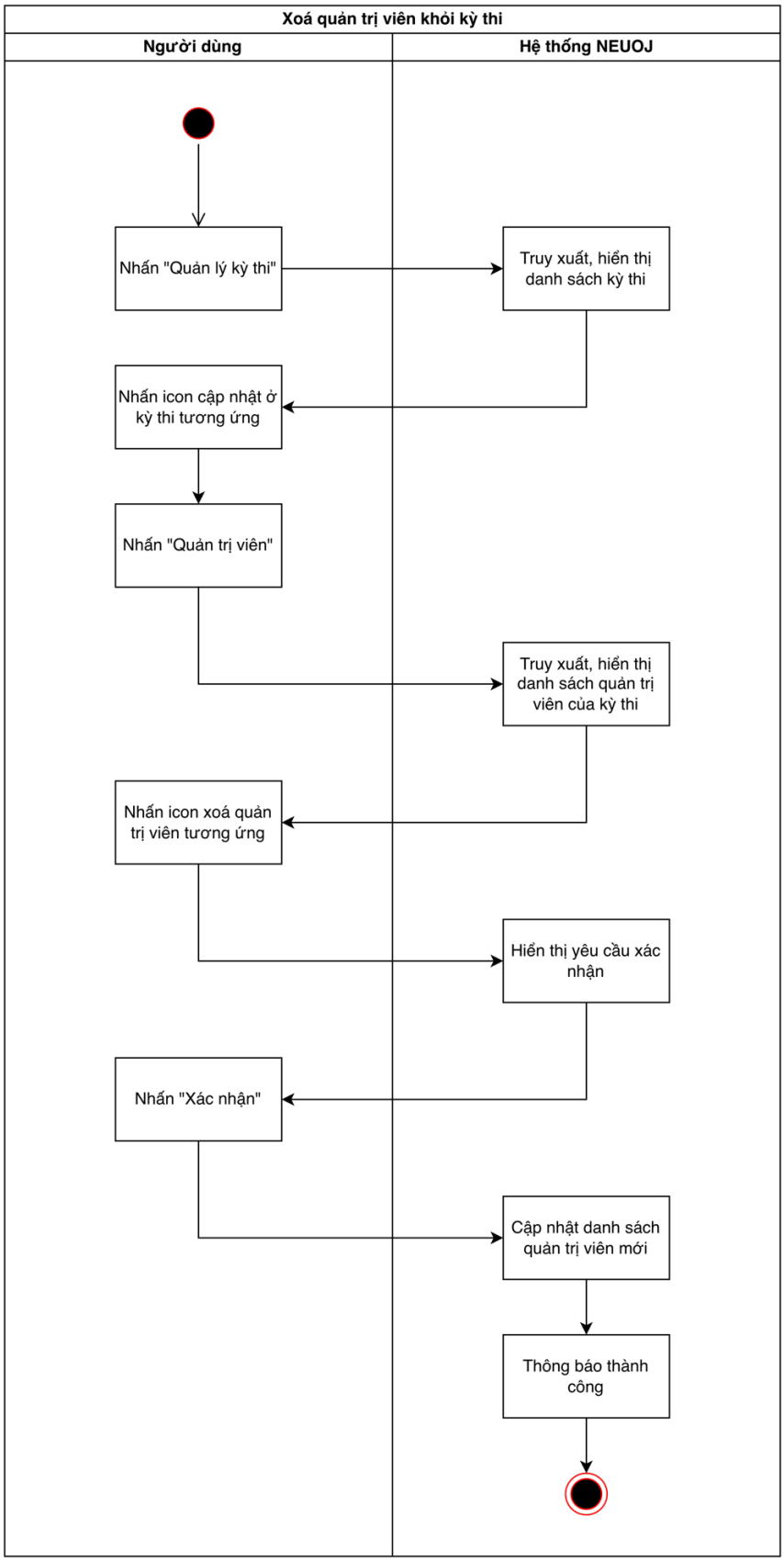
Hình 3.14. Biểu đồ hoạt động chức năng tạo đề cho kỳ thi.



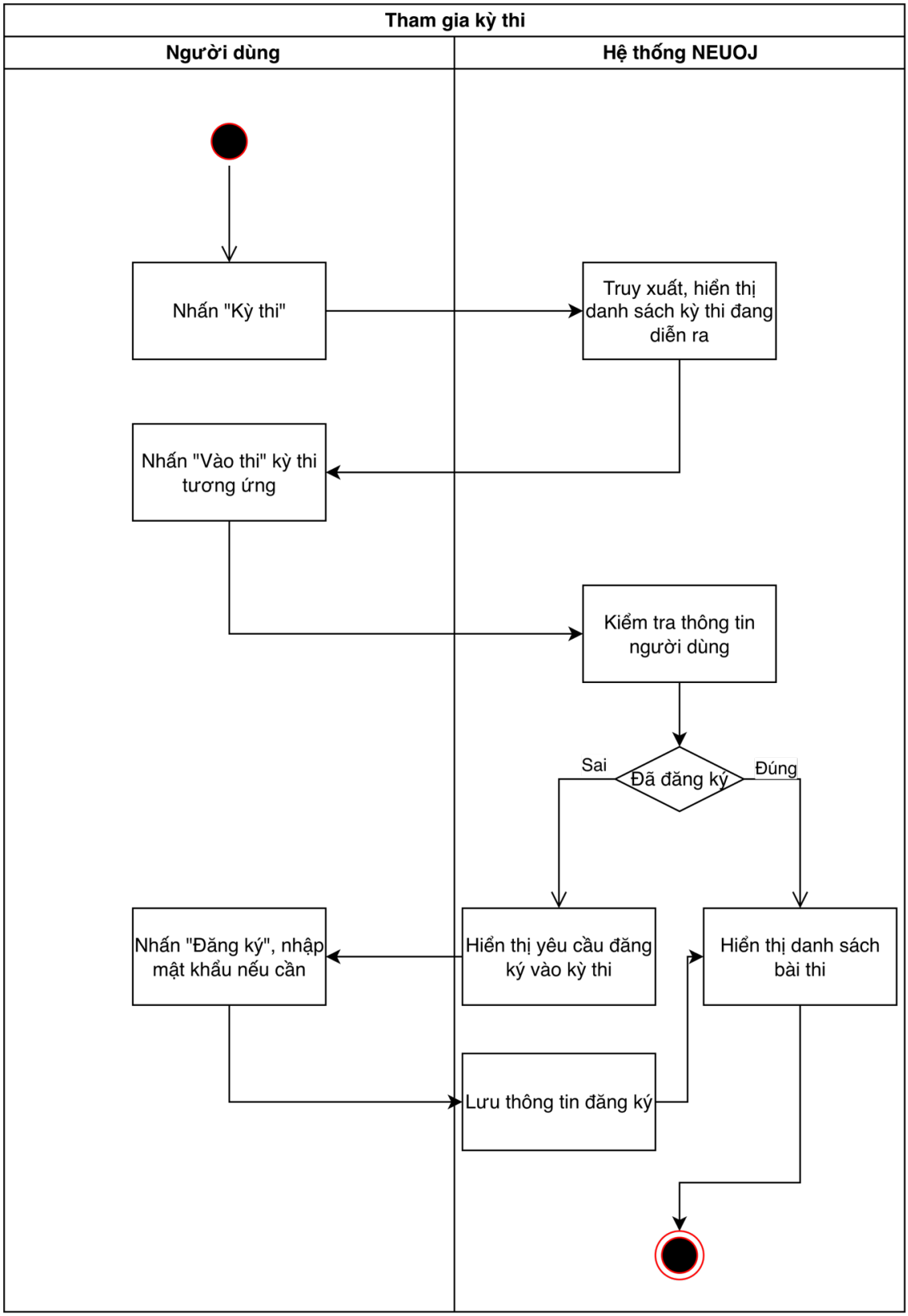
Hình 3.15. Biểu đồ hoạt động chức năng xoá bài tập khỏi kỳ thi.



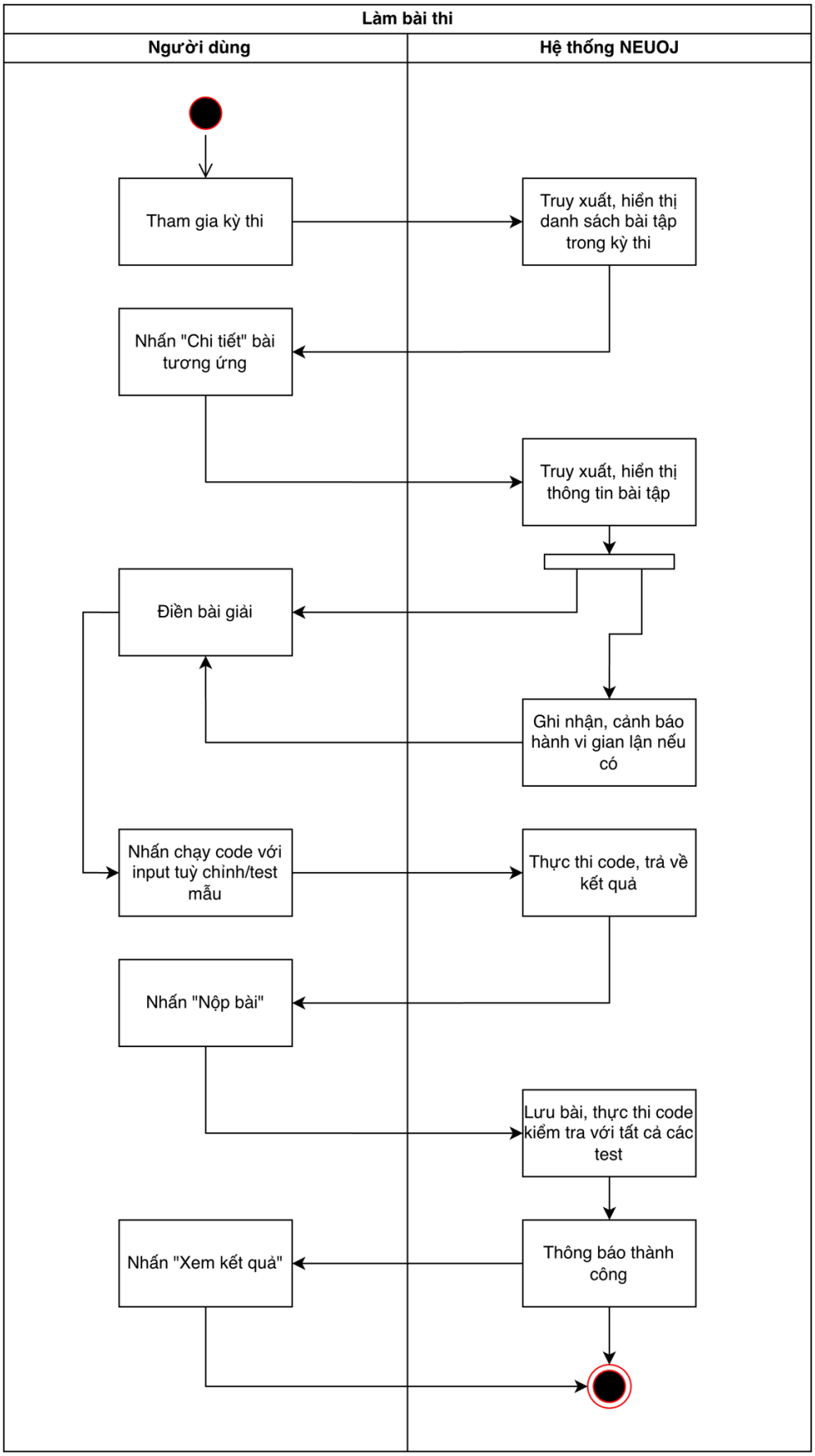
Hình 3.16. Biểu đồ hoạt động chức năng thêm quản trị viên cho kỳ thi.



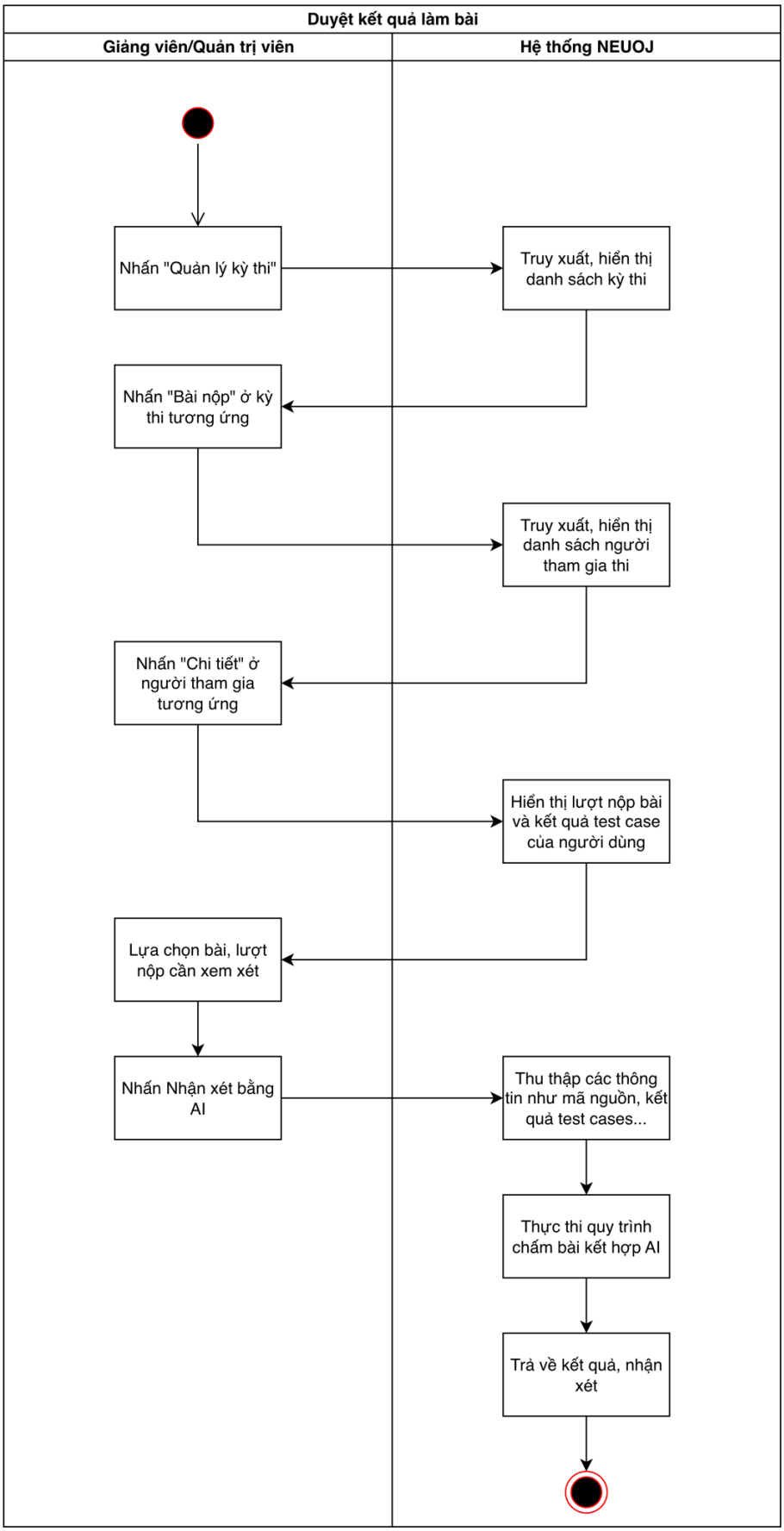
Hình 3.17. Biểu đồ hoạt động chức năng xoá quản trị viên khỏi kỳ thi.



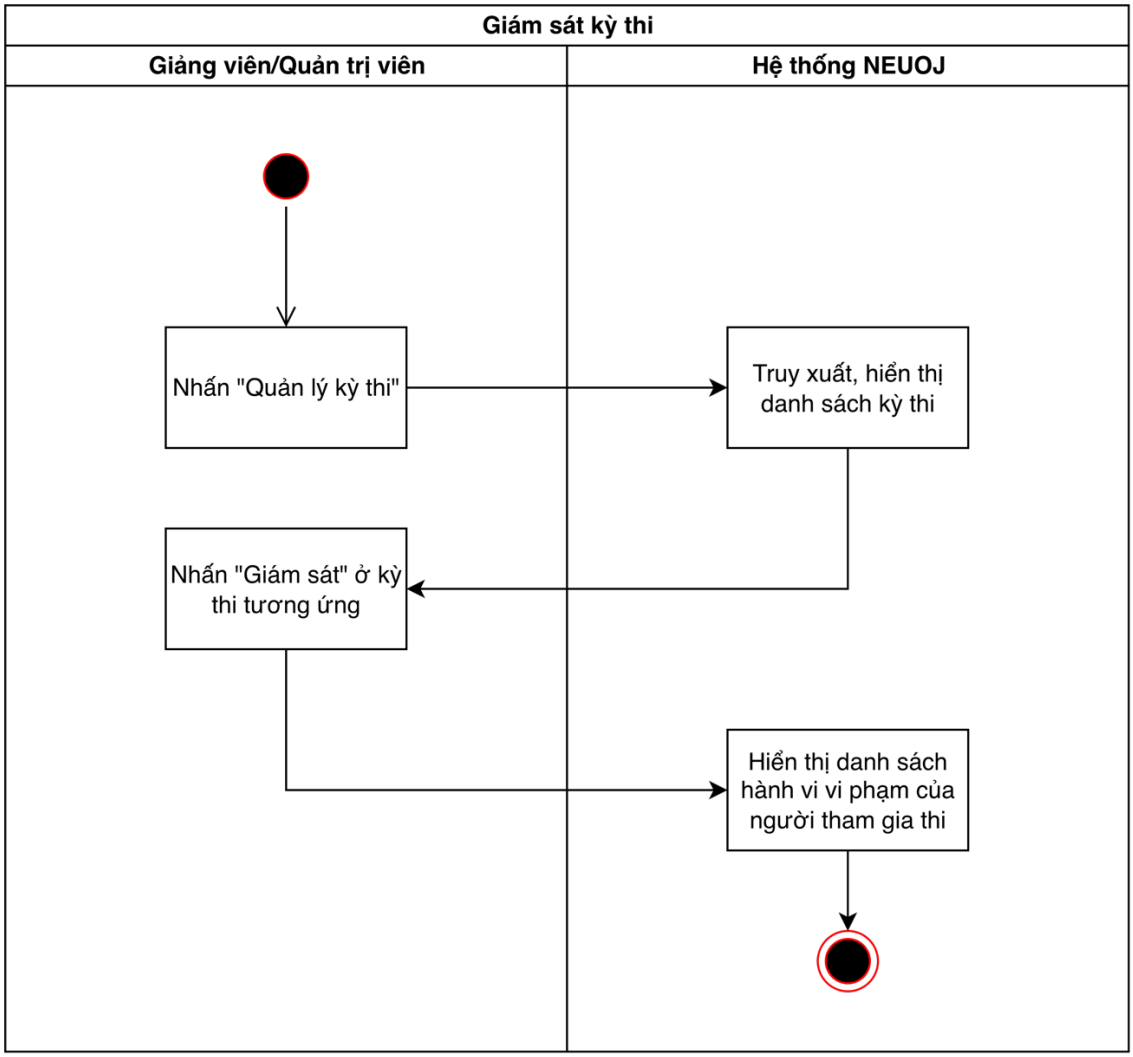
Hình 3.18. Biểu đồ hoạt động chức năng tham gia kỳ thi.



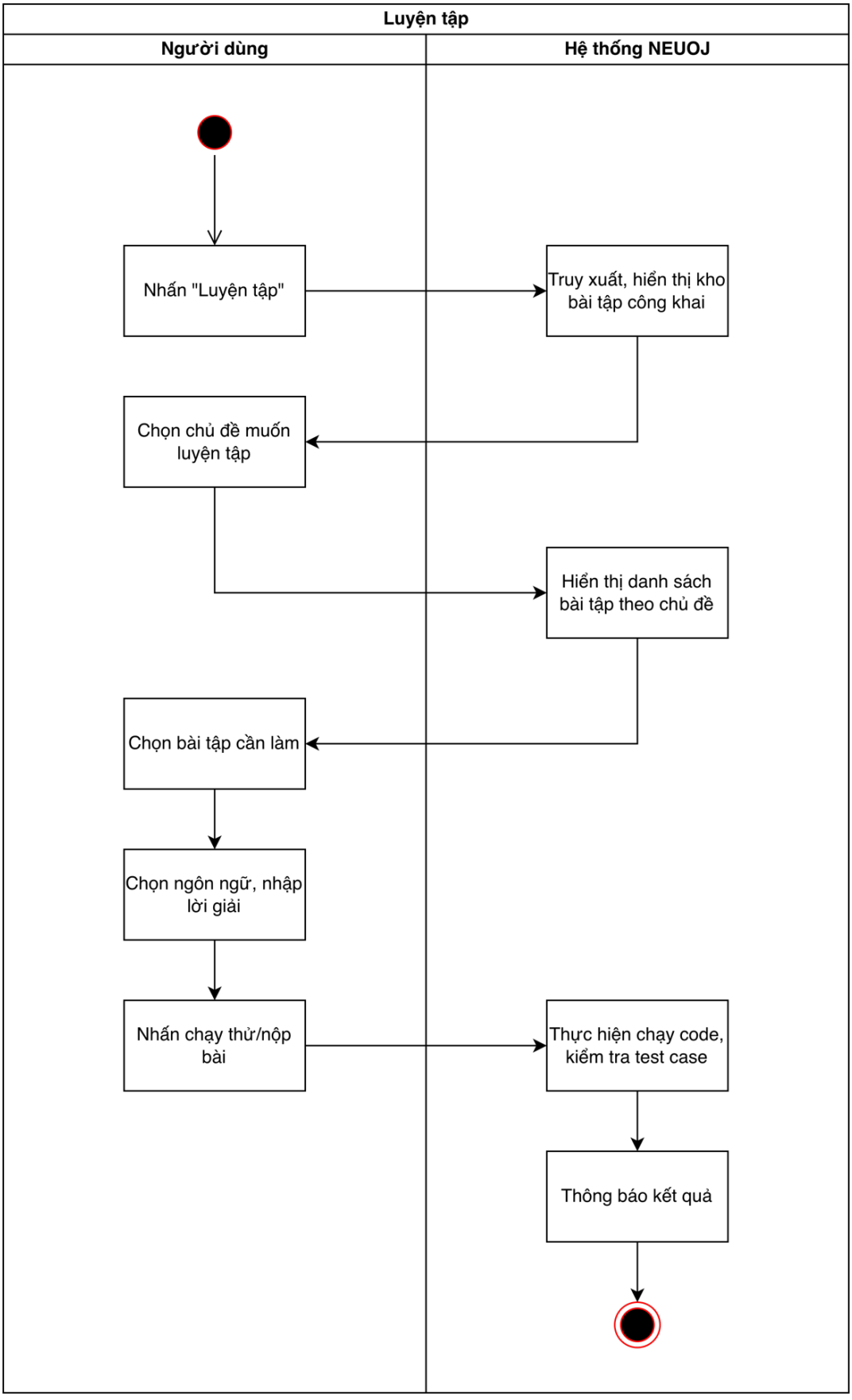
Hình 3.19. Biểu đồ hoạt động chức năng làm bài thi.



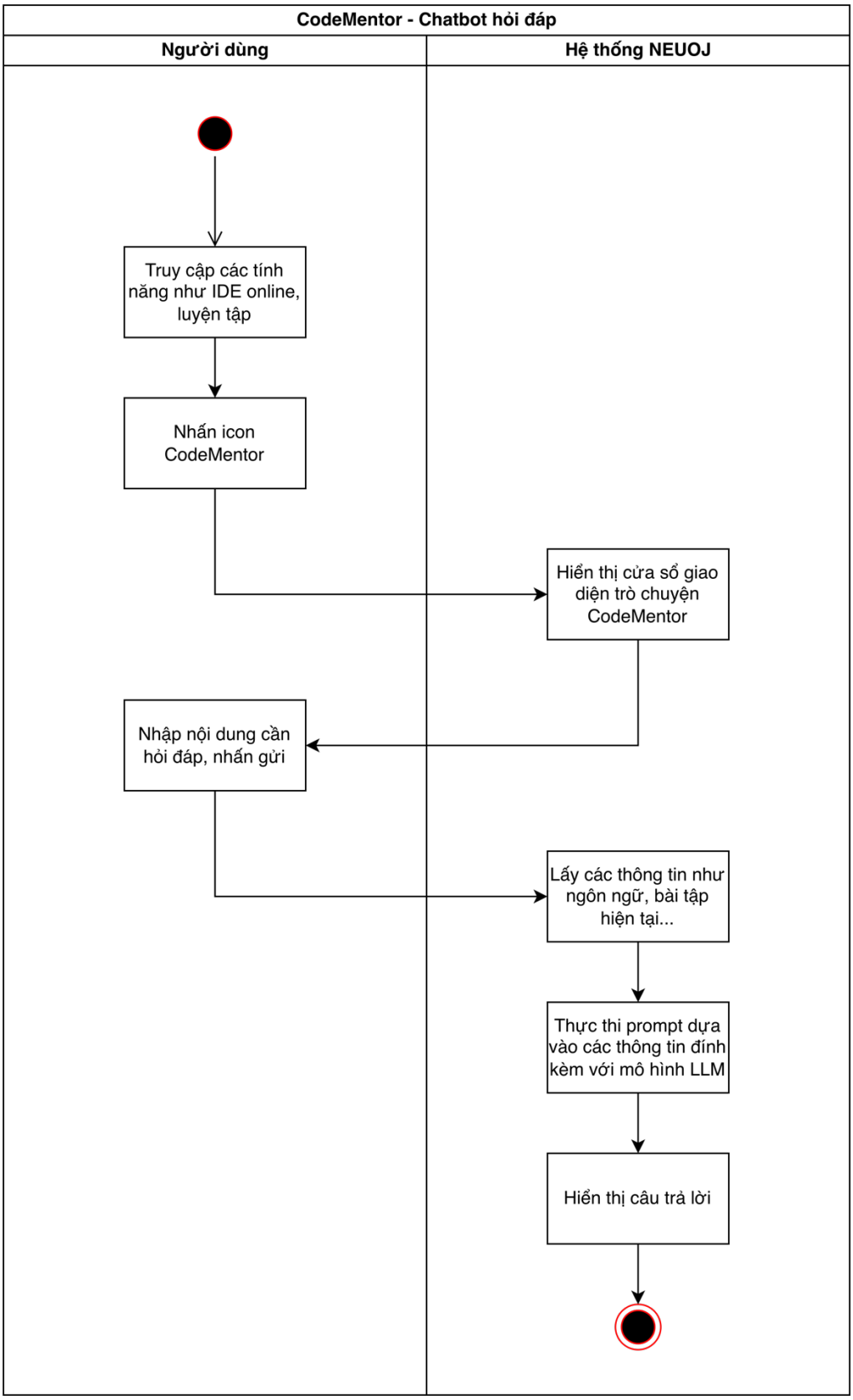
Hình 3.20. Biểu đồ hoạt động chức năng duyệt kết quả làm bài.



Hình 3.21. Biểu đồ hoạt động chức năng giám sát kỳ thi.

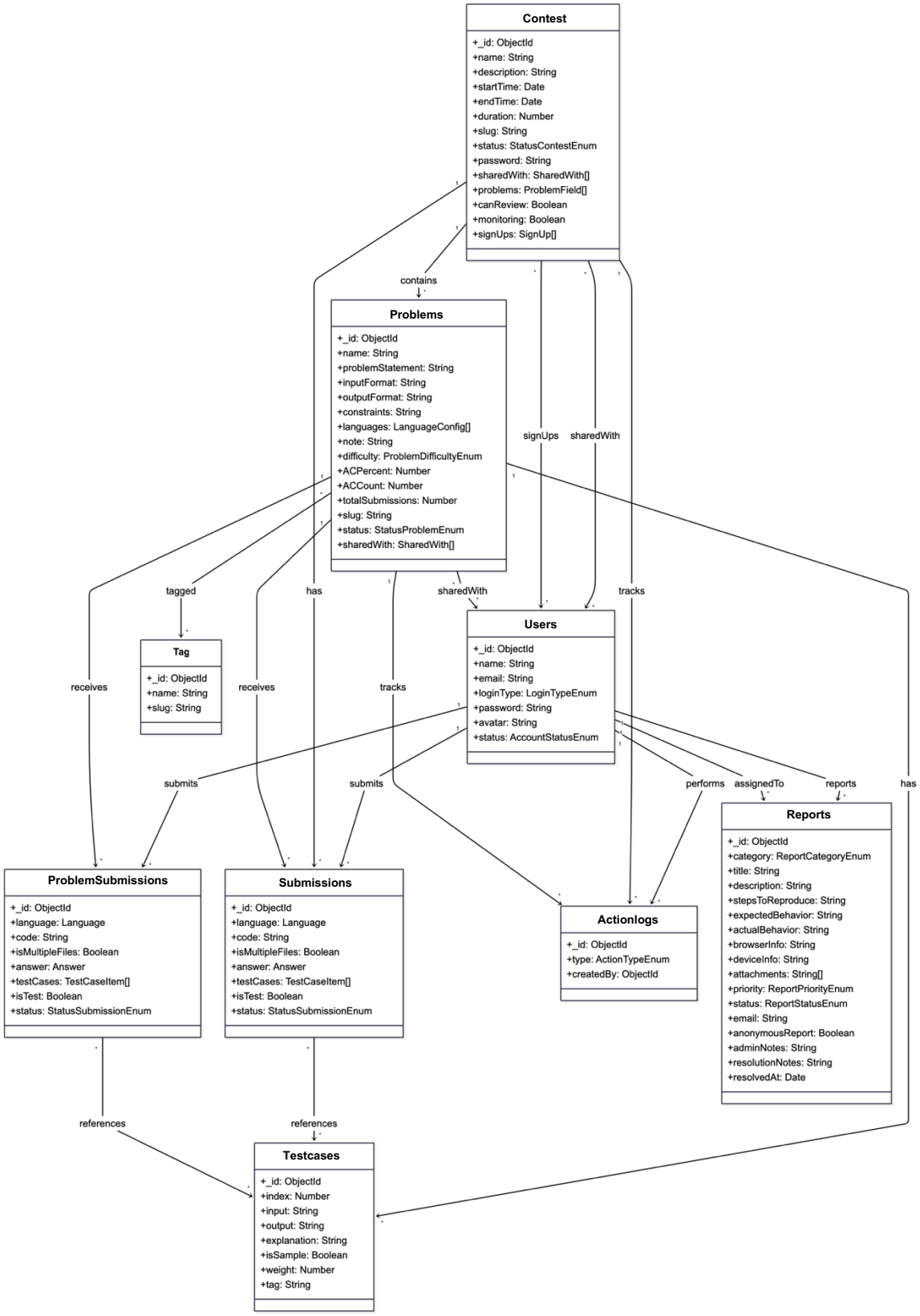


Hình 3.22. Biểu đồ hoạt động chức năng luyện tập.



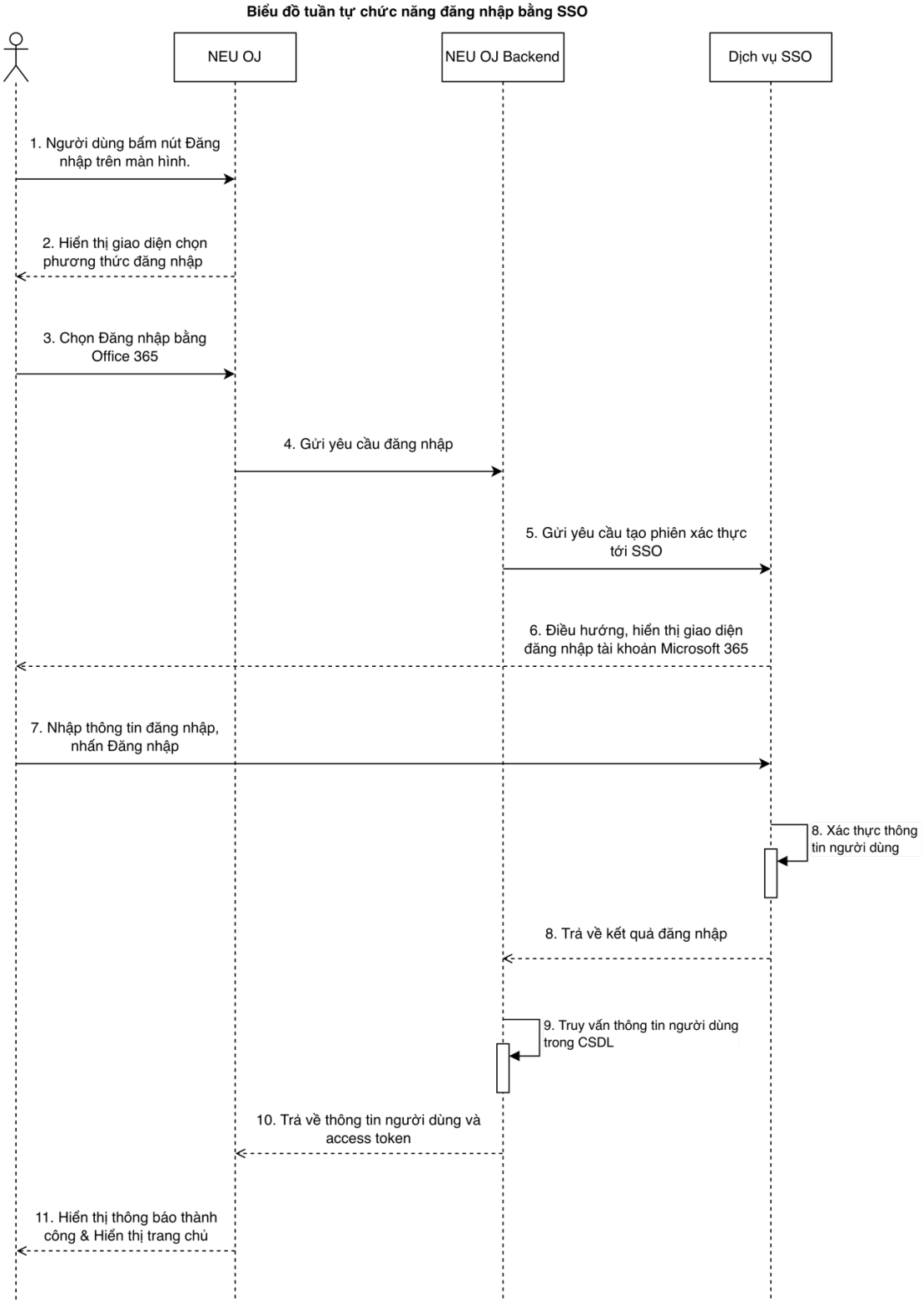
Hình 3.23. Biểu đồ hoạt động chức năng CodeMentor - Chatbot AI hỏi đáp.

3.1.2. Phân tích cấu trúc

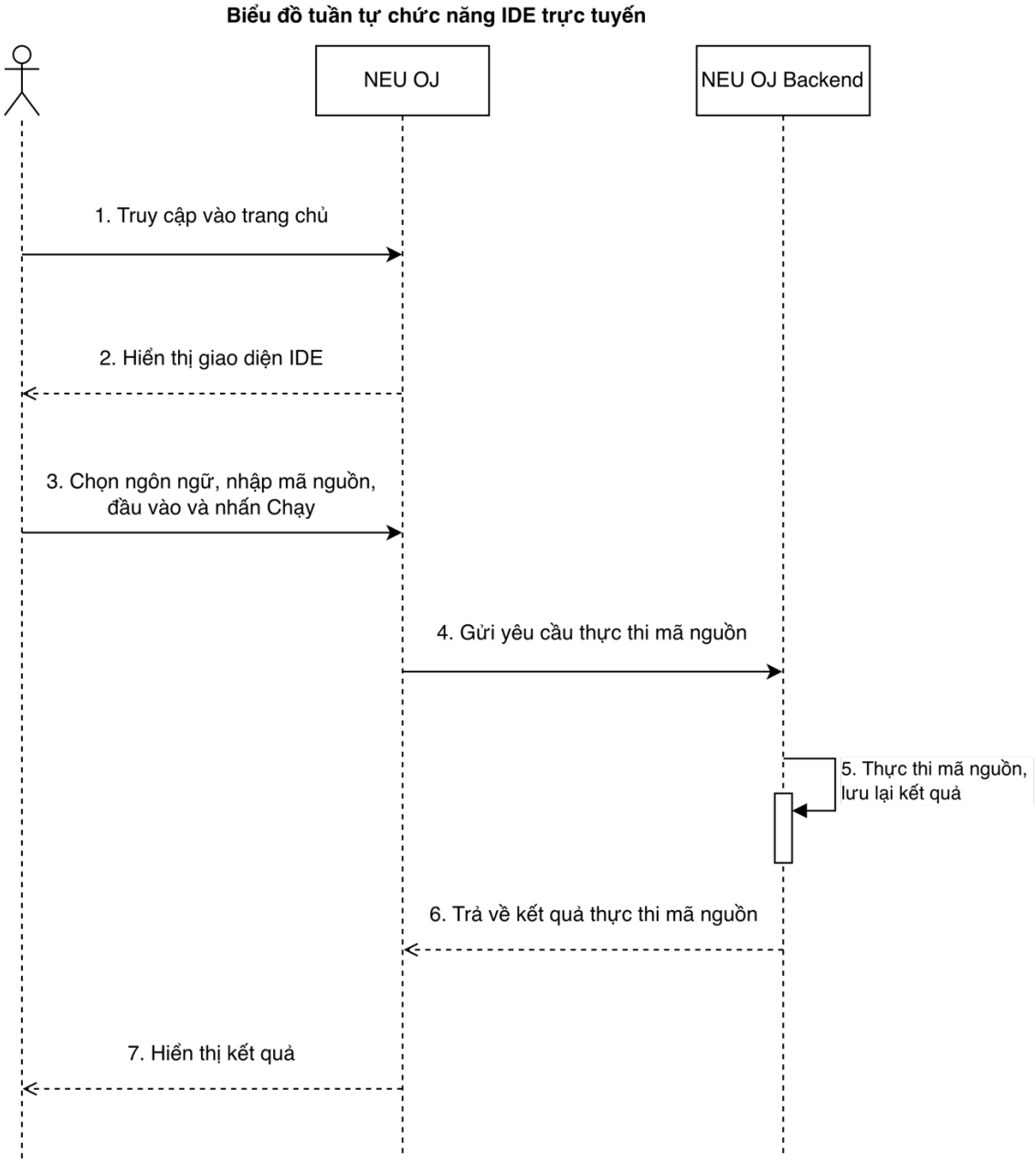


Hình 3.24. Biểu đồ lớp hệ thống NEU OJ

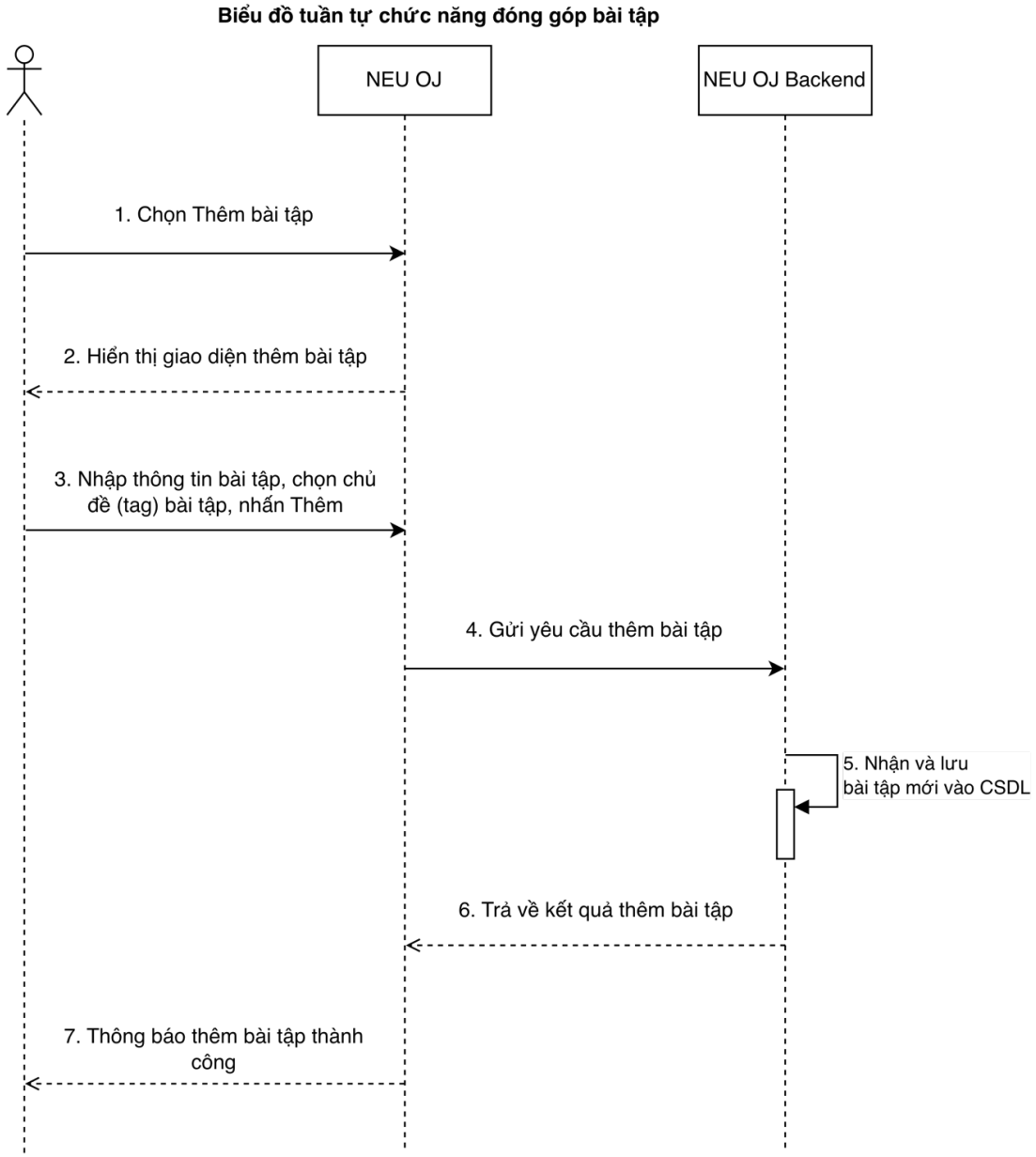
3.1.3. Phân tích hành vi



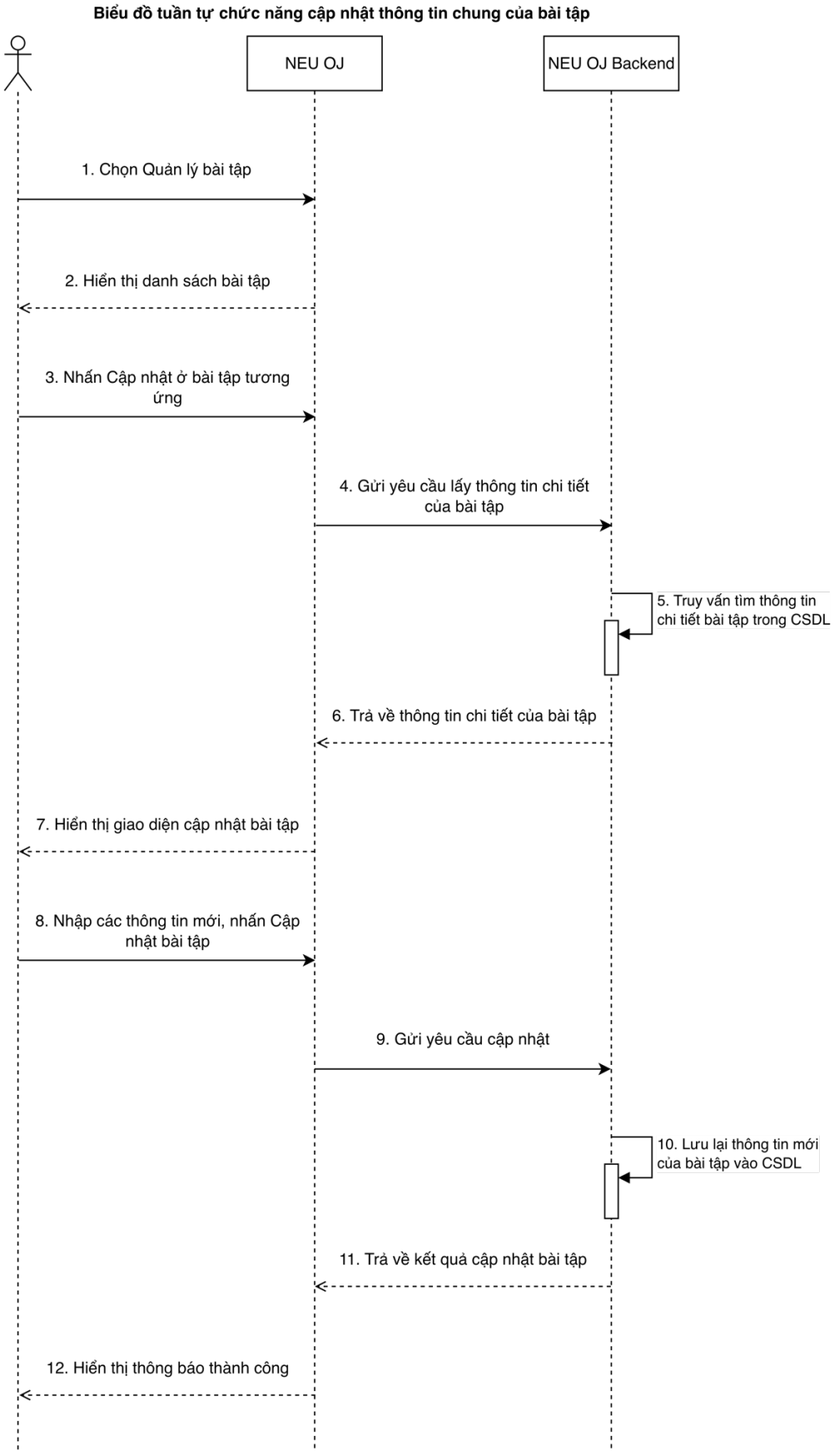
Hình 3.25. Biểu đồ tuần tự chức năng đăng nhập bằng SSO.



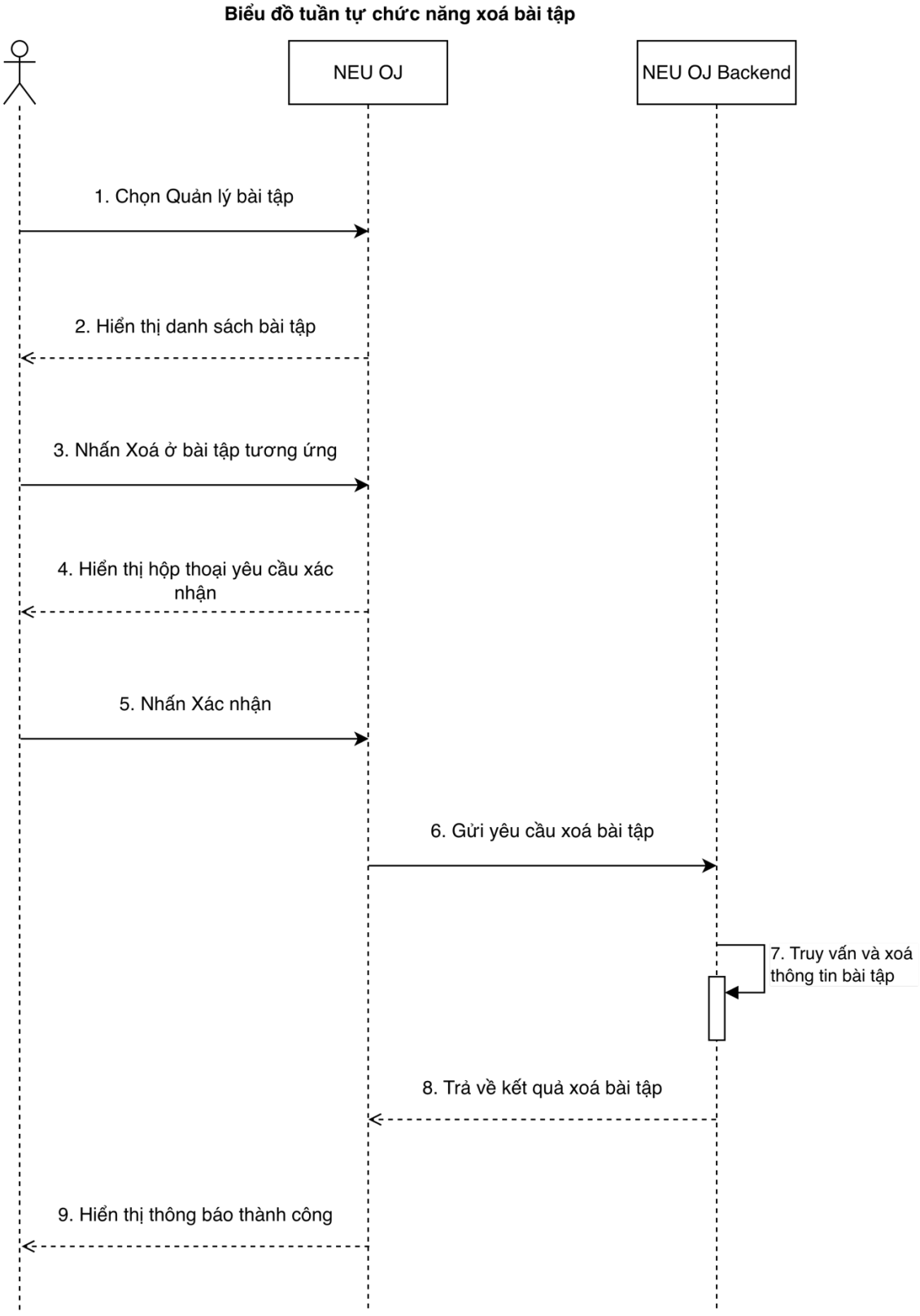
Hình 3.26. Biểu đồ tuần tự chức năng IDE trực tuyến.



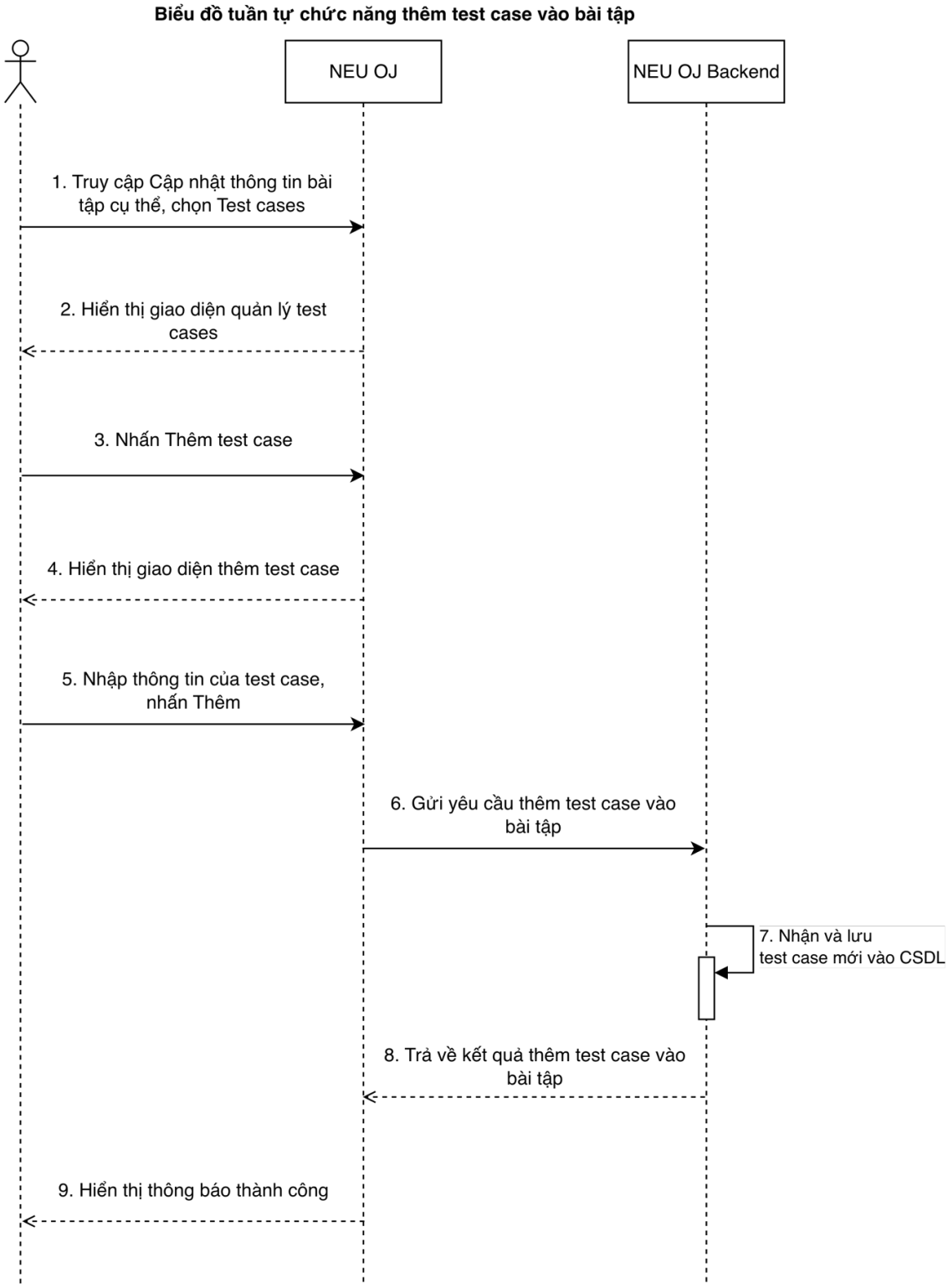
Hình 3.27. Biểu đồ tuần tự chức năng đóng góp bài tập.



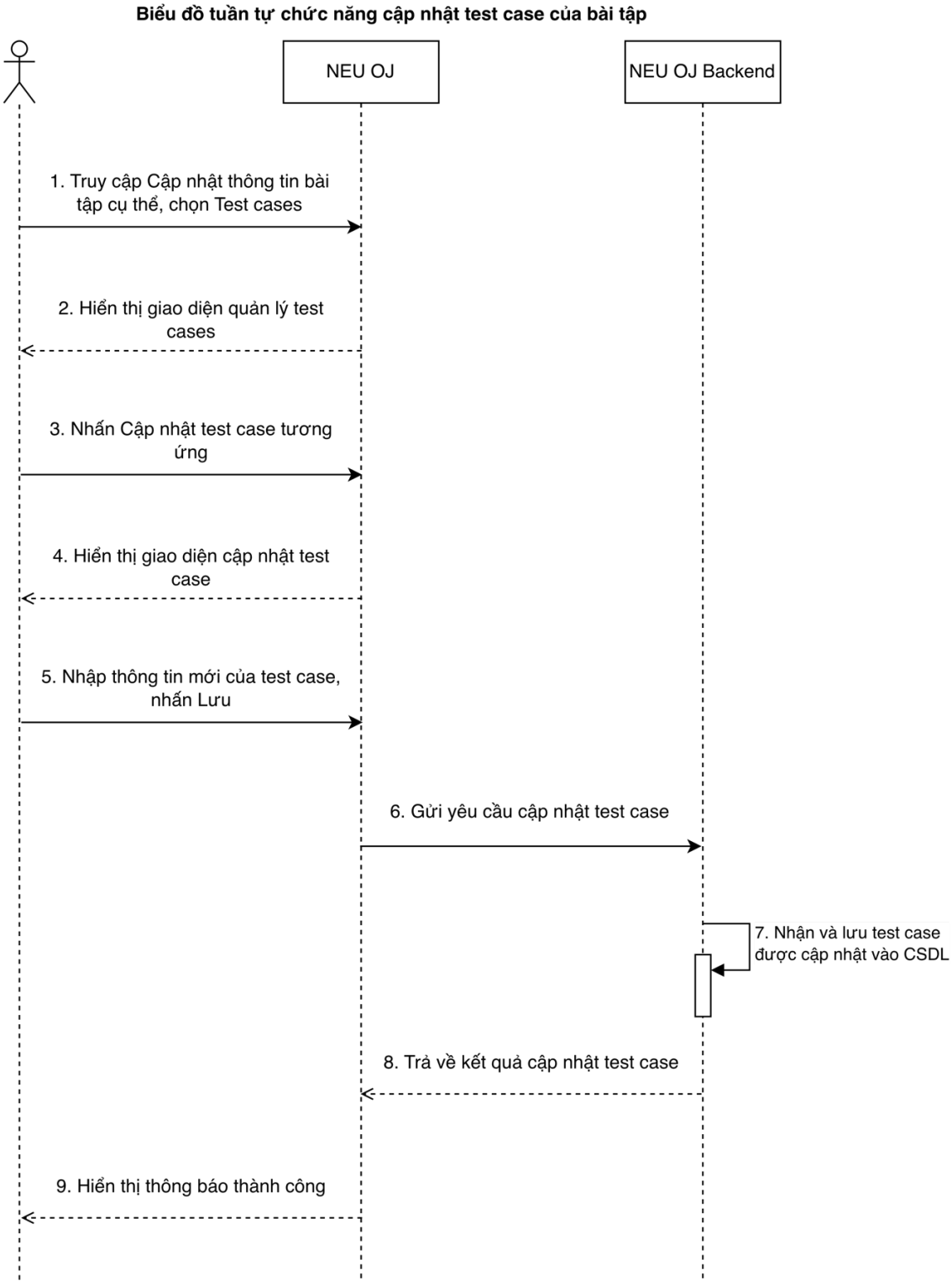
Hình 3.28. Biểu đồ tuần tự chức năng cập nhật thông tin chung của bài tập.



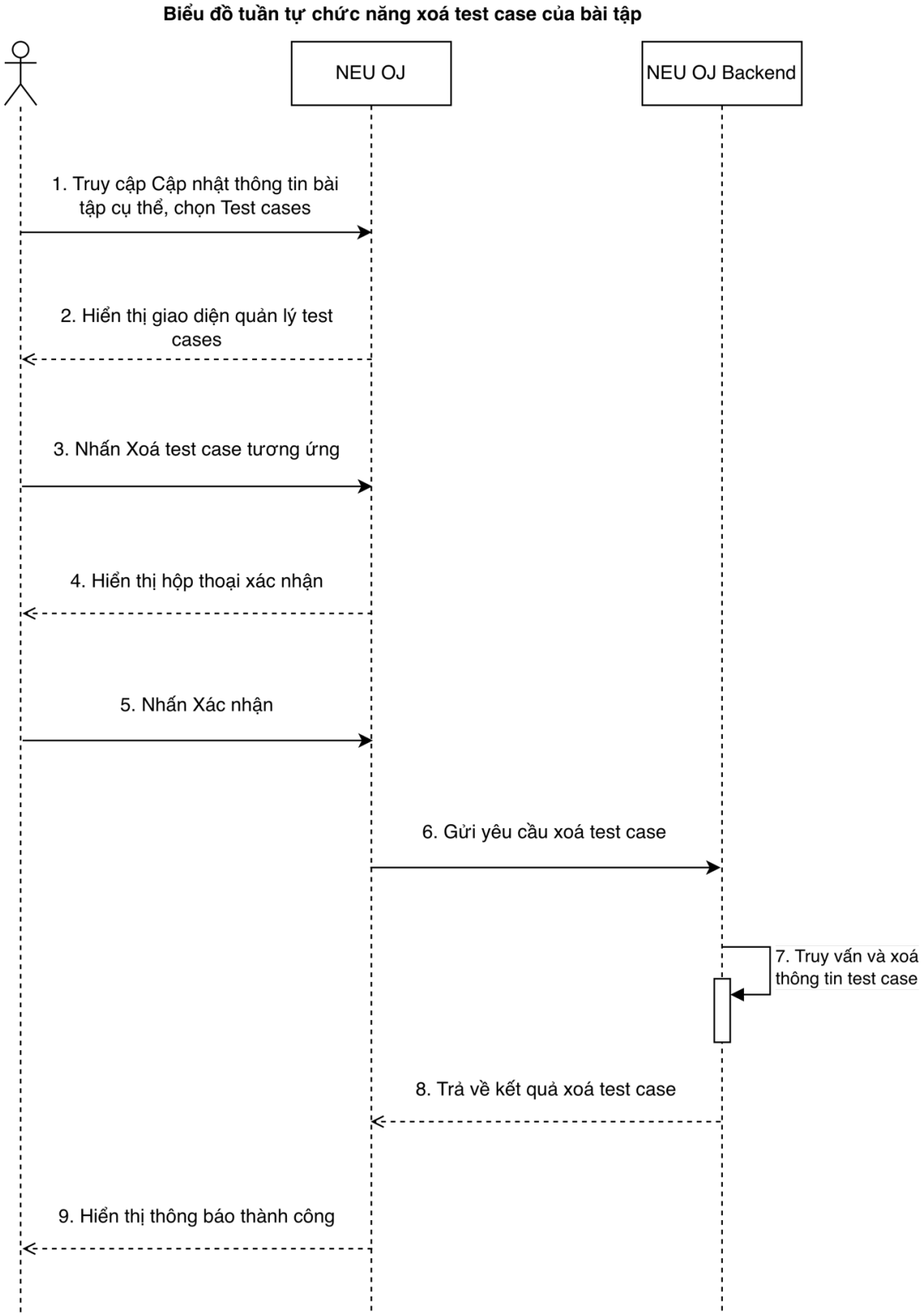
Hình 3.29. Biểu đồ tuần tự chức năng xoá bài tập.



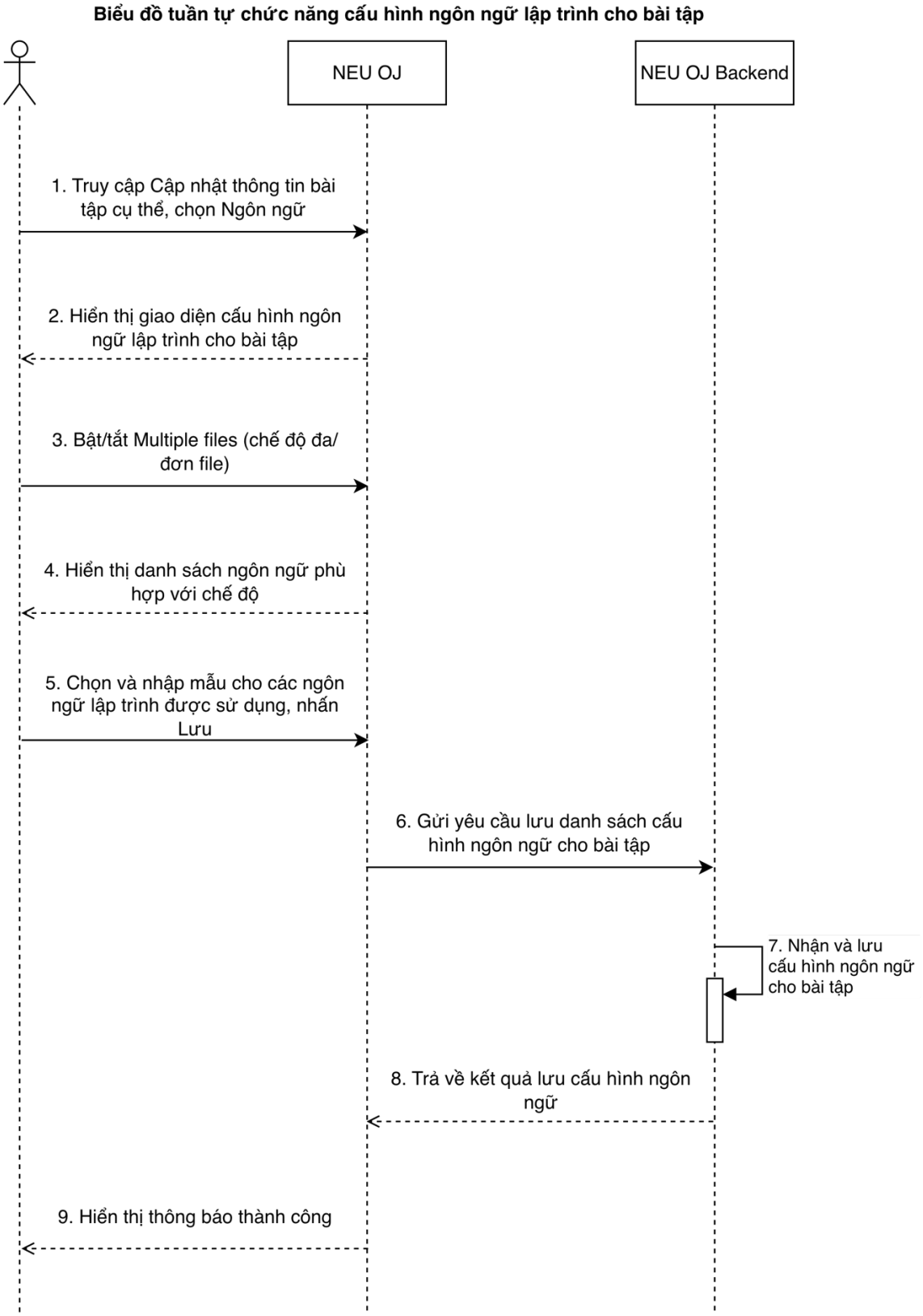
Hình 3.30. Biểu đồ tuần tự chức năng thêm test case vào bài tập.



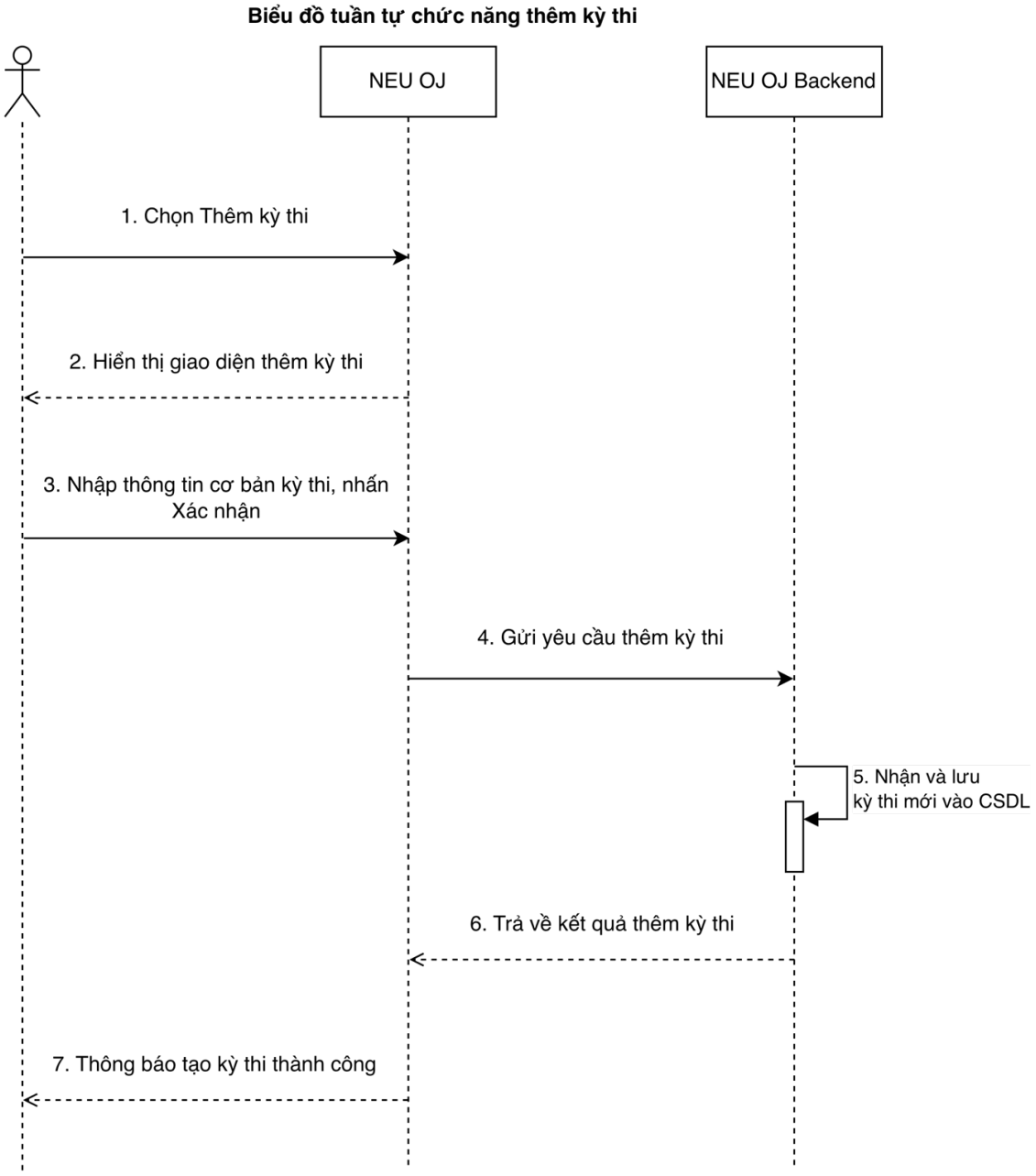
Hình 3.31. Biểu đồ tuần tự chức năng cập nhật test case.



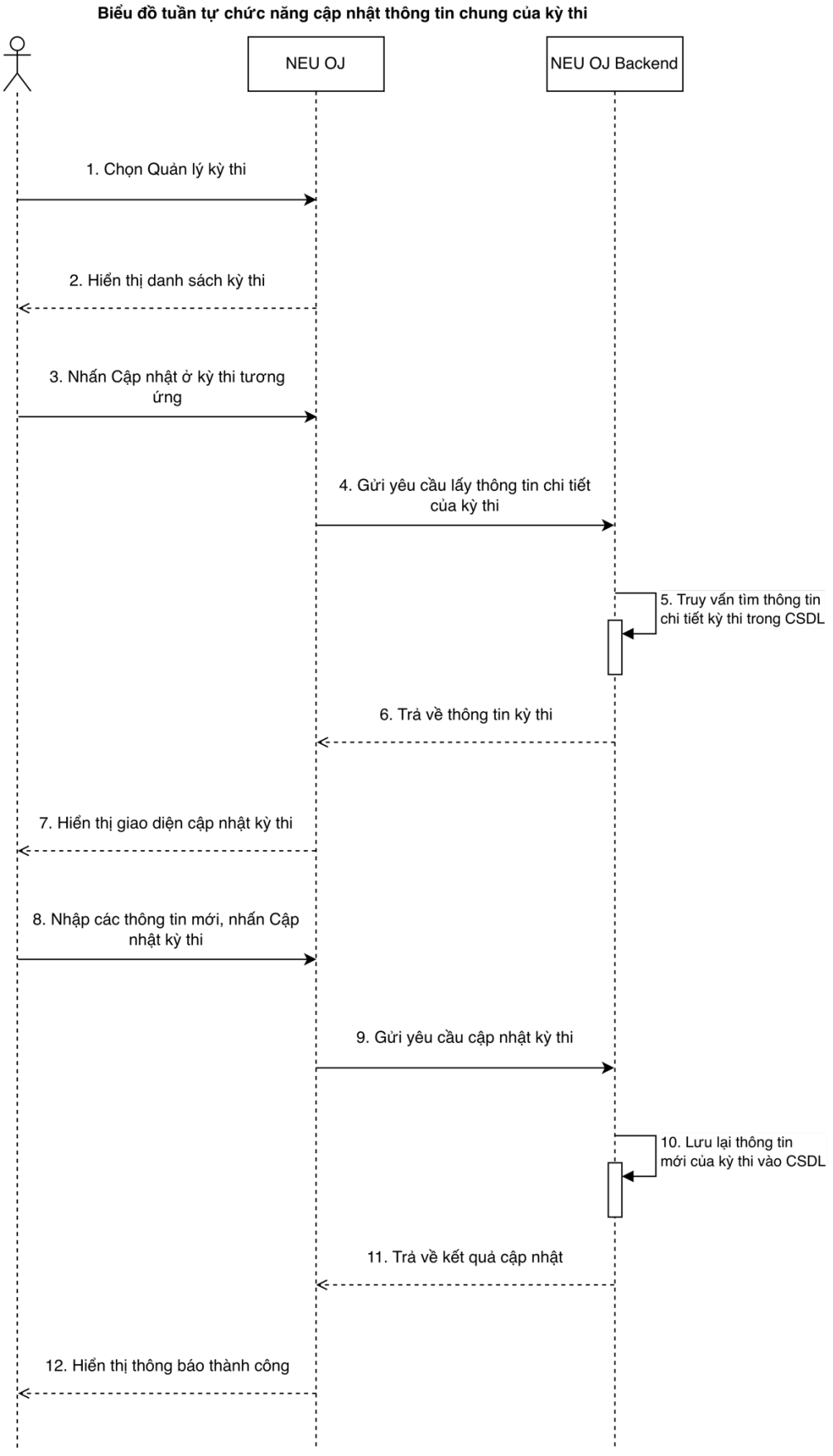
Hình 3.32. Biểu đồ tuần tự chức năng xoá test case.



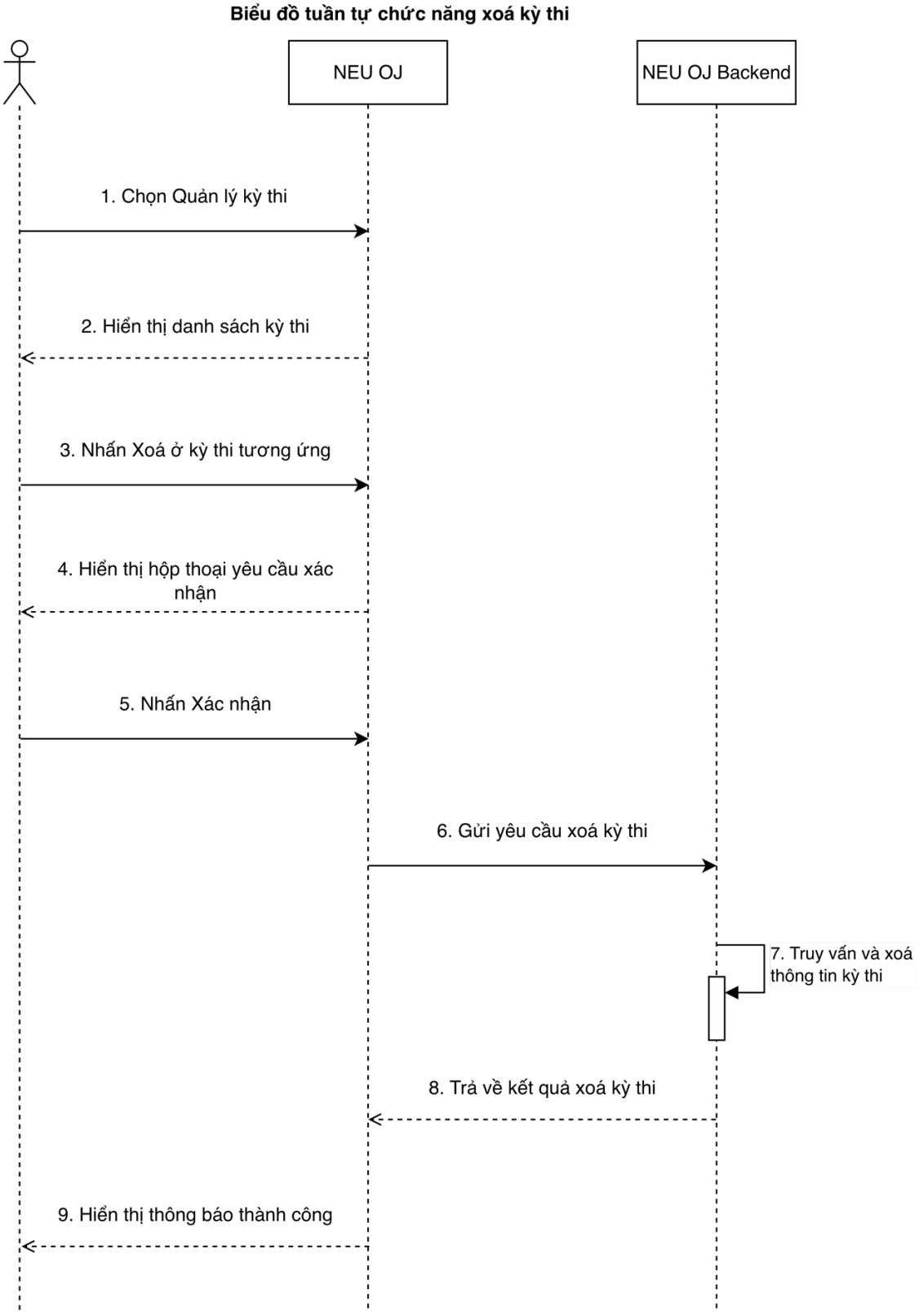
Hình 3.33. Biểu đồ tuần tự chức năng cấu hình ngôn ngữ cho bài tập.



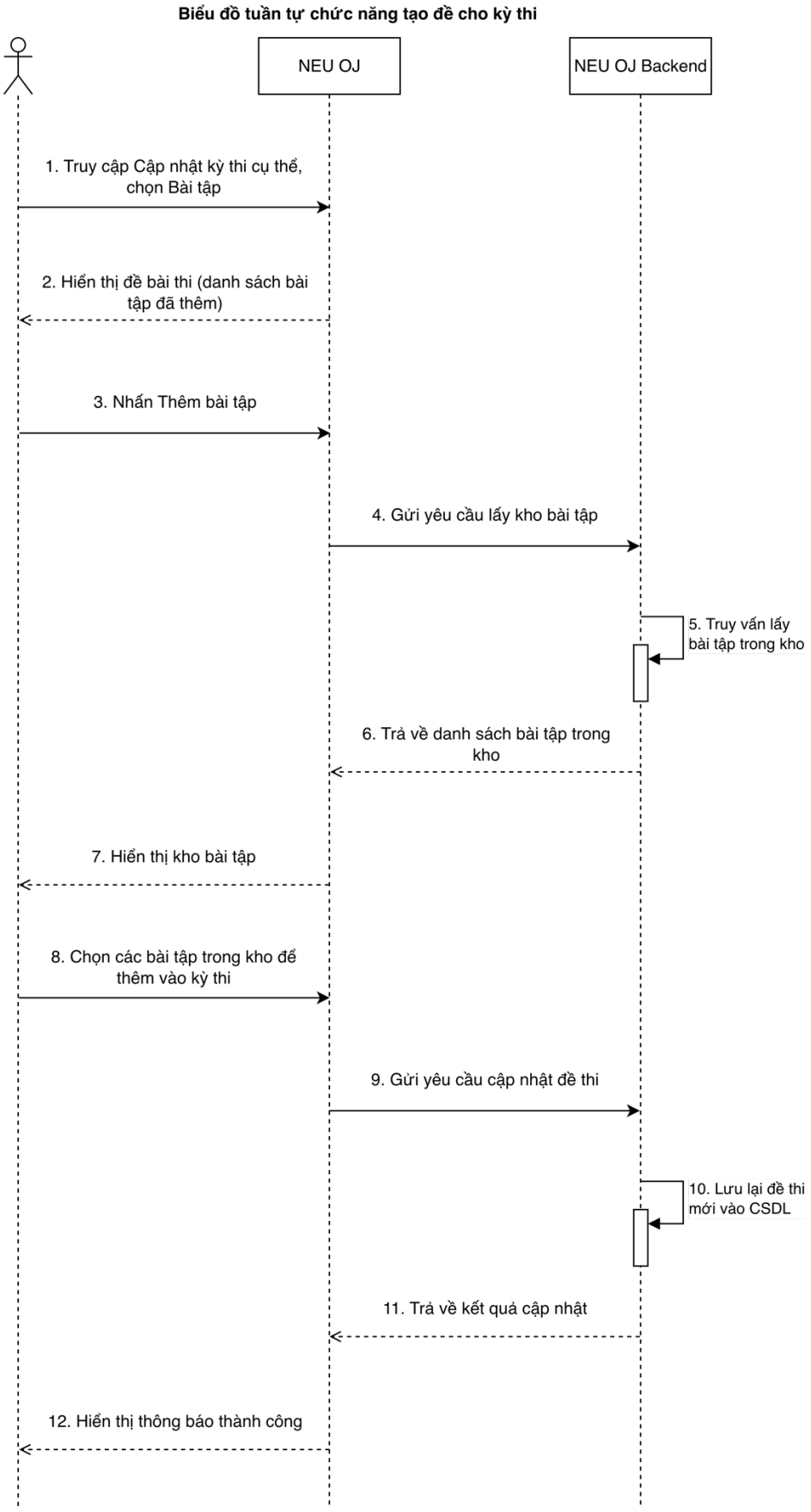
Hình 3.34. Biểu đồ tuần tự chức năng thêm kỳ thi.



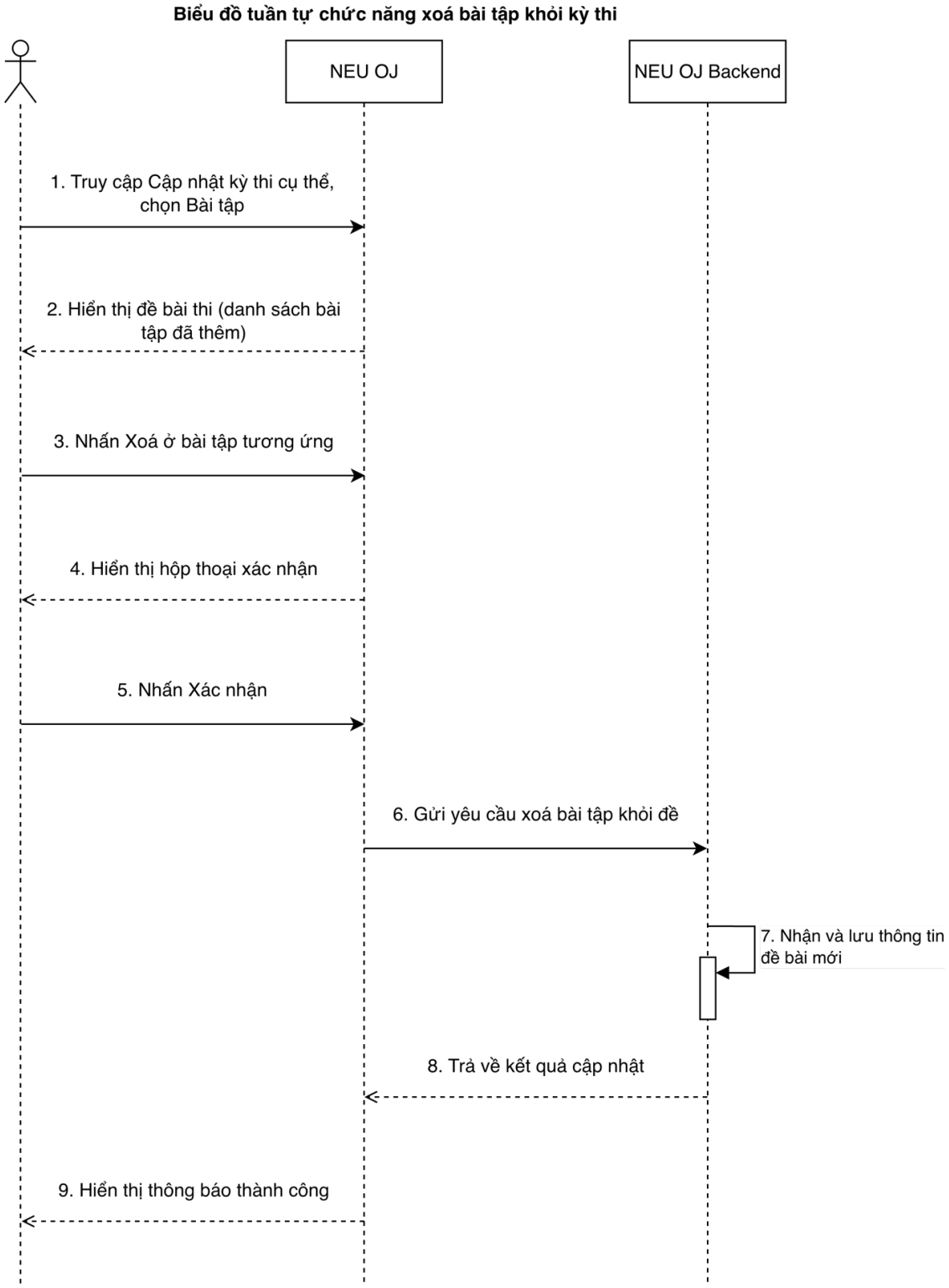
Hình 3.35. Biểu đồ tuần tự chức năng cập nhật thông tin chung của kỳ thi.



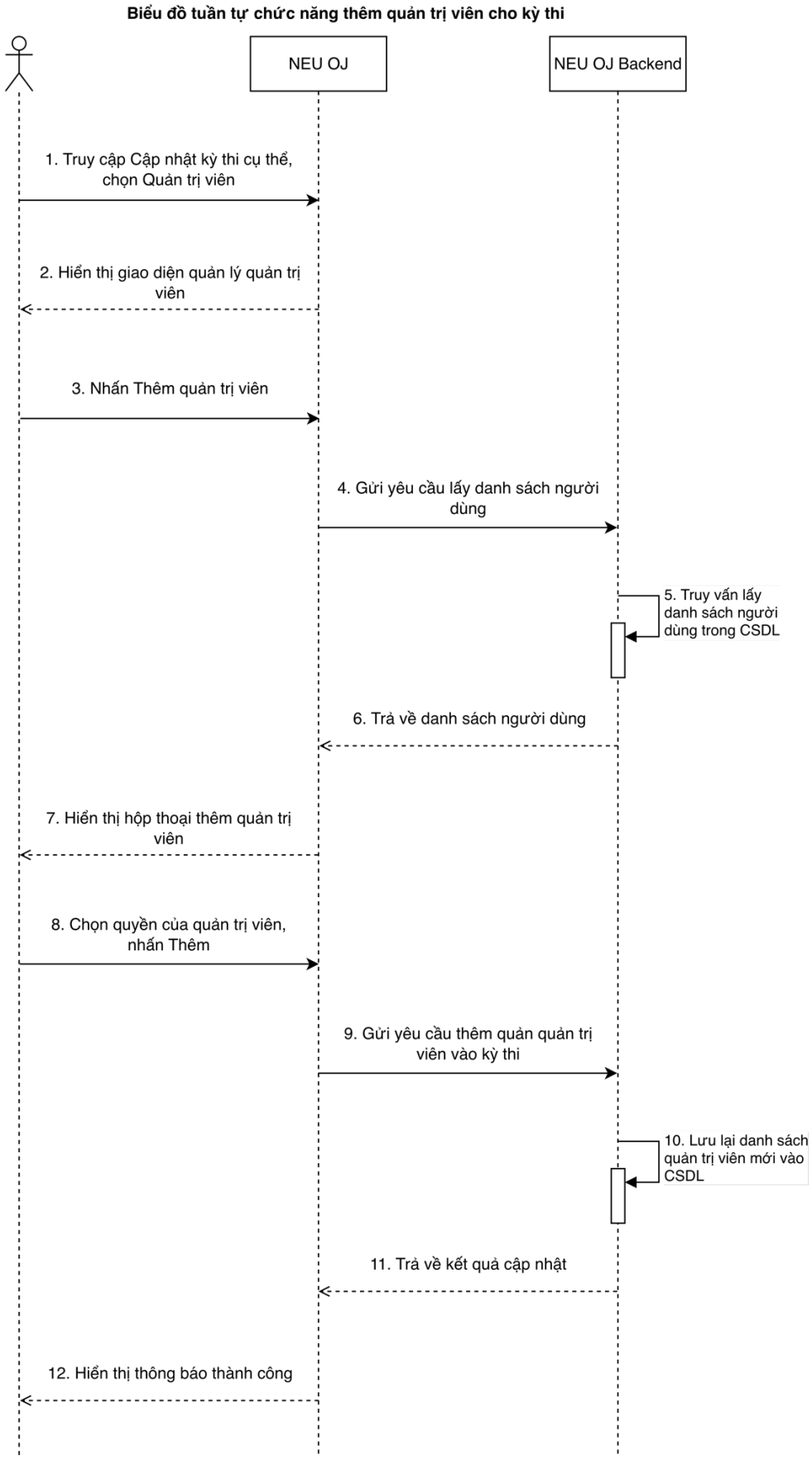
Hình 3.36. Biểu đồ tuần tự chức năng xoá kỳ thi.



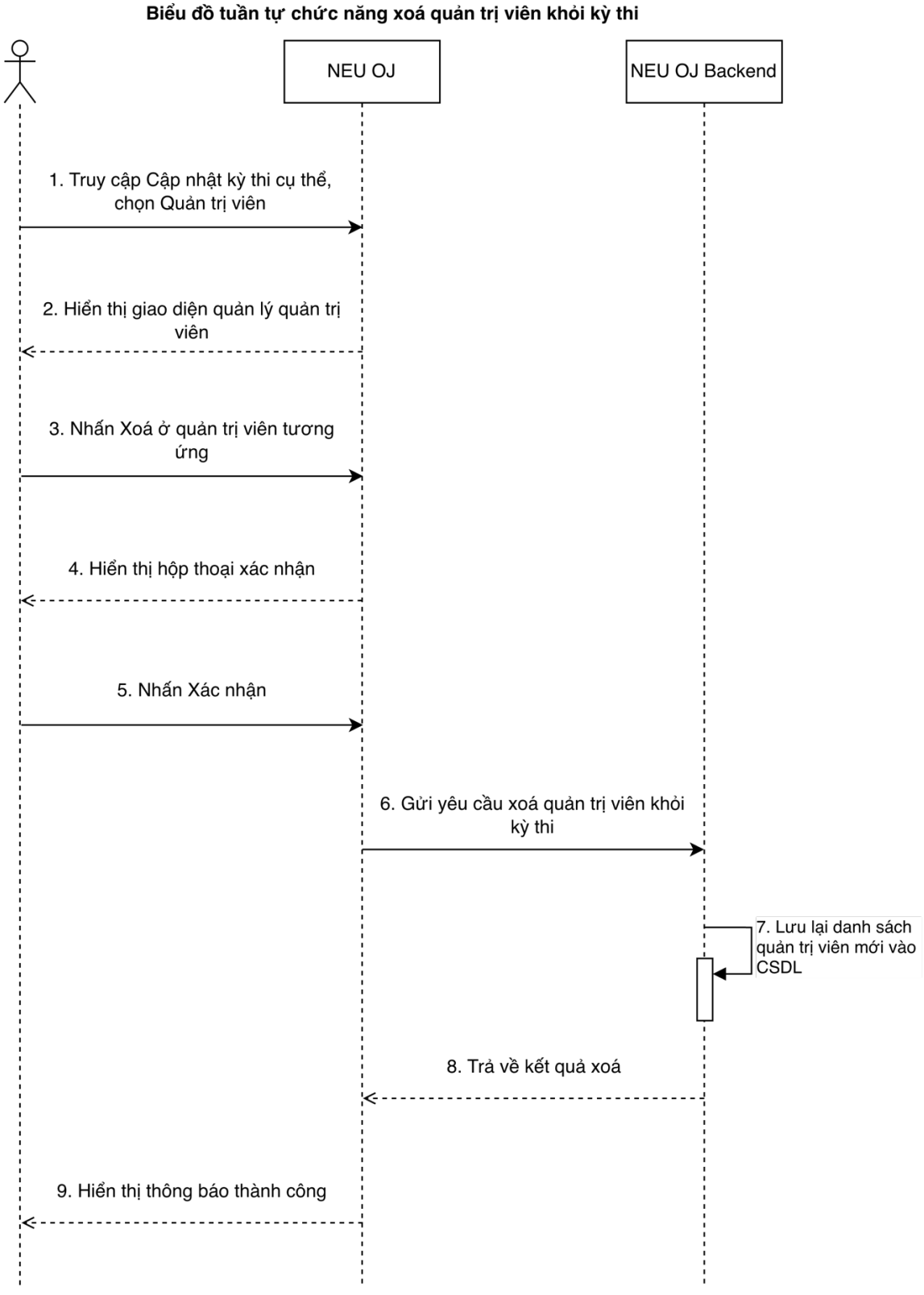
Hình 3.37. Biểu đồ tuần tự chức năng tạo đề cho kỳ thi.



Hình 3.38. Biểu đồ tuần tự chức năng xoá bài tập khỏi kỳ thi.

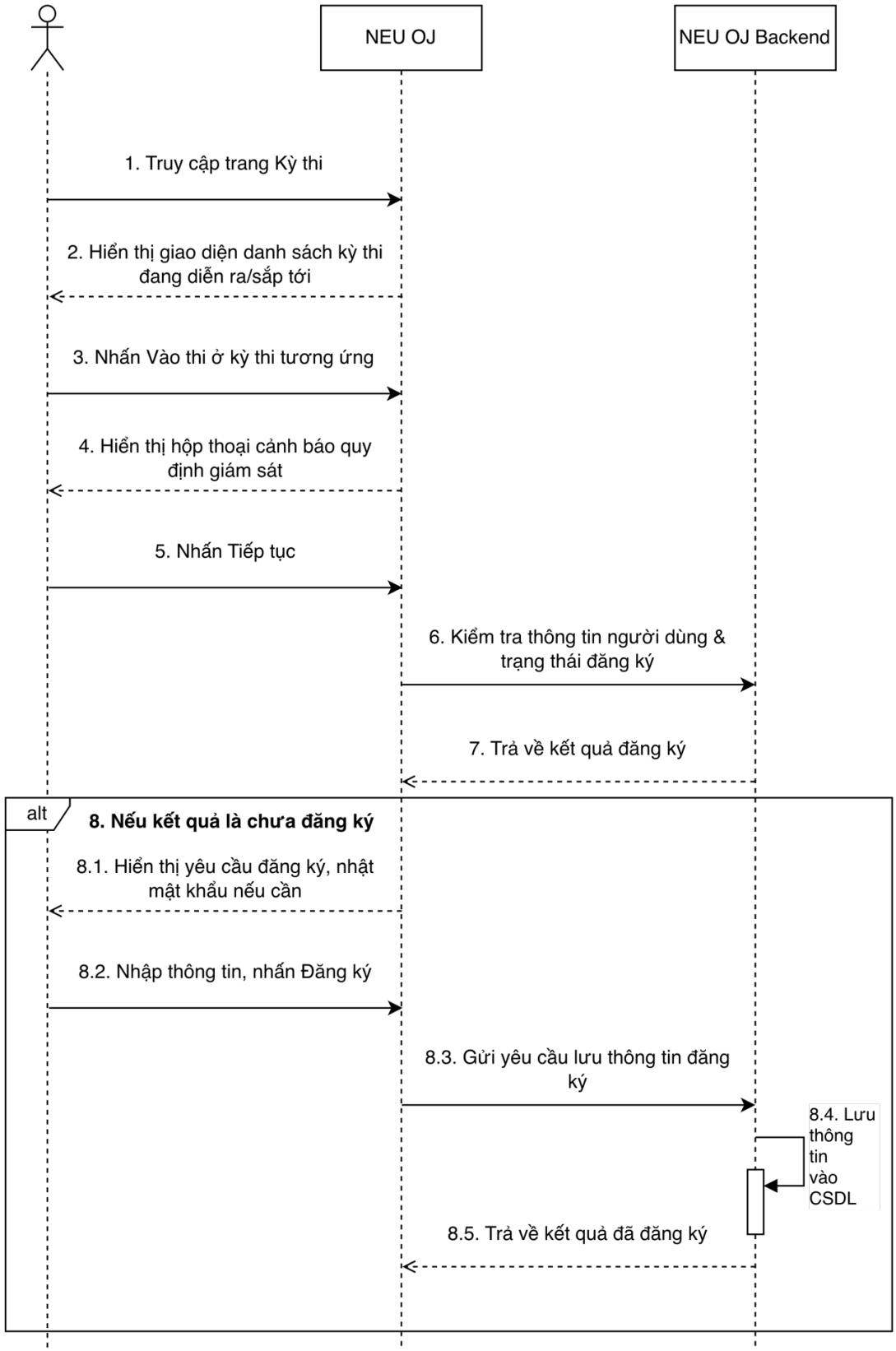


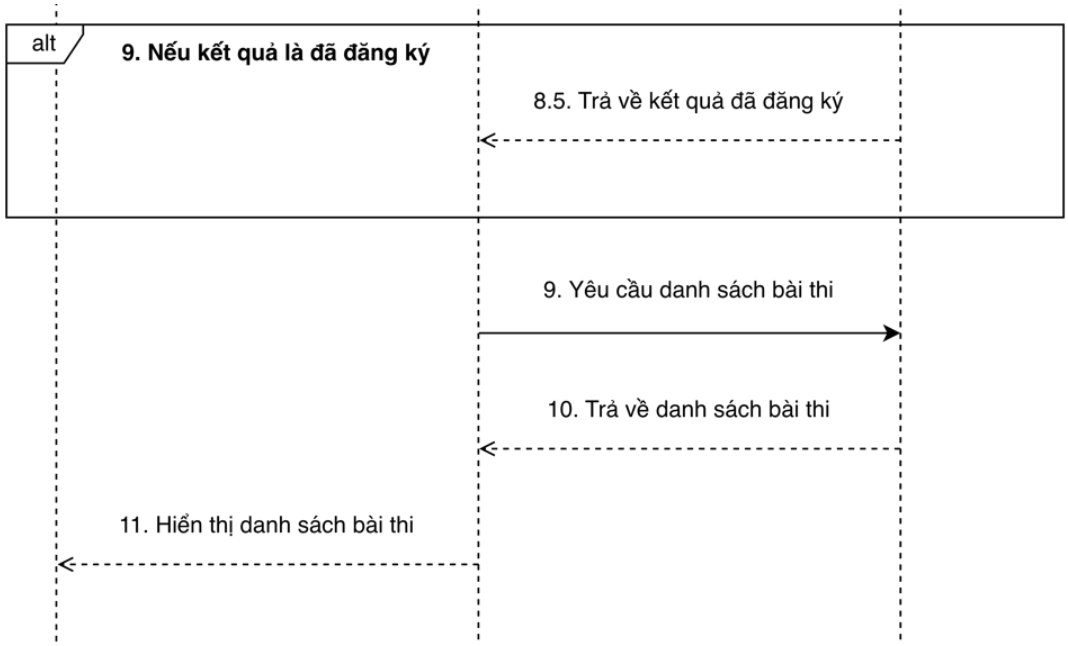
Hình 3.39. Thêm quản trị viên cho kỳ thi.



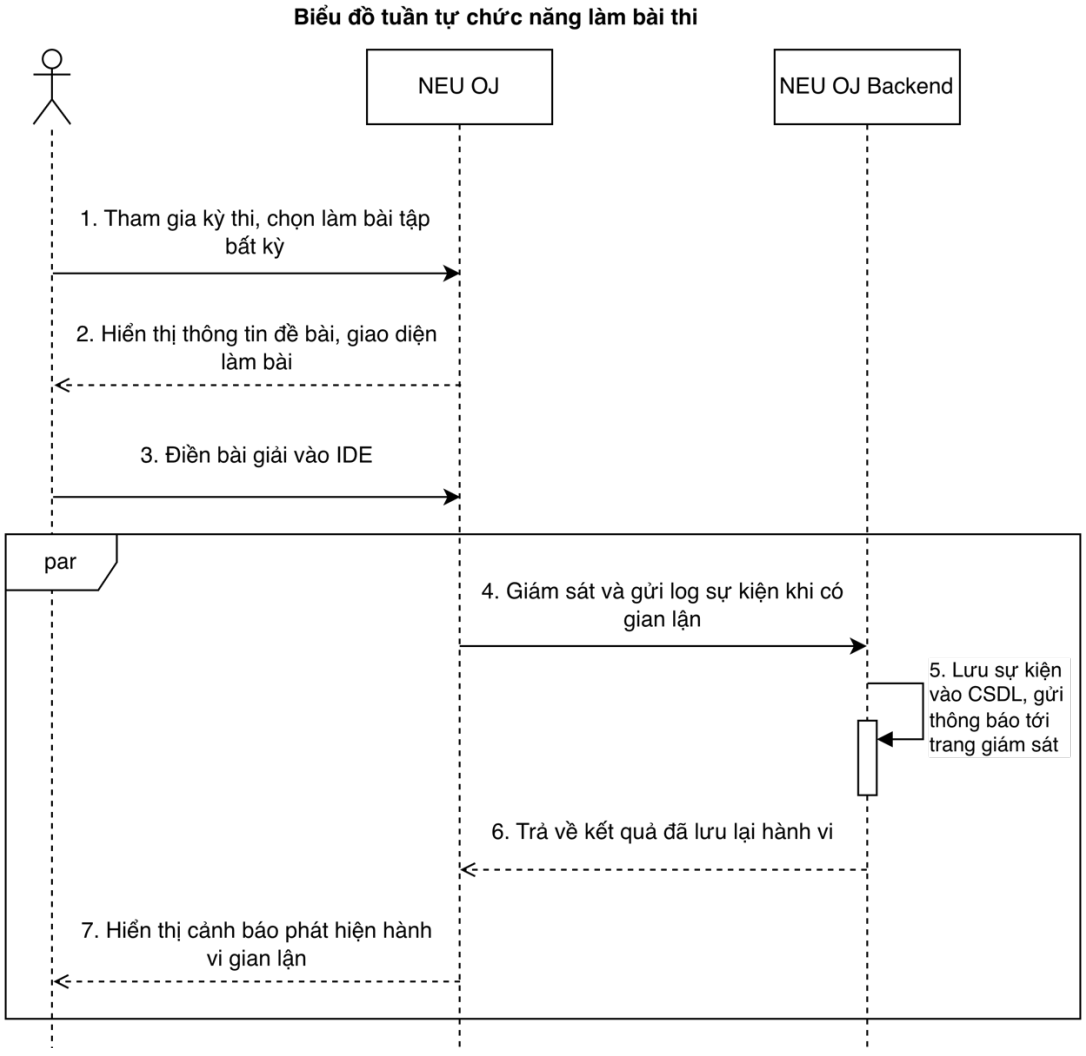
Hình 3.40. Biểu đồ tuần tự chức năng xoá quản trị viên khỏi kỳ thi.

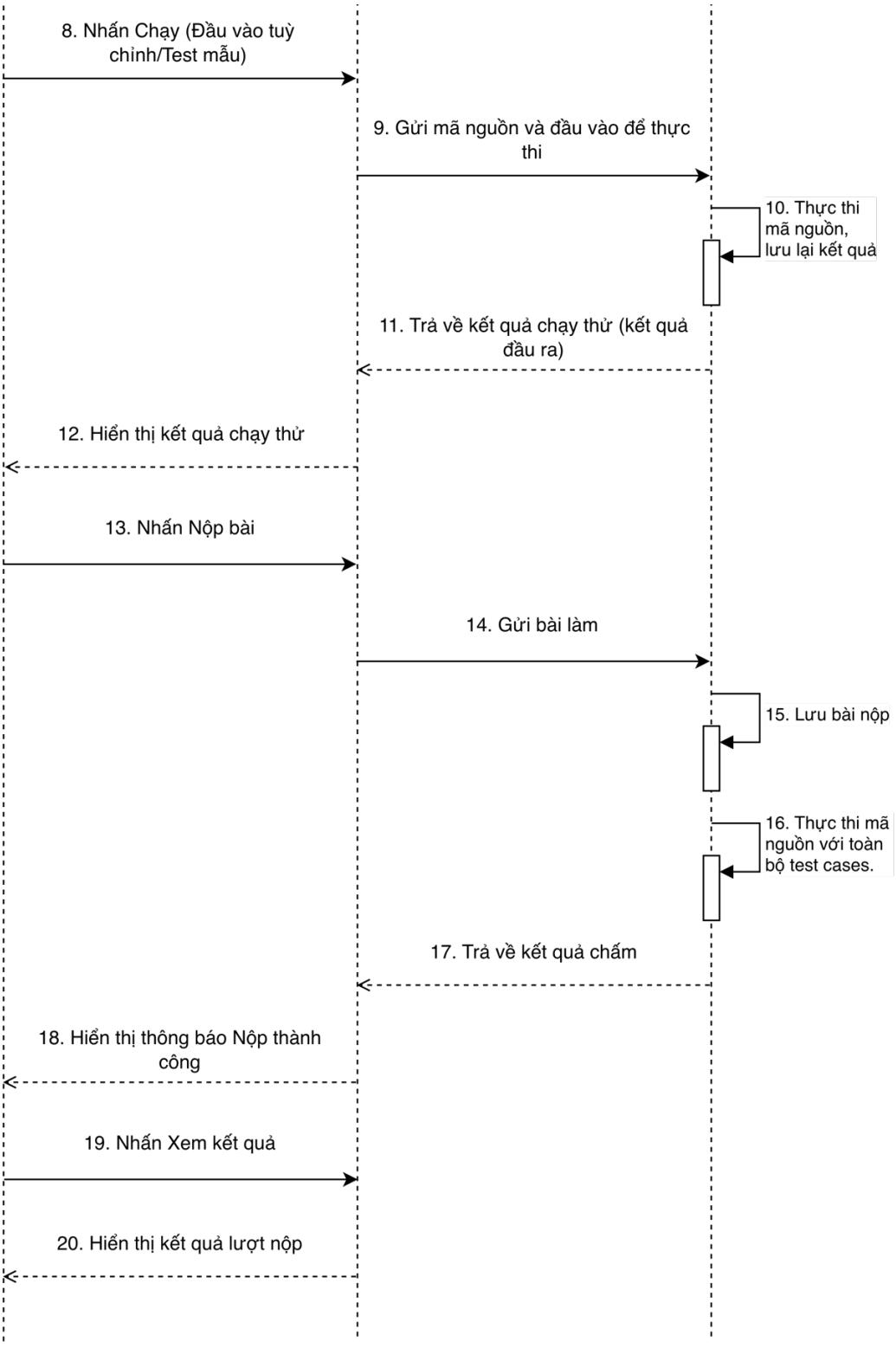
Biểu đồ tuần tự chức năng tham gia kỳ thi





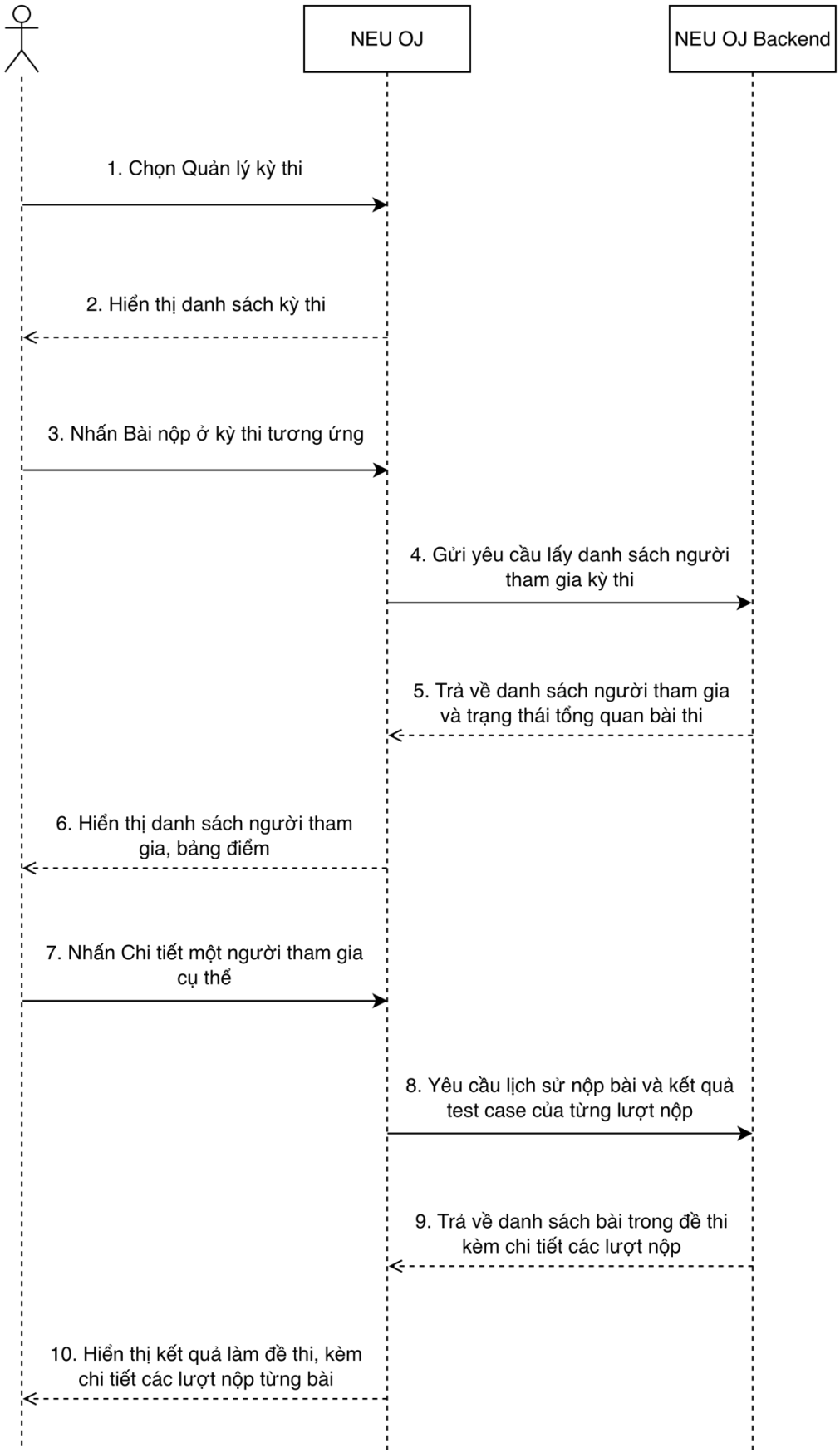
Hình 3.41. Biểu đồ tuần tự chức năng tham gia kỳ thi.

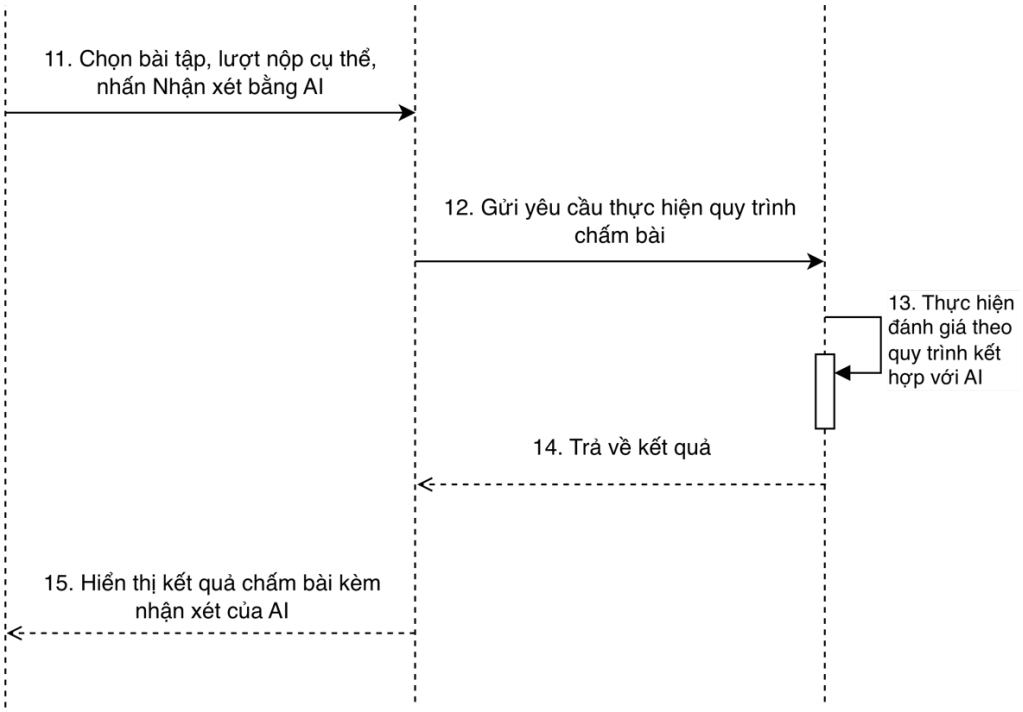




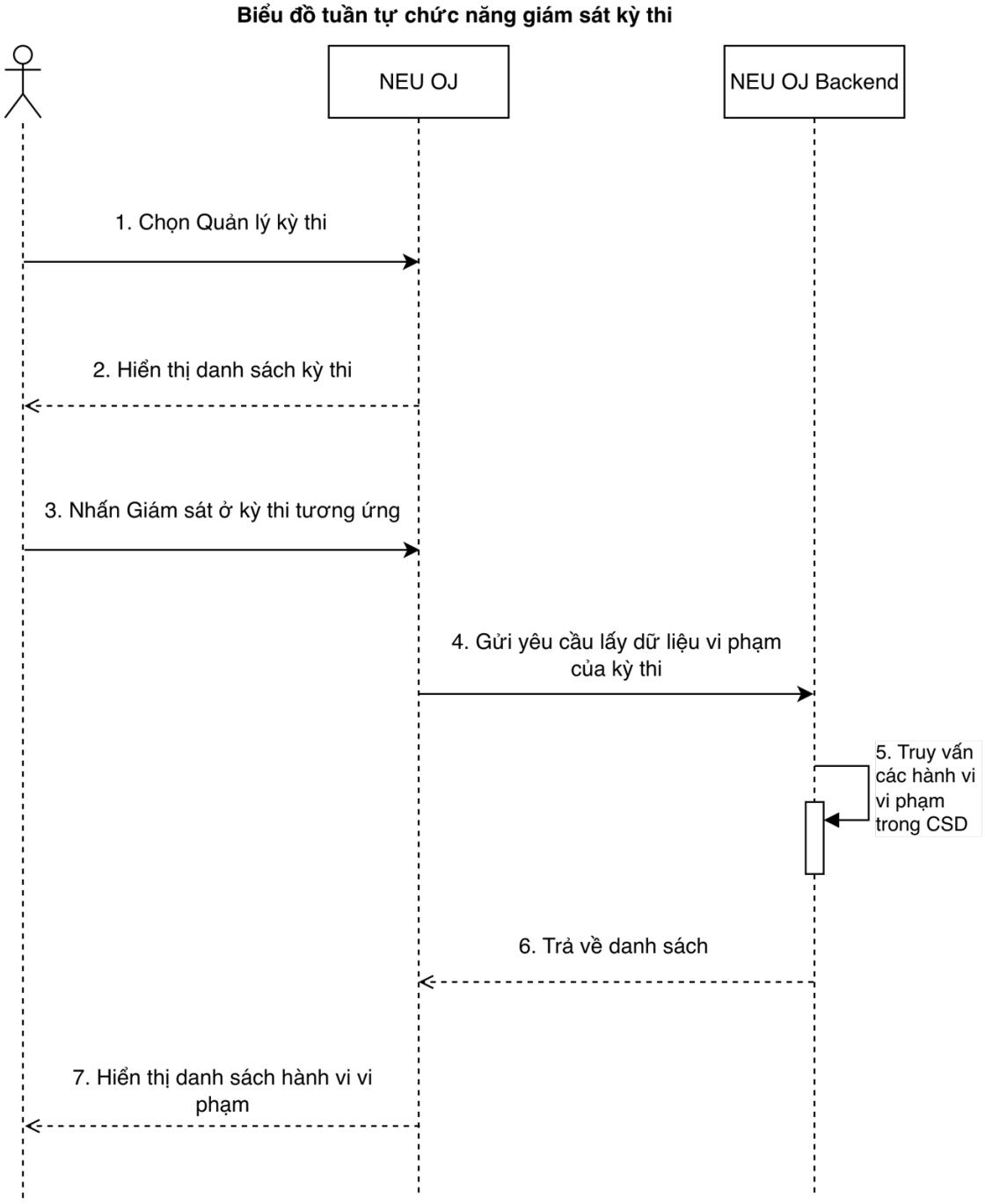
Hình 3.42. Biểu đồ tuần tự chức năng làm bài thi.

Biểu đồ tuần tự chức năng duyệt kết quả làm bài

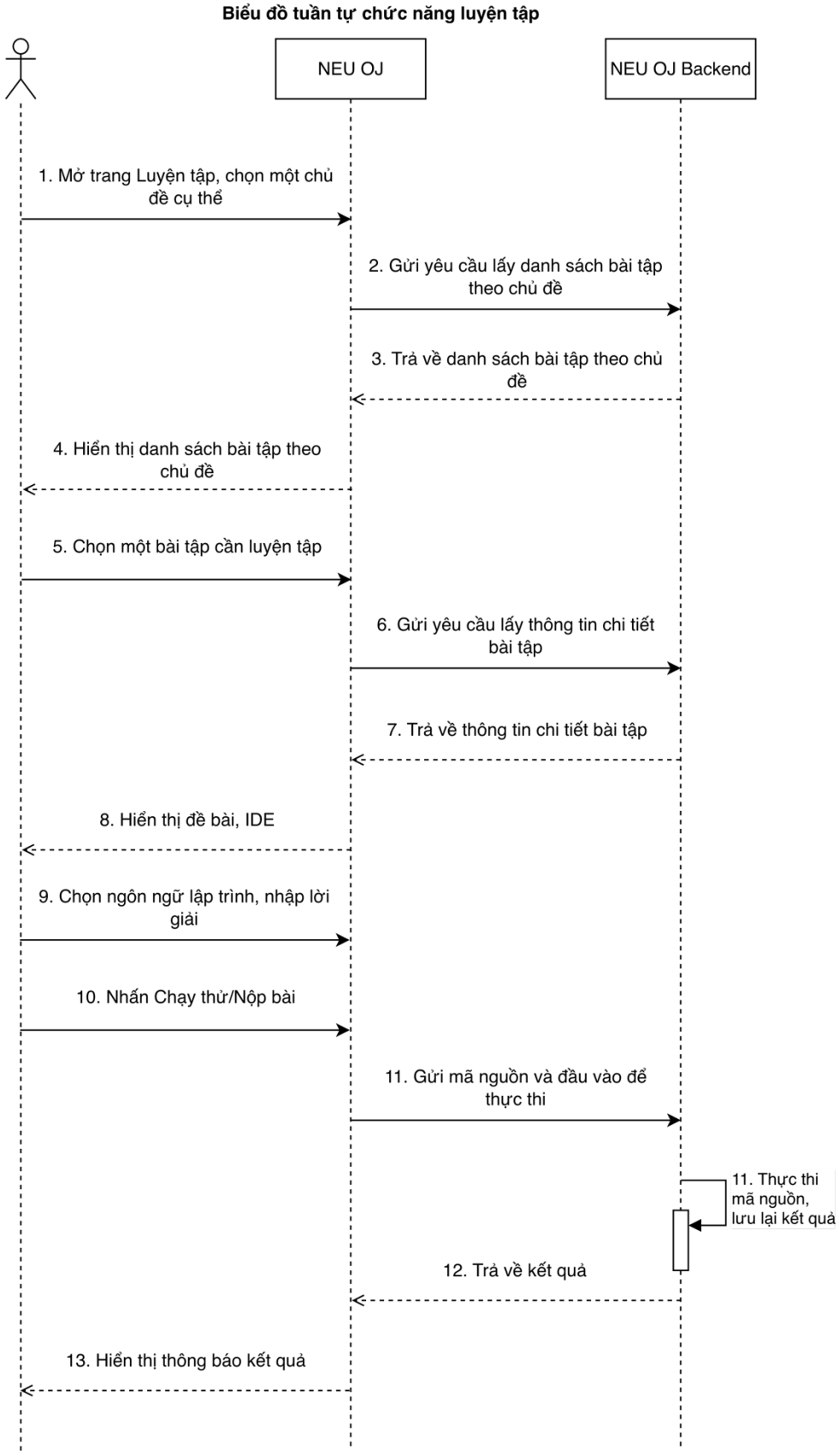




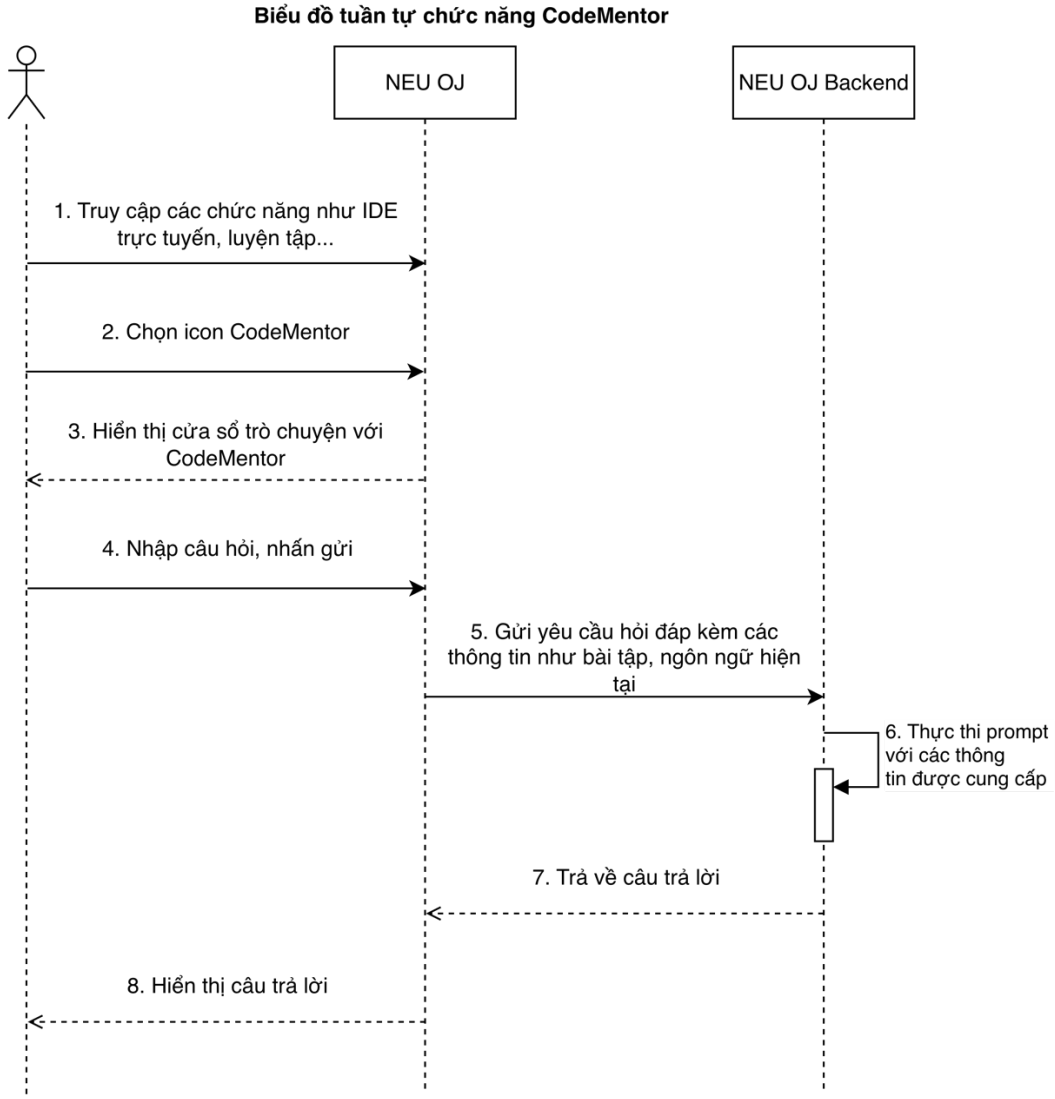
Hình 3.43. Biểu đồ tuần tự chức năng duyệt kết quả làm bài.



Hình 3.44. Biểu đồ tuần tự chức năng giám sát kỳ thi.



Hình 3.45. Biểu đồ tuần tự chức năng luyện tập.



Hình 3.46. Biểu đồ tuần tự chức năng CodeMentor.

3.2. Thiết kế hệ thống

3.2.1. Nền tảng công nghệ và môi trường phát triển

3.2.1.1. Hệ thống thực thi mã nguồn Judge0

Để giải quyết bài toán cốt lõi và phức tạp nhất của hệ thống là biên dịch và thực thi mã nguồn một cách an toàn, đề tài đã lựa chọn tích hợp Judge0 – một hệ thống thực thi mã nguồn trực tuyến mã nguồn mở, mạnh mẽ và có khả năng mở rộng.

Judge0 là một nền tảng chuyên dụng, được thiết kế để cung cấp một môi trường cô lập an toàn cho việc chạy mã lệnh từ người dùng. Mục đích chính của nó là phục vụ các ứng dụng cần khả năng chấm bài tự động như các nền tảng thi lập trình thi đấu, các hệ thống học tập trực tuyến, công cụ tuyển dụng và các môi trường phát triển tích hợp (IDE) trên nền web.

Mặc dù đã có rất nhiều nền tảng tương tự cũng đã được giới thiệu, nhưng việc lựa chọn Judge0 cho đề tài này dựa trên một số ưu thế như sau:

- Hỗ trợ đa ngôn ngữ: Judge0 hỗ trợ hơn 60 ngôn ngữ lập trình phổ biến, đáp ứng đầy đủ nhu cầu đa dạng của sinh viên và các môn học khác nhau.
- Bảo mật và an toàn: Toàn bộ quá trình từ biên dịch đến thực thi đều được diễn ra trong một môi trường cô lập, giúp ngăn chặn các mã lệnh độc hại có thể ảnh hưởng đến máy chủ hệ thống. Đây là yếu tố sống còn đối với một hệ thống cho phép người dùng tự do nộp và chạy mã nguồn.
- Kiến trúc API mạnh mẽ: Judge0 cung cấp một bộ API đầy đủ và được tài liệu hóa chi tiết. Điều này cho phép hệ thống NEU OJ có thể dễ dàng gửi mã nguồn, các bộ dữ liệu đầu vào, và nhận lại kết quả chi tiết về quá trình thực thi (trạng thái, thời gian chạy, bộ nhớ sử dụng).
- Tiết kiệm thời gian phát triển: Bằng việc tích hợp một engine đã được kiểm chứng và tối ưu như Judge0, đề tài có thể tập trung nguồn lực vào việc phát triển các module nghiệp vụ khác như giao diện người dùng, quản lý cuộc thi, và thư viện bài tập, thay vì phải xây dựng một lõi chấm phức tạp từ đầu.

Trong kiến trúc của NEU OJ, Judge0 đóng vai trò là trái tim của hệ thống, chịu trách nhiệm xử lý toàn bộ các yêu cầu chấm bài, đảm bảo tính chính xác, công bằng và an toàn cho mọi bài nộp của người dùng.

3.2.1.2. Framework phát triển phía back-end NestJS

Để xây dựng phần máy chủ của hệ thống NEU OJ, đề tài lựa chọn NestJS, một framework Node.js mã nguồn mở, tiến bộ để xây dựng các ứng dụng phía máy chủ hiệu quả, đáng tin cậy và có khả năng mở rộng.

Việc lựa chọn NestJS làm nền tảng cho backend của dự án mang lại nhiều lợi ích quan trọng về mặt kiến trúc và phát triển:

- Kiến trúc Module hóa rõ ràng: NestJS áp dụng một kiến trúc module hóa chặt chẽ, giúp mã nguồn được tổ chức một cách logic, dễ dàng quản lý, bảo trì và tái sử dụng, đặc biệt là khi dự án phát triển với quy mô lớn.
- Xây dựng trên nền tảng TypeScript: Việc xây dựng hoàn toàn bằng TypeScript mang lại lợi ích của kiểu tĩnh, giúp phát hiện lỗi ngay trong quá trình phát triển, tăng cường tính ổn định của mã nguồn và hệ thống.
- Hệ thống Dependency Injection: NestJS tích hợp sẵn một hệ thống Dependency Injection mạnh mẽ. Cơ chế này giúp giảm sự phụ thuộc trực tiếp giữa các thành phần trong ứng dụng, làm cho mã nguồn trở nên linh hoạt, dễ kiểm thử và dễ bảo trì hơn.

Trong kiến trúc của hệ thống NEU OJ, NestJS đóng vai trò là bộ não xử lý trung tâm. Nó chịu trách nhiệm xử lý tất cả các logic nghiệp vụ, quản lý kết nối và truy vấn đến cơ sở dữ liệu MongoDB, xử lý xác thực và phân quyền người dùng, đồng thời cung cấp một hệ thống API mạnh mẽ để giao diện người dùng có thể tương tác và trao đổi dữ liệu. Nhờ cấu trúc có tổ chức cao, NestJS giúp xây dựng một back-end vững chắc, an toàn và dễ dàng mở rộng các tính năng mới trong tương lai.

3.2.1.3. Framework phát triển phía front-end Next.js

Để xây dựng giao diện hệ thống hiện đại, tốc độ cao và thân thiện cho nền tảng NEU OJ, dự án sử dụng Next.js – một framework phát triển web mã nguồn mở được xây dựng dựa trên thư viện React.

Việc lựa chọn Next.js cho phần giao diện của dự án dựa trên nhiều ưu điểm kỹ thuật vượt trội:

- Cơ chế Rendering linh hoạt: Khác với các ứng dụng React truyền thống thường chỉ render phía máy khách, Next.js cung cấp các chiến lược rendering linh hoạt cho từng trang riêng biệt. Framework này hỗ trợ mạnh mẽ cả render phía máy chủ và tạo các trang tĩnh. Điều này đặc biệt quan

trọng đối với một nền tảng như NEU OJ, vì nó đảm bảo thời gian tải trang ban đầu nhanh hơn đáng kể, mang lại trải nghiệm người dùng tốt hơn.

- Định tuyến dựa trên hệ thống tệp: Next.js đơn giản hóa việc định tuyến trong ứng dụng web bằng cơ chế định tuyến trong đó các đường dẫn được tự động tạo ra dựa trên cấu trúc tệp và thư mục bên trong thư mục *app*. Cách tiếp cận này giúp đẩy nhanh tốc độ phát triển và làm cho cấu trúc điều hướng của dự án trở nên dễ quản lý và dễ hiểu.
- Tích hợp sẵn các cơ chế tối ưu hóa hiệu năng: Hiệu năng là một trong những trọng tâm của Next.js. Framework này đi kèm với một loạt các tối ưu hóa tự động giúp tăng cường tốc độ và hiệu quả của ứng dụng.

Trong bối cảnh của hệ thống NEU OJ, Next.js cung cấp nền tảng kiến trúc vững chắc để xây dựng một giao diện người dùng có khả năng mở rộng, hiệu năng cao và giàu tính năng. Framework này giúp đơn giản hóa các cấu hình phức tạp, cho phép tập trung vào việc xây dựng các chức năng và trải nghiệm người dùng đặc thù mà nền tảng yêu cầu.

3.2.1.4. Hệ quản trị cơ sở dữ liệu MongoDB

Để lưu trữ và quản lý dữ liệu cho hệ thống NEU OJ, đề tài sử dụng MongoDB, một hệ quản trị cơ sở dữ liệu NoSQL mã nguồn mở, đa nền tảng và hướng tài liệu. Được phân loại là cơ sở dữ liệu NoSQL, MongoDB khác biệt với các hệ quản trị cơ sở dữ liệu quan hệ truyền thống bằng cách lưu trữ dữ liệu trong các bản ghi linh hoạt, có cấu trúc giống JSON, được gọi là Binary JSON.

Việc lựa chọn MongoDB làm cơ sở dữ liệu chính cho dự án dựa trên những đặc tính kỹ thuật ưu việt, phù hợp với yêu cầu của một ứng dụng web hiện đại:

- Mô hình dữ liệu linh hoạt: Đây là một trong những ưu điểm lớn nhất của MongoDB. Không giống như cơ sở dữ liệu SQL, MongoDB cho phép các tài liệu trong cùng một bộ sưu tập có thể có các trường và cấu trúc khác nhau. Sự linh hoạt này đặc biệt hữu ích trong các trường hợp cấu trúc dữ liệu cần dễ dàng thay đổi và tiến hóa cùng với các yêu cầu của ứng dụng mà không cần thực hiện các thao tác chỉnh sửa cấu trúc dữ liệu phức tạp.
- Mô hình hướng tài liệu: MongoDB lưu trữ các dữ liệu liên quan với nhau trong cùng một tài liệu duy nhất, thay vì phải tách ra nhiều bảng và thực hiện các phép JOIN tốn kém hiệu năng. Mô hình này giúp đơn giản hóa việc truy vấn và cải thiện đáng kể hiệu suất đọc dữ liệu cho các tác vụ phổ biến.

- Khả năng mở rộng ngang: MongoDB được thiết kế để dễ dàng mở rộng theo chiều ngang thông qua kỹ thuật sharding. Sharding cho phép phân tán dữ liệu trên nhiều máy chủ, giúp hệ thống có thể xử lý các tập dữ liệu cực lớn và lưu lượng truy cập cao mà các hệ thống cơ sở dữ liệu đơn lẻ không thể đáp ứng được.

Trong hệ thống NEU OJ, MongoDB chịu trách nhiệm lưu trữ toàn bộ dữ liệu của ứng dụng, bao gồm thông tin người dùng, danh sách bài tập, chi tiết các cuộc thi, và lịch sử bài nộp. Sự linh hoạt và hiệu suất của MongoDB là nền tảng quan trọng giúp hệ thống hoạt động ổn định và dễ dàng phát triển, mở rộng trong tương lai.

3.2.1.5. Công nghệ container hoá Docker

Docker là một nền tảng mã nguồn mở giúp tự động hóa việc tạo, triển khai và vận hành các ứng dụng bên trong một môi trường cô lập được gọi là container. Về cơ bản, container là một phương pháp ảo hóa ở cấp hệ điều hành, cho phép một ứng dụng và toàn bộ các phụ thuộc của nó - như thư viện, mã nguồn và tệp cấu hình - được đóng gói lại thành một đơn vị duy nhất.

Một trong những ưu điểm lớn nhất của Docker là tính di động và khả năng mở rộng vượt trội. Vì container chứa tất cả các thành phần cần thiết để chạy, ứng dụng có thể hoạt động nhất quán trên nhiều nền tảng khác nhau, từ máy tính cá nhân của lập trình viên đến các máy chủ đám mây. Ngoài ra, Docker giúp tối ưu hóa việc sử dụng tài nguyên hệ thống hiệu quả hơn nhiều so với các máy ảo truyền thống. Bằng cách sử dụng cơ chế sandbox cho mỗi container, Docker cũng tăng cường tính bảo mật và cô lập cho ứng dụng.

Trong dự án NEU OJ, Docker được sử dụng để "container hóa" các thành phần chính của hệ thống, bao gồm ứng dụng backend NestJS, ứng dụng frontend Next.js, cơ sở dữ liệu MongoDB và một số nền tảng hỗ trợ khác. Điều này không chỉ giúp đơn giản hóa quá trình phát triển mà còn đặt nền tảng vững chắc cho việc triển khai và mở rộng hệ thống một cách hiệu quả trong tương lai.

3.2.1.6. Giao tiếp thời gian thực Socket.IO

Để xây dựng các tính năng tương tác thời gian thực cho hệ thống NEU OJ, chẳng hạn như giám sát gian lận hay cập nhật bảng xếp hạng cuộc thi trực tiếp, dự án đã sử dụng Socket.IO. Đây là một thư viện JavaScript mạnh mẽ, giúp đơn giản hóa việc thiết lập kết nối hai chiều, liên tục và có độ trễ thấp giữa client và server.

Socket.IO không phải là một triển khai của WebSocket, mà là một thư viện cung cấp một lớp trừu tượng hóa cấp cao hơn. Nó sử dụng WebSocket làm phương tiện giao tiếp chính khi có thể, nhưng sẽ tự động "hạ cấp" xuống các kỹ thuật khác như HTTP long-polling nếu WebSocket không được hỗ trợ. Điều này đảm bảo kết nối thời gian thực có thể hoạt động ổn định trên hầu hết mọi môi trường và trình duyệt.

Trong hệ thống NEU OJ, Socket.IO được sử dụng để đẩy các cập nhật từ máy chủ xuống trình duyệt của người dùng một cách tức thì, chẳng hạn như khi có bài nộp mới trong một cuộc thi, kết quả chấm bài được cập nhật, hay bảng xếp hạng thay đổi. Điều này tạo ra một trải nghiệm sống động và hấp dẫn, đặc biệt quan trọng trong môi trường thi đấu cạnh tranh.

3.2.1.7. Bộ nhớ đệm và hàng đợi Redis

Để tối ưu hóa hiệu năng và xử lý các tác vụ nền, dự án sử dụng Redis, một kho lưu trữ dữ liệu khoá-giá trị trong bộ nhớ tốc độ cao. Trong hệ thống NEU OJ, Redis đảm nhiệm hai vai trò quan trọng: làm bộ nhớ đệm và hàng đợi tác vụ.

Với vai trò làm bộ nhớ đệm

Redis lưu trữ các dữ liệu thường xuyên được truy cập - như thông tin bài tập hay hồ sơ người dùng - vào bộ nhớ RAM. Điều này giúp tăng tốc độ phản hồi cho người dùng và giảm tải đáng kể cho cơ sở dữ liệu chính là MongoDB bằng cách hạn chế các truy vấn lặp đi lặp lại.

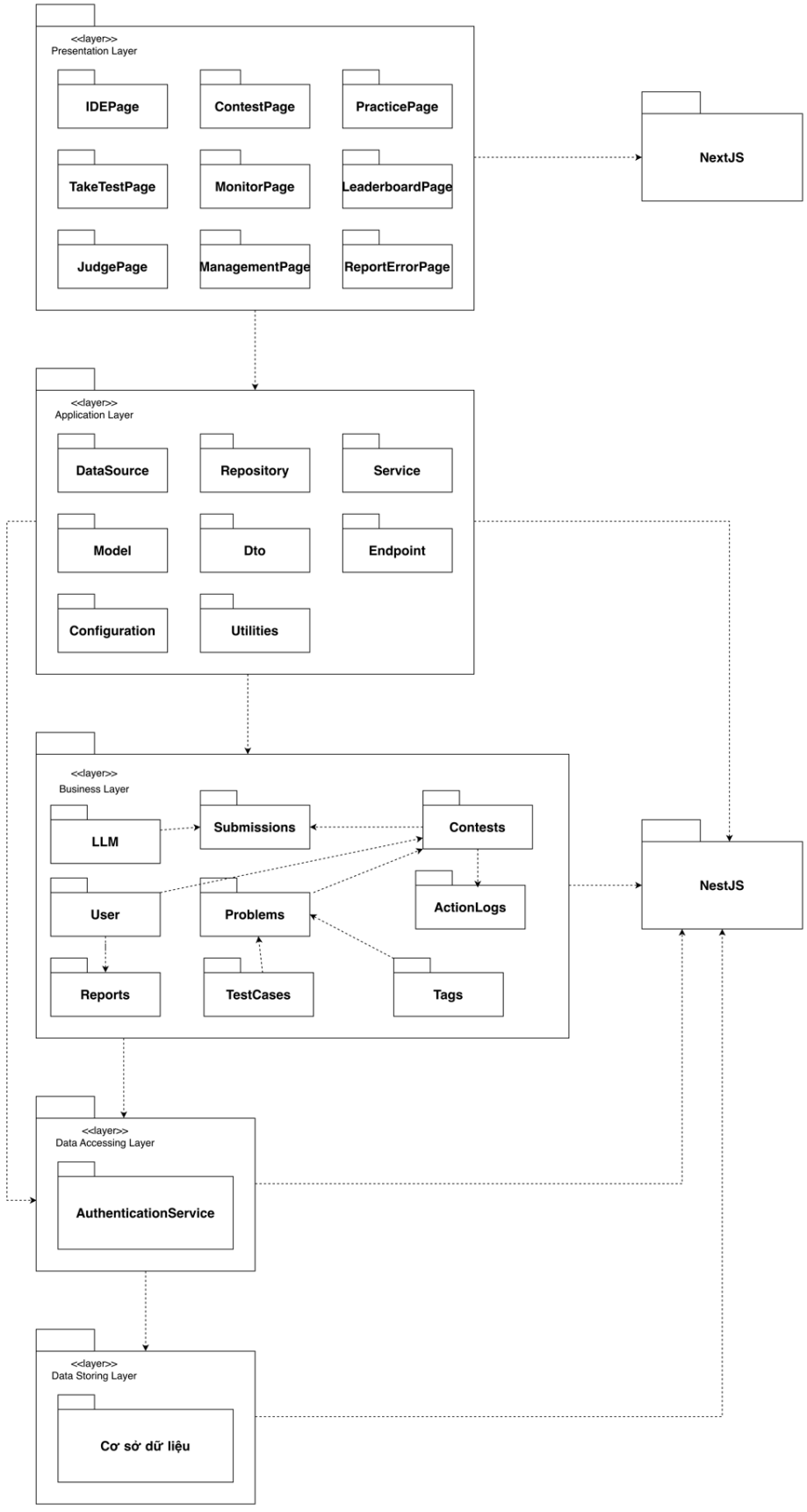
Với vai trò làm hàng chờ

Redis được dùng để quản lý các tác vụ nặng một cách bất đồng bộ. Ví dụ, khi một sinh viên nộp bài, yêu cầu chấm bài sẽ được đưa vào một hàng đợi trong Redis thay vì xử lý ngay lập tức. Một tiến trình nền sẽ lấy tác vụ từ hàng đợi để xử lý. Cách tiếp cận này giúp hệ thống phản hồi người dùng ngay lập tức và xử lý việc chấm bài một cách ổn định, tăng khả năng chịu lỗi và mở rộng của hệ thống.

3.2.1.8. Các công nghệ phụ trợ và hạ tầng

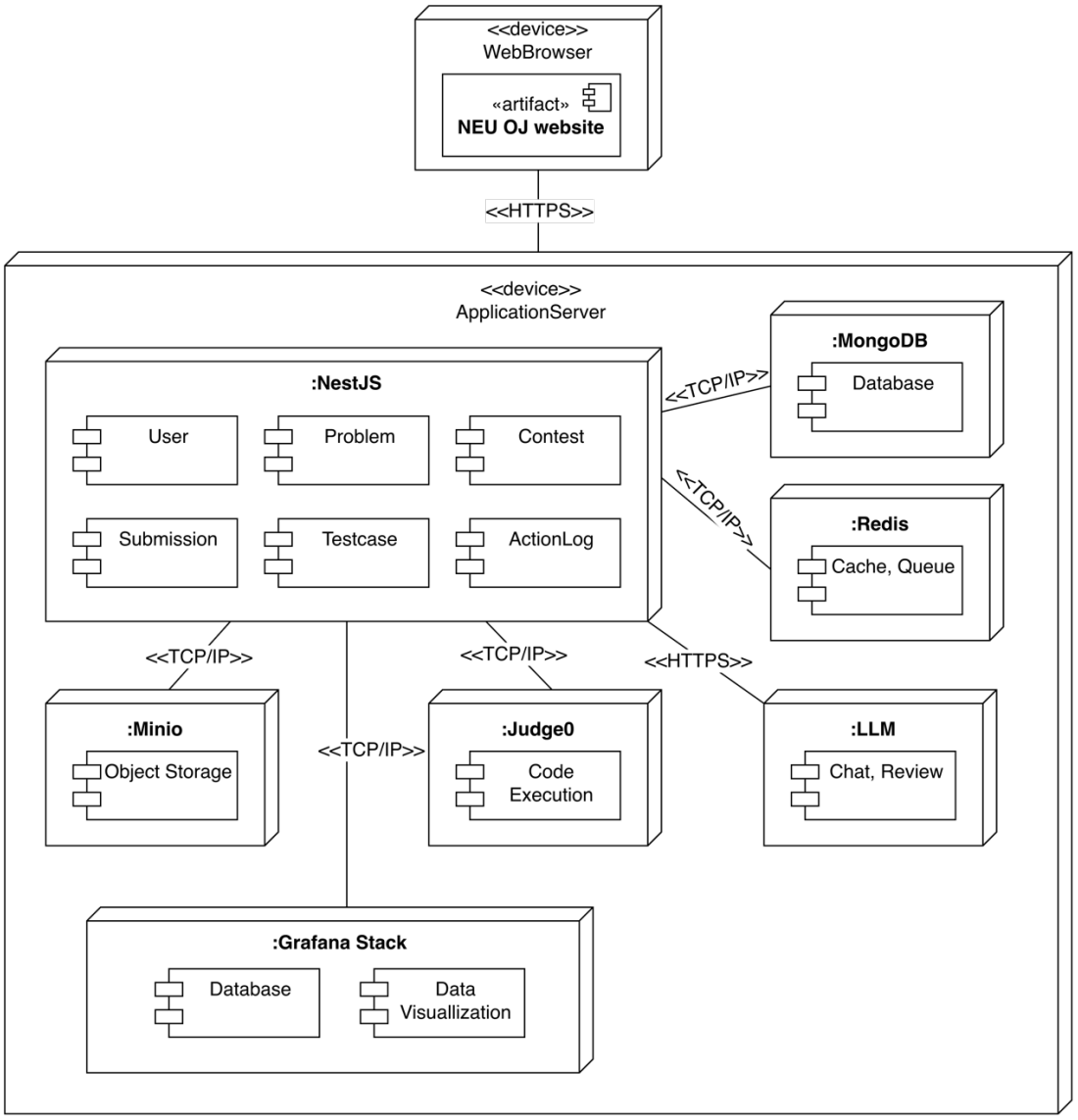
Bên cạnh các thành phần cốt lõi, hệ thống tích hợp các giải pháp chuyên biệt để giải quyết các bài toán về lưu trữ, giám sát và điều phối như MinIO - Lưu trữ dữ liệu phi cấu trúc; Grafana & Prometheus - Giám sát hệ thống và hạ tầng; Nginx - Cân bằng tải và Reverse Proxy.

3.2.2. Biểu đồ gói



Hình 3.47. Biểu đồ gói hệ thống NEU OJ

3.2.3. Biểu đồ triển khai



Hình 3.48. Biểu đồ triển khai hệ thống NEU OJ.

Triển khai hệ thống bao gồm các thành phần phân tán được tổ chức như sau:

- Máy trạm (Client Workstation): Là thiết bị cá nhân của người dùng (Laptop, PC) sử dụng trình duyệt web để truy cập vào hệ thống. Giao diện người dùng (NEU OJ Website) được xây dựng bằng Next.js.
- Máy chủ ứng dụng (Application Server): Là một VPS - máy chủ trung tâm chạy hệ điều hành Linux (Ubuntu 24.04), sử dụng nền tảng Docker để ảo hóa và quản lý các dịch vụ thành phần. Trong đó bao gồm:
    - Hệ thống Backend (NestJS): Được xây dựng bằng NestJS Framework. Đây là trái tim của hệ thống, đóng vai trò xử lý toàn bộ

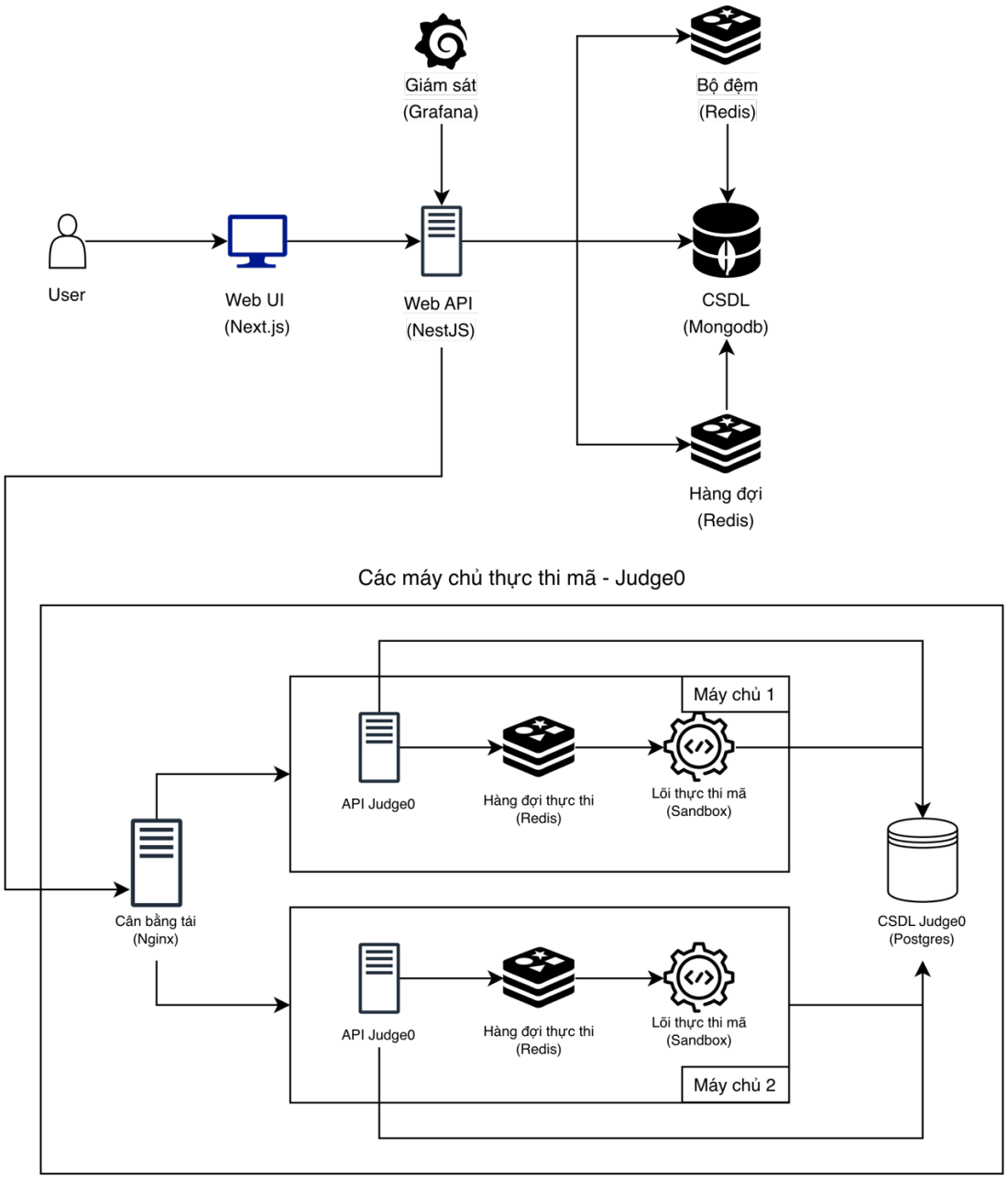
logic nghiệp vụ, quản lý các module chức năng như: người dùng, bài tập, kỳ thi và bài nộp.

- Judge Engine (Judge0): Dịch vụ chuyên biệt dùng để thực thi mã nguồn của người dùng trong môi trường cô lập, đảm bảo an toàn và hỗ trợ đa ngôn ngữ lập trình.
- Mô hình ngôn ngữ lớn (LLM): Mô hình ngôn ngữ lớn được triển khai nhằm phục vụ chức năng CodeMentor hoặc tích hợp vào quy trình nhận xét, đánh giá bài tập.
- Object Storage (MinIO): Hệ thống lưu trữ đối tượng (tương thích S3) dùng để lưu trữ các file phương tiện như đề bài dạng PDF hoặc hình ảnh.

- Cơ sở dữ liệu và Bộ nhớ đệm: Hệ thống sử dụng kết hợp các giải pháp lưu trữ để tối ưu hiệu năng:
  - MongoDB: Cơ sở dữ liệu văn bản (NoSQL) dùng để lưu trữ các dữ liệu có cấu trúc linh hoạt như thông tin người dùng, nội dung bài tập và log vi phạm.
  - Redis: Đóng vai trò là bộ nhớ đệm để tăng tốc độ truy vấn dữ liệu thường dùng, đồng thời hoạt động như một hàng đợi để điều phối các tác vụ chấm bài gửi đến Judge0, giúp hệ thống không bị quá tải khi có nhiều người nộp bài cùng lúc.
- Hệ thống giám sát:
  - Grafana Stack: Công cụ dùng để thu thập log, trực quan hóa dữ liệu và theo dõi sức khỏe của toàn bộ các container trong hệ thống theo thời gian thực.

Người dùng giao tiếp với máy chủ thông qua giao thức HTTPS an toàn (RESTful APIs). Bên trong máy chủ ứng dụng, các container (NestJS, Database, Judge0...) giao tiếp với nhau thông qua mạng nội bộ Docker sử dụng giao thức TCP/IP nội bộ để đảm bảo tốc độ cao nhất.

Sau khi triển khai, luồng dữ liệu và thứ tự các lớp thực hiện từng nhiệm vụ của hệ thống có thể được thể hiện một cách trực quan như hình dưới.



Hình 3.49. Sơ đồ kiến trúc tổng quan NEU OJ.

CHƯƠNG 4. KẾT QUẢ TRIỂN KHAI, ĐÁNH GIÁ

4.1. Môi trường triển khai và thử nghiệm

Hiện tại, toàn bộ hệ thống bao gồm dịch vụ web, CSDL, dịch vụ thực thi mã nguồn đang được triển khai trên 02 máy chủ thuộc khoa Công nghệ thông tin hỗ trợ. Các máy chủ được đặt trong khuôn viên đại học, giao tiếp với nhau bằng TCP/IP thông qua mạng nội bộ của đại học.

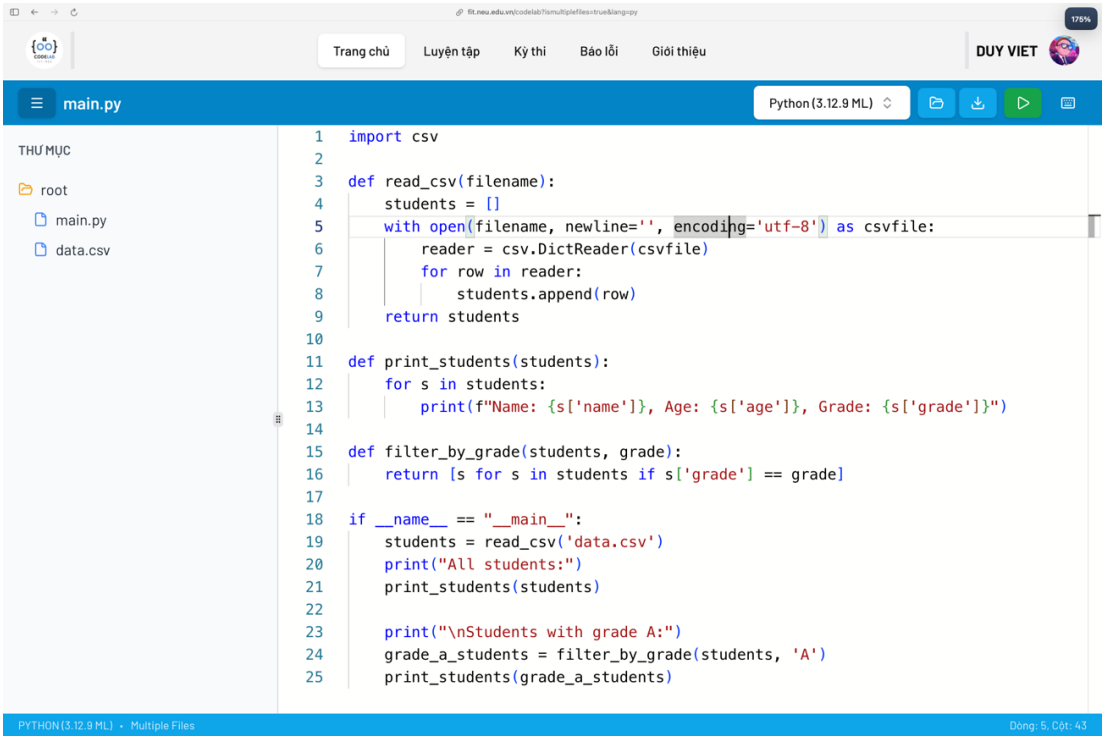
Một máy chủ sẽ đảm nhiệm vai trò triển khai dịch vụ web (front-end, back-end), CSDL và một dịch vụ Judge0 thực thi mã nguồn. Máy chủ còn lại sẽ nhận nhiệm vụ như một dịch vụ Judge0 thứ hai, chia sẻ phần công việc thực thi với dịch vụ Judge0 nằm ở máy chủ đầu tiên.

Cấu hình chi tiết của cả hai máy chủ kể trên:

- CPU: Intel Xeon Silver 4310 CPU @ 2.10GHz, 8 nhân, 16 luồng
- RAM: 32GB
- Bộ nhớ: 512GB SSD
- Băng thông mạng: 10000 Mbps
- Hệ điều hành: Ubuntu 24.04 LTS

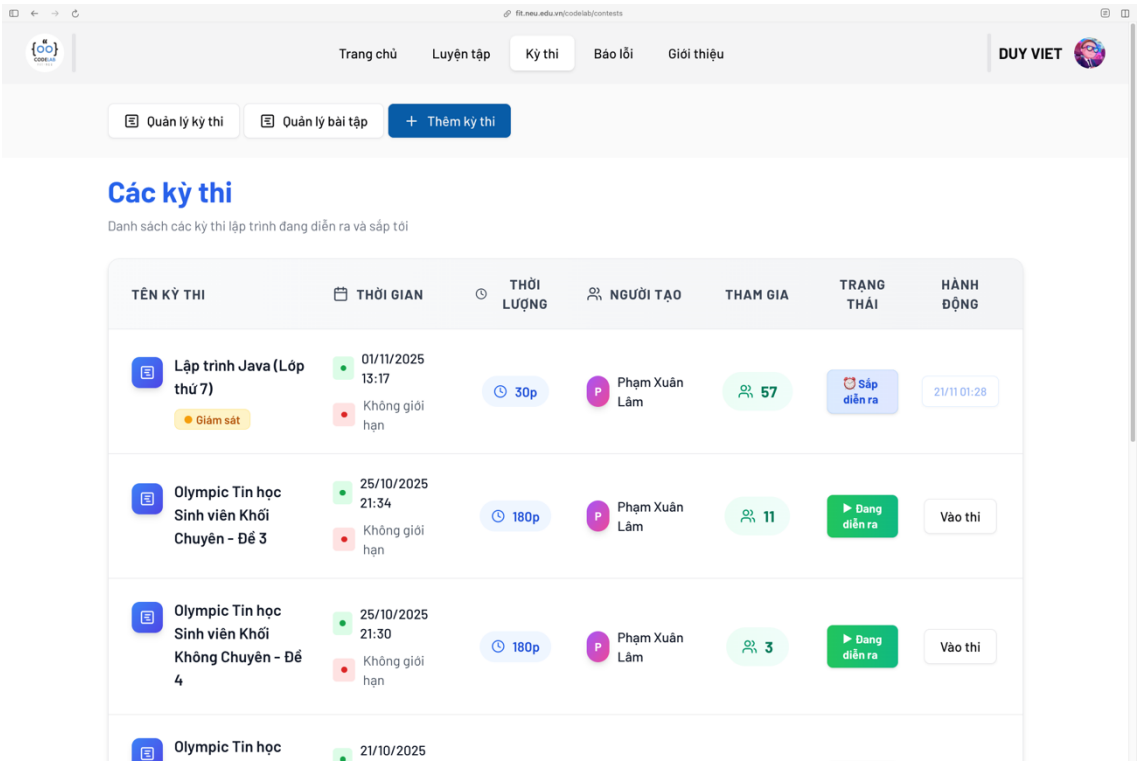
4.2. Kết quả xây dựng chức năng

Khi người dùng truy cập vào tên miền dự án, có thể nhìn thấy và sử dụng ngay IDE trực tuyến, hỗ trợ sẵn nhiều ngôn ngữ lập trình phổ biến cùng tính năng quản lý thư mục cho một số ngôn ngữ, phím tắt,…

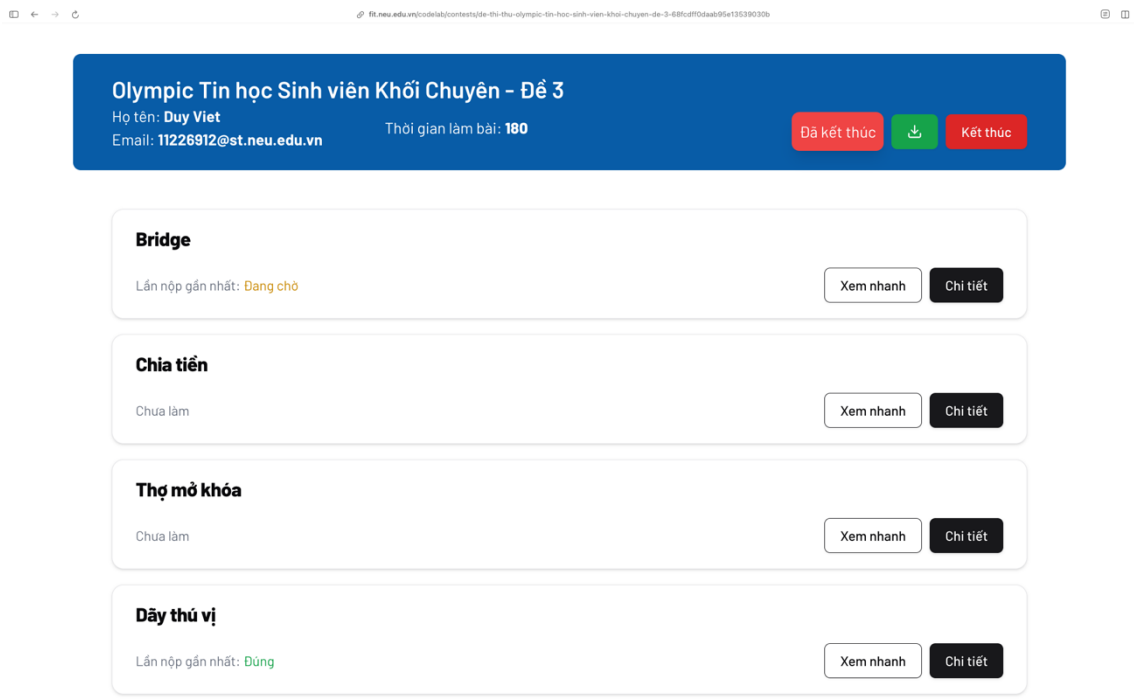


Hình 4.1. Giao diện màn hình IDE trực tuyến.

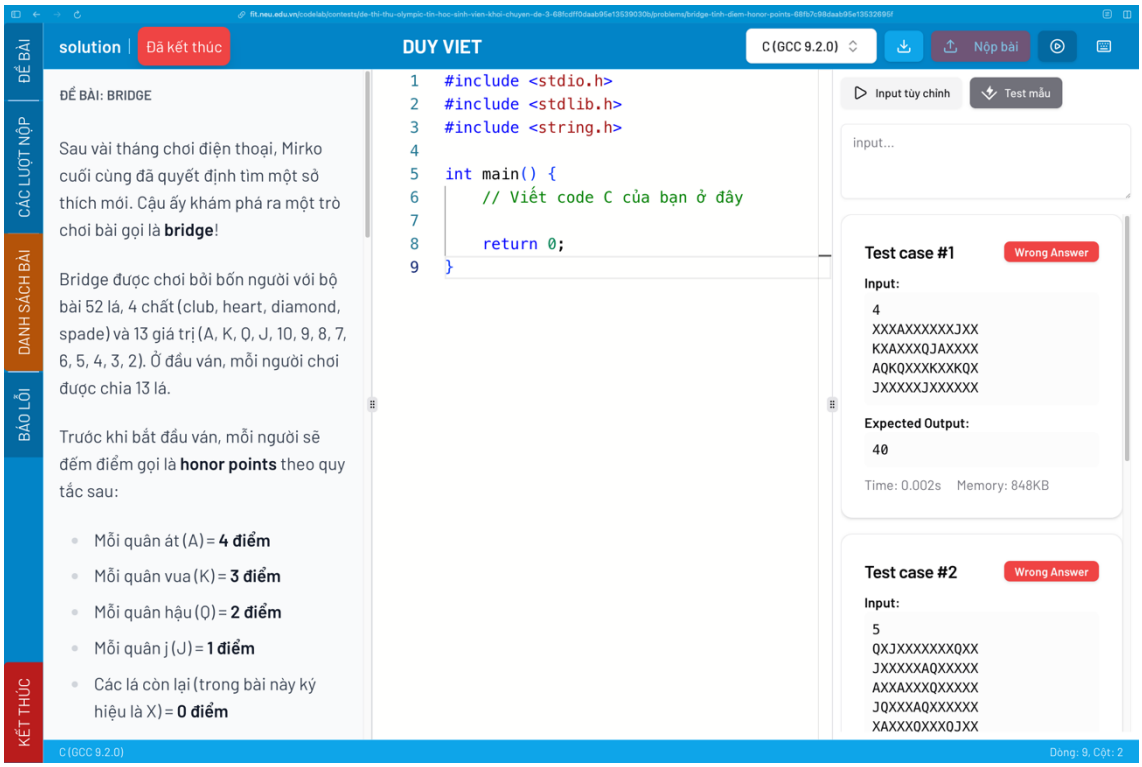
Với tính năng tham gia kỳ thi, người dùng chỉ cần truy cập vào trang "Kỳ thi" và chọn kỳ thi mong muốn.



Hình 4.2. Giao diện màn hình danh sách kỳ thi.

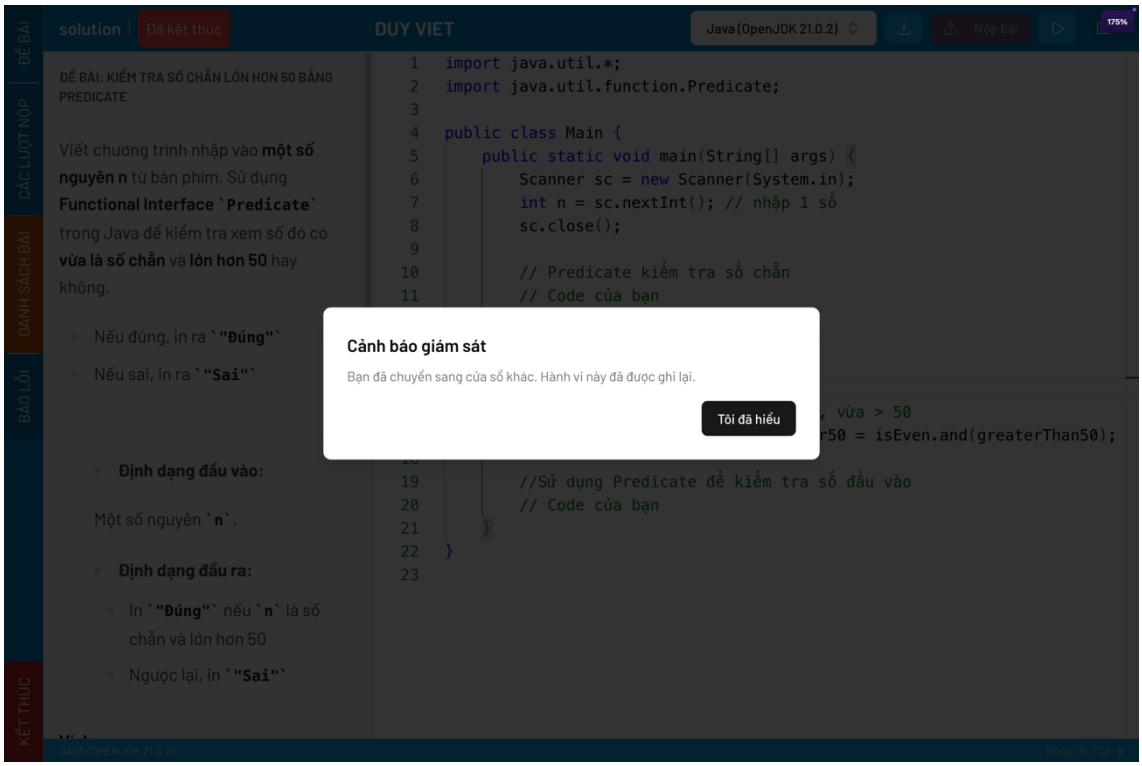


Hình 4.3. Giao diện màn hình đề thi.



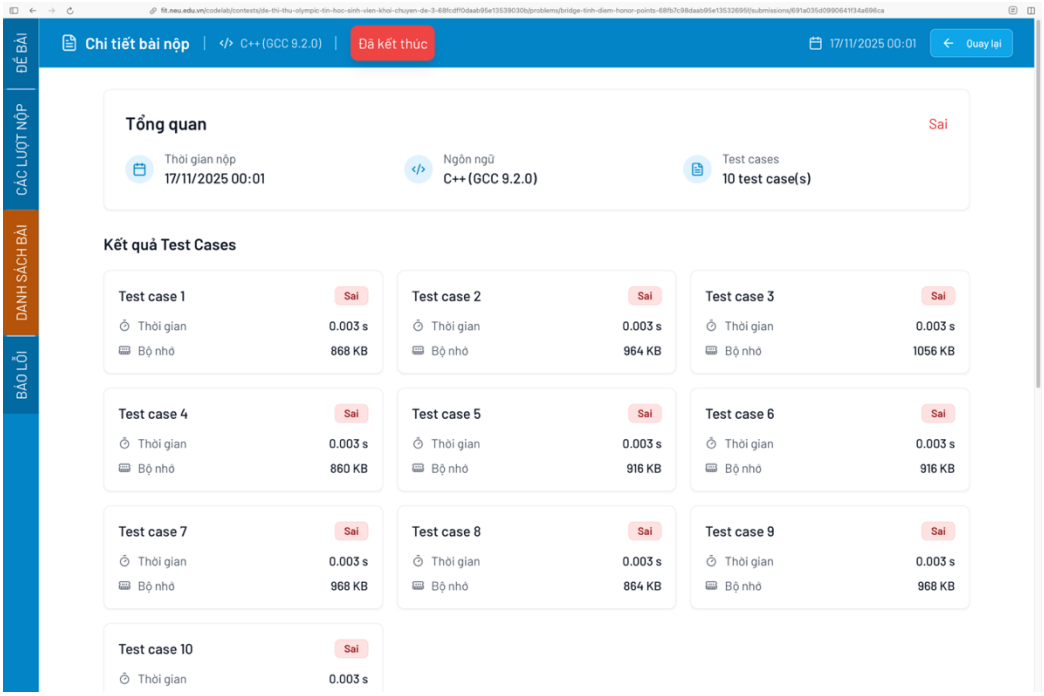
Hình 4.4. Giao diện màn hình làm bài thi.

Trong quá trình tham gia thi, nếu như người dùng có hành vi gian lận như chuyển tab, thoát khỏi cửa sổ… hệ thống sẽ ghi nhận và cảnh báo người dùng.



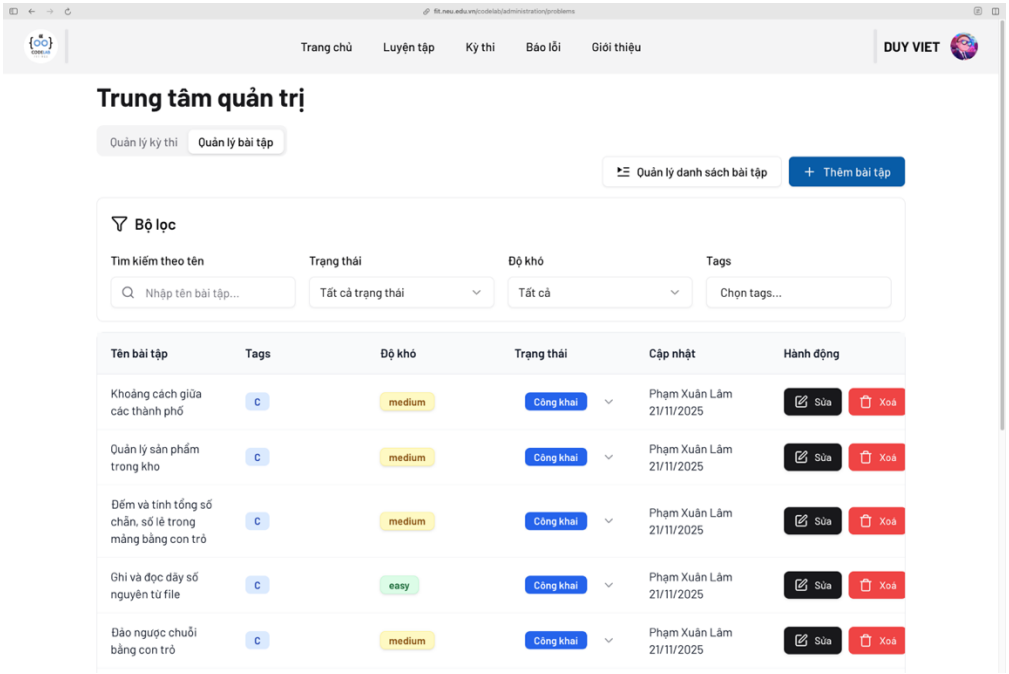
Hình 4.5. Giao diện cảnh báo gian lận.

Sau nộp bài thành công, người dùng có thể xem ngay kết quả chạy test cases cùng với các thông số khác như thời gian, bộ nhớ chiếm dụng của chương trình, ở chức năng xem lại lượt nộp.

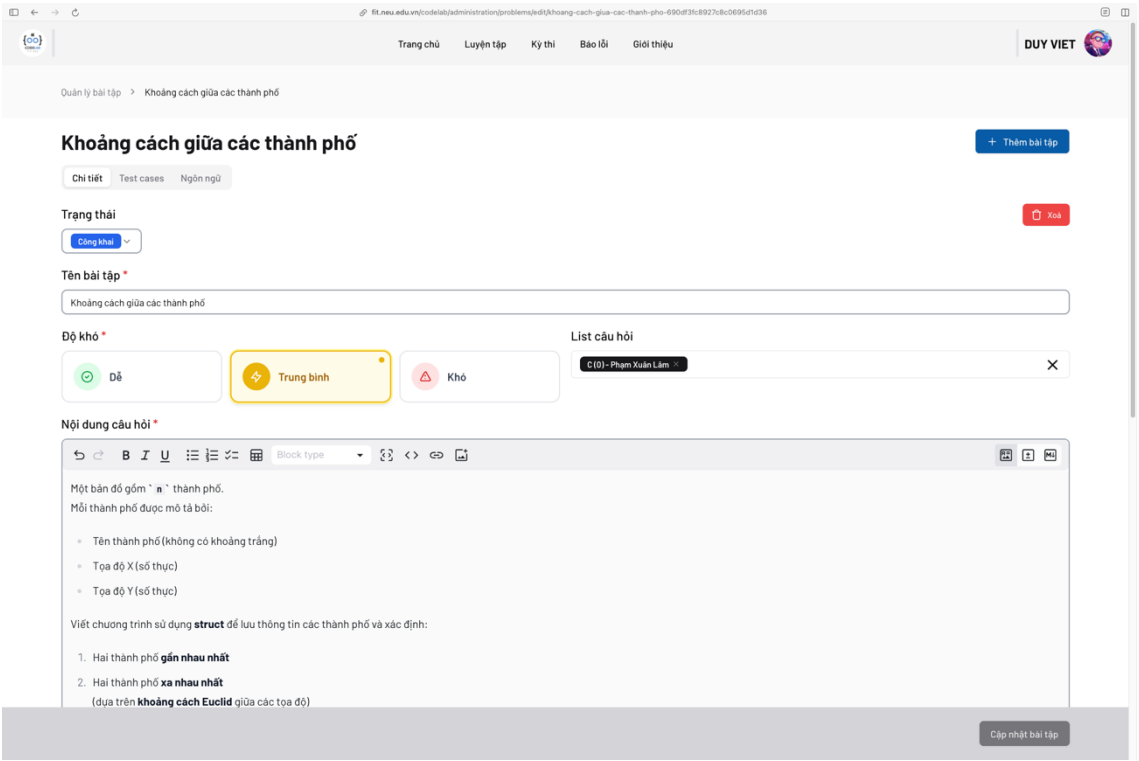


Hình 4.6. Giao diện chức năng xem kết quả nộp bài.

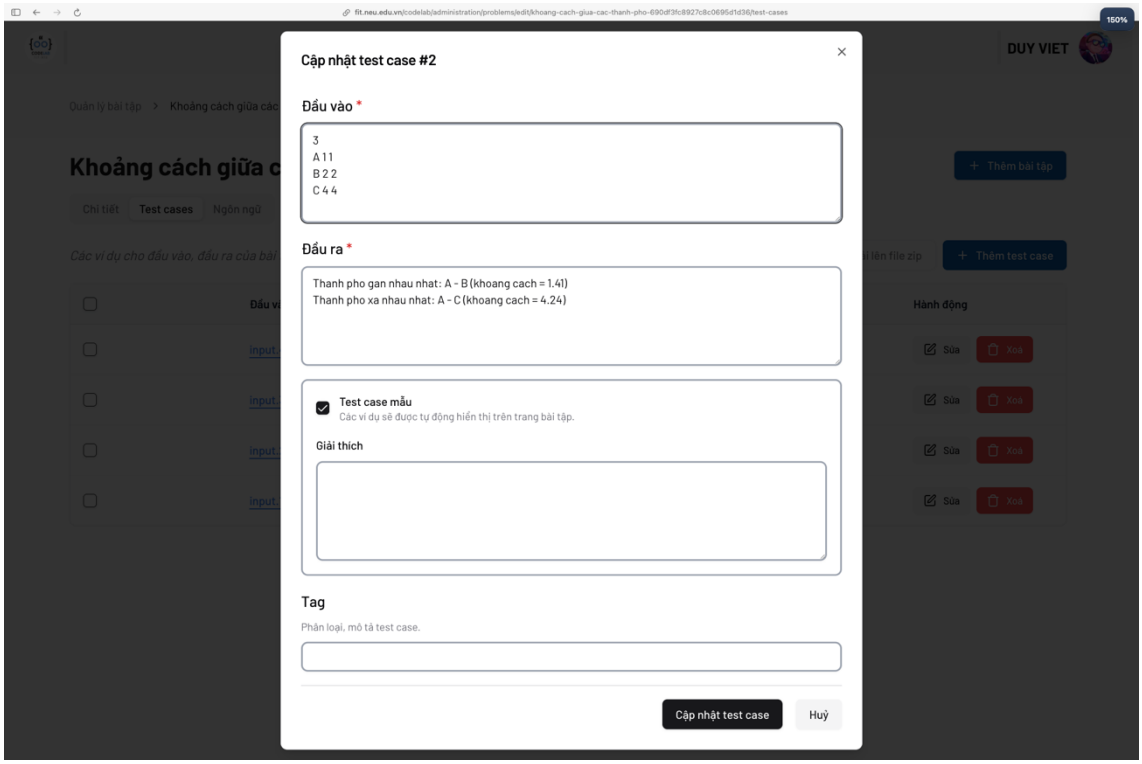
Với cụm chức năng quản lý, hệ thống đã có thể đáp ứng được nhu cầu xây dựng kho bài tập, ngân hàng đề thi cá nhân với trình quản lý đầy đủ thông tin, chức năng trạng thái và cấu hình.



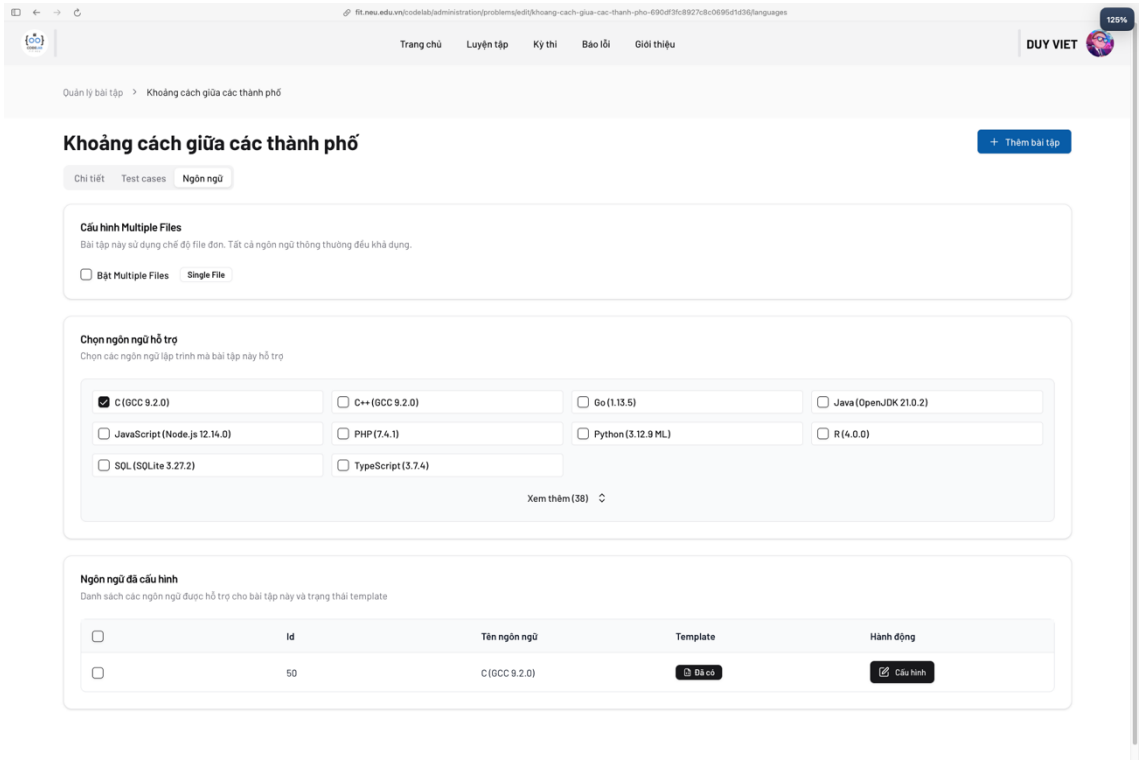
Hình 4.7. Giao diện màn hình quản lý bài tập.



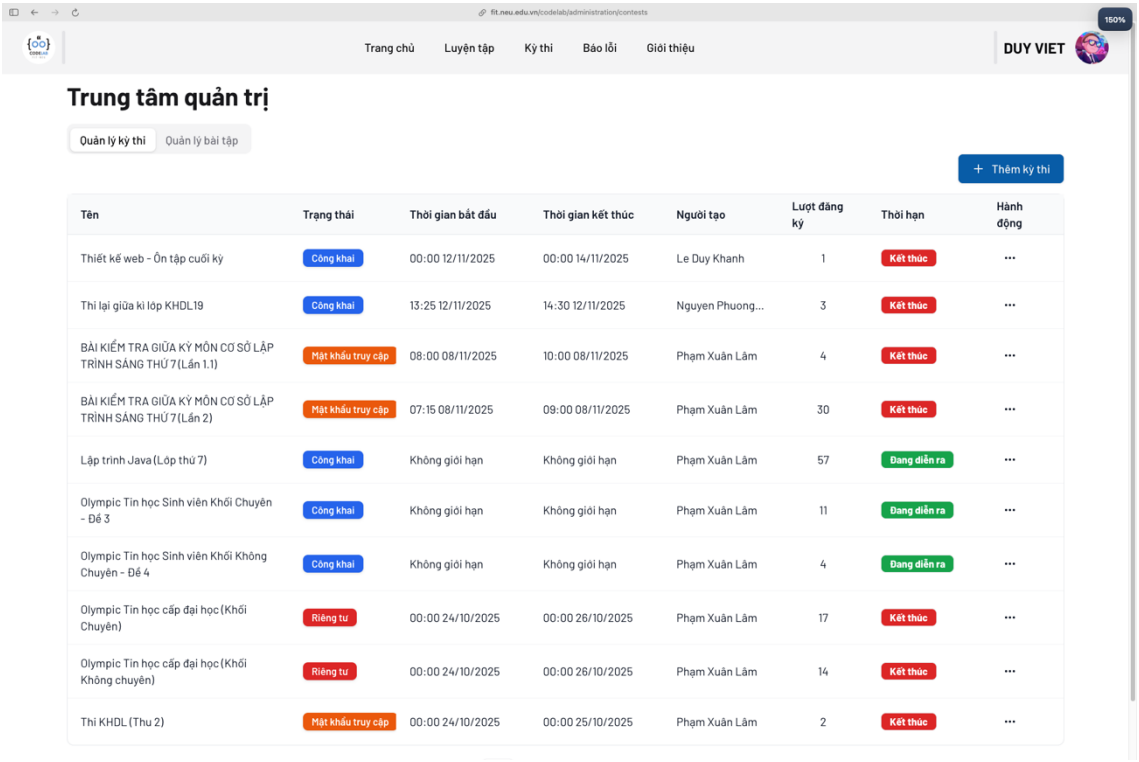
Hình 4.8. Giao diện màn hình đóng góp/cập nhật bài tập.



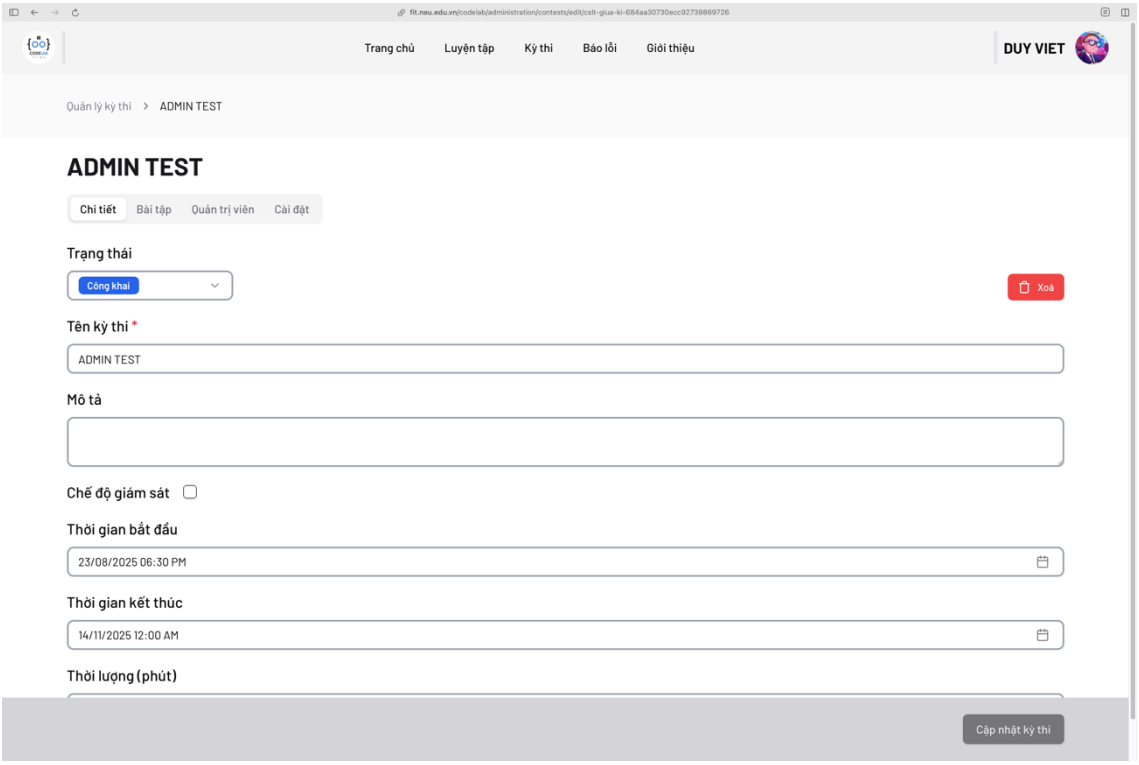
Hình 4.9. Giao diện màn hình chức năng thêm/cập nhật test case.



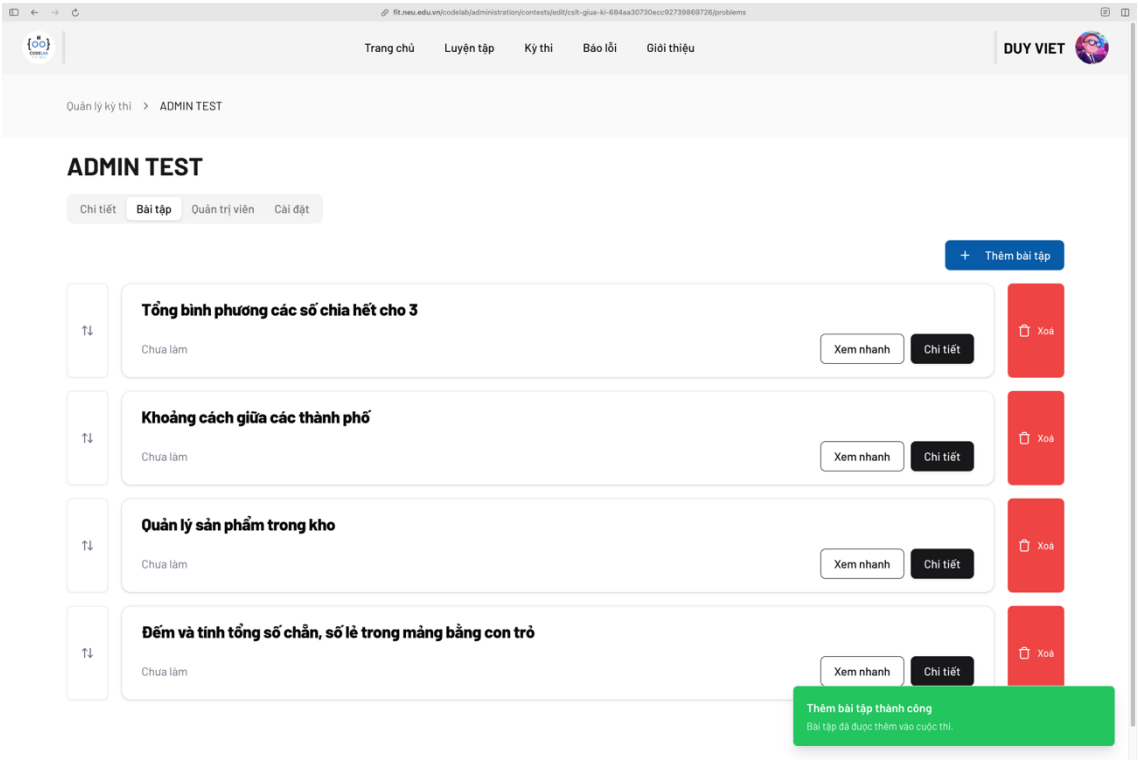
Hình 4.10. Giao diện màn hình chức năng cấu hình ngôn ngữ cho bài tập.



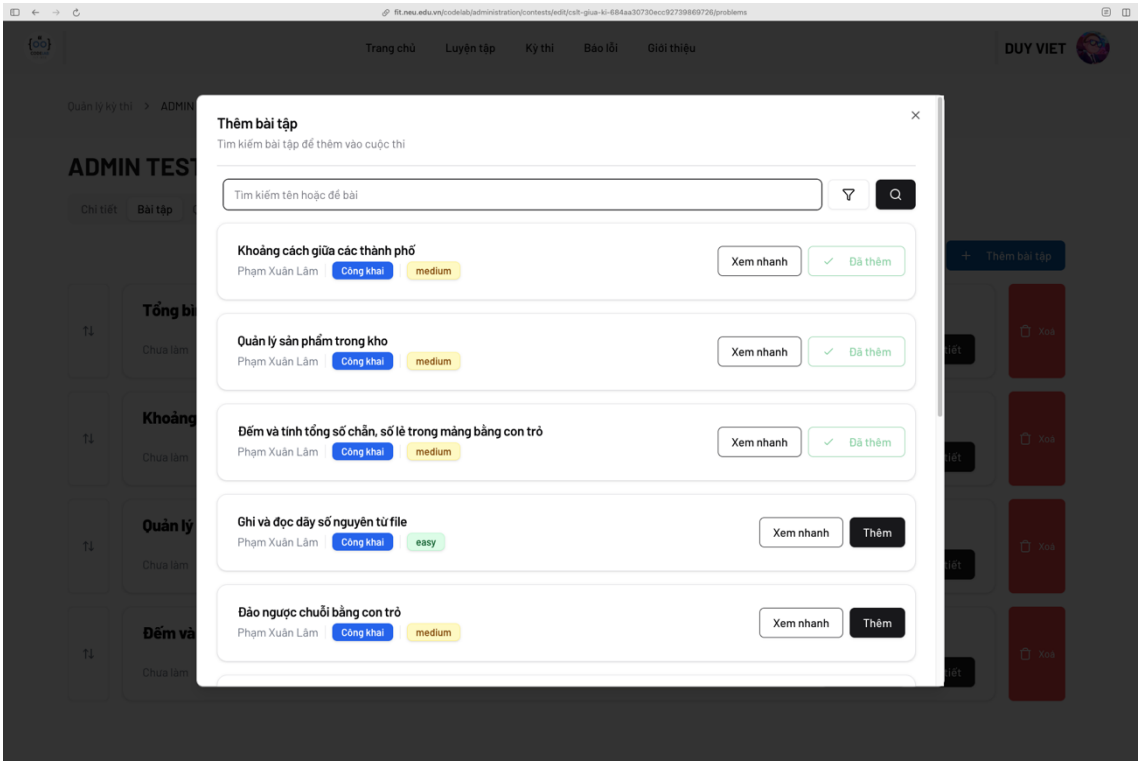
Hình 4.11. Giao diện màn hình quản lý kỳ thi.



Hình 4.12. Giao diện màn hình chức năng thêm/cập nhật kỳ thi.

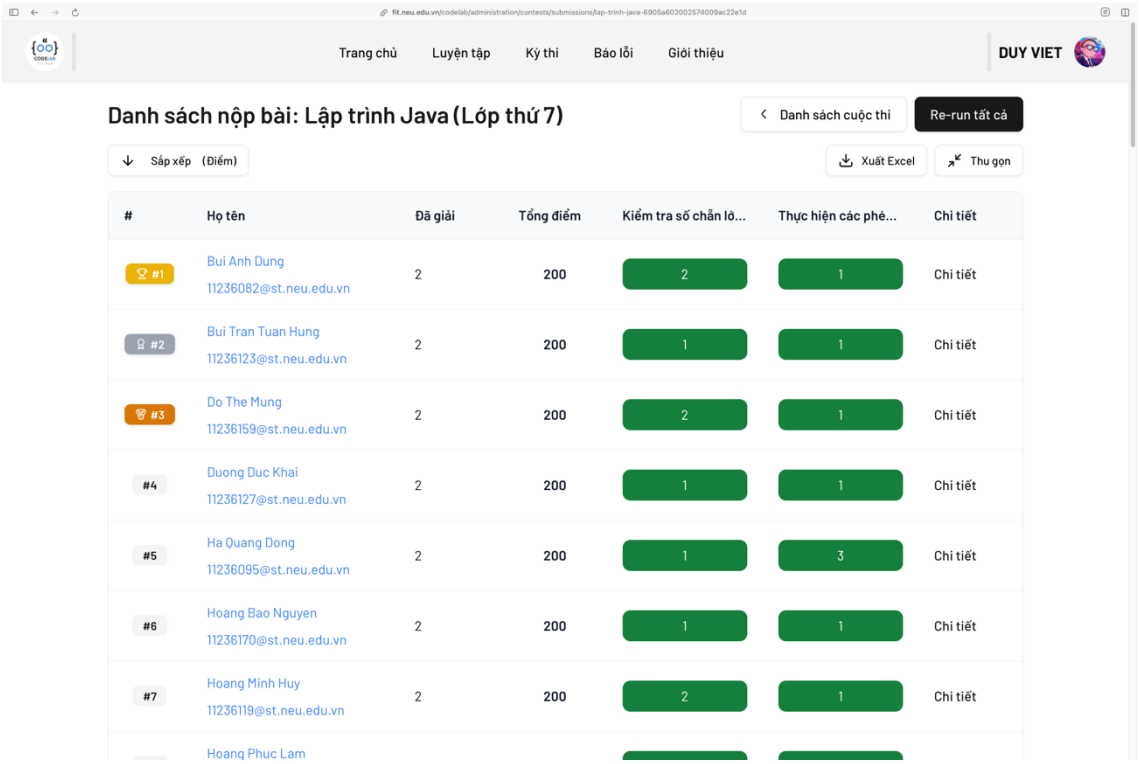


Hình 4.13. Giao diện màn hình chức năng quản lý đề thi.

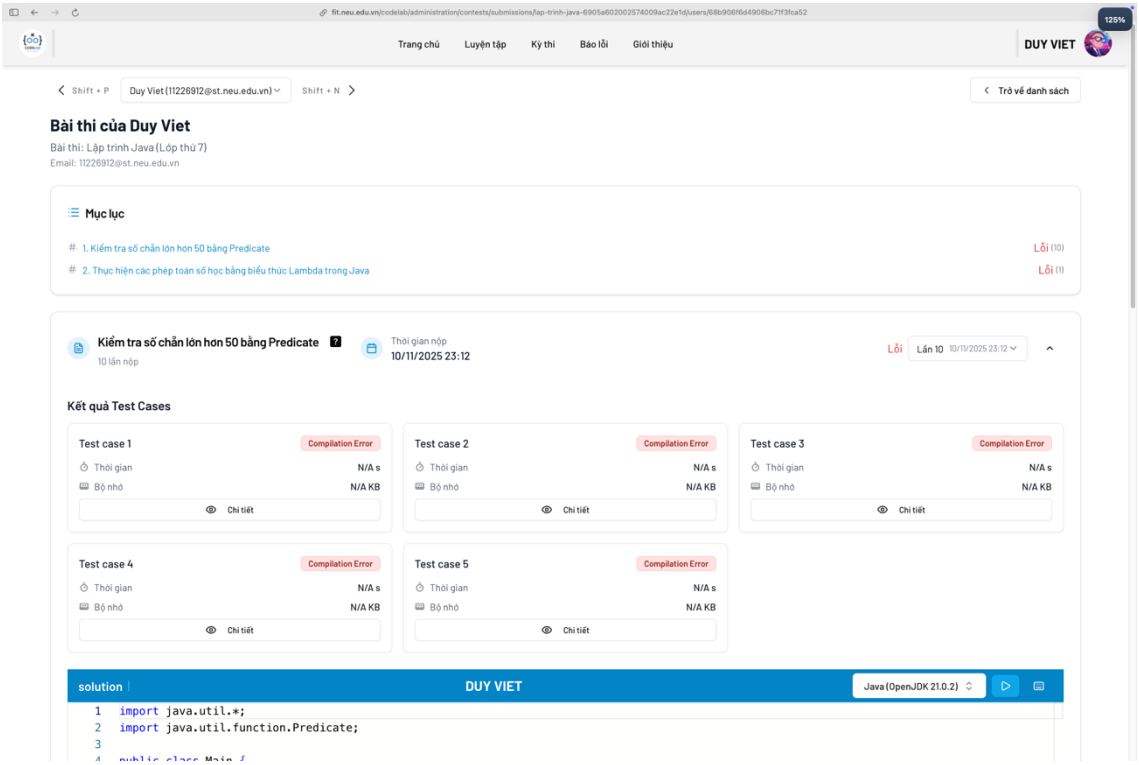


Hình 4.14. Giao diện màn hình chức năng thêm bài tập vào đề thi.

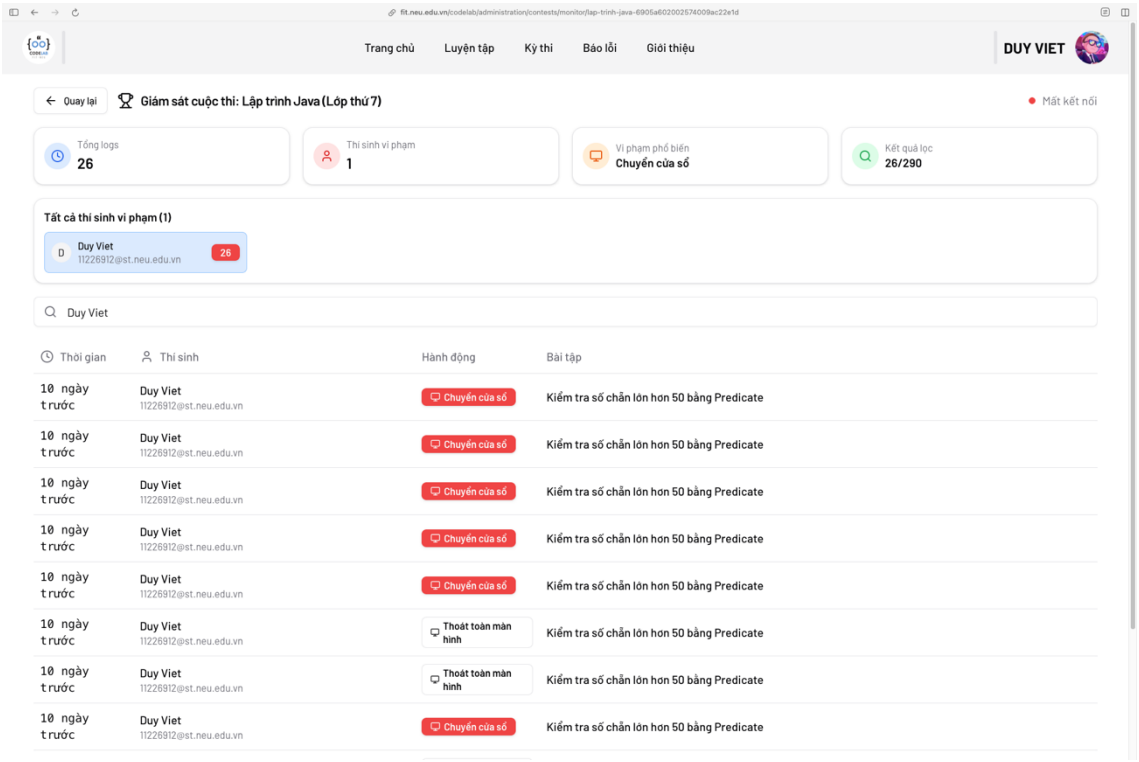
Đối với người tổ chức kỳ thi, các chức năng như leaderboard (bảng điểm), giám sát và duyệt bài nộp cũng đáp ứng về mặt thống kê, trực quan và tiện lợi khi sử dụng với các chức năng hỗ trợ xuất danh sách, sắp xếp theo từng tiêu chí.



Hình 4.15. Giao diện màn hình tổng quan điểm số kỳ thi.

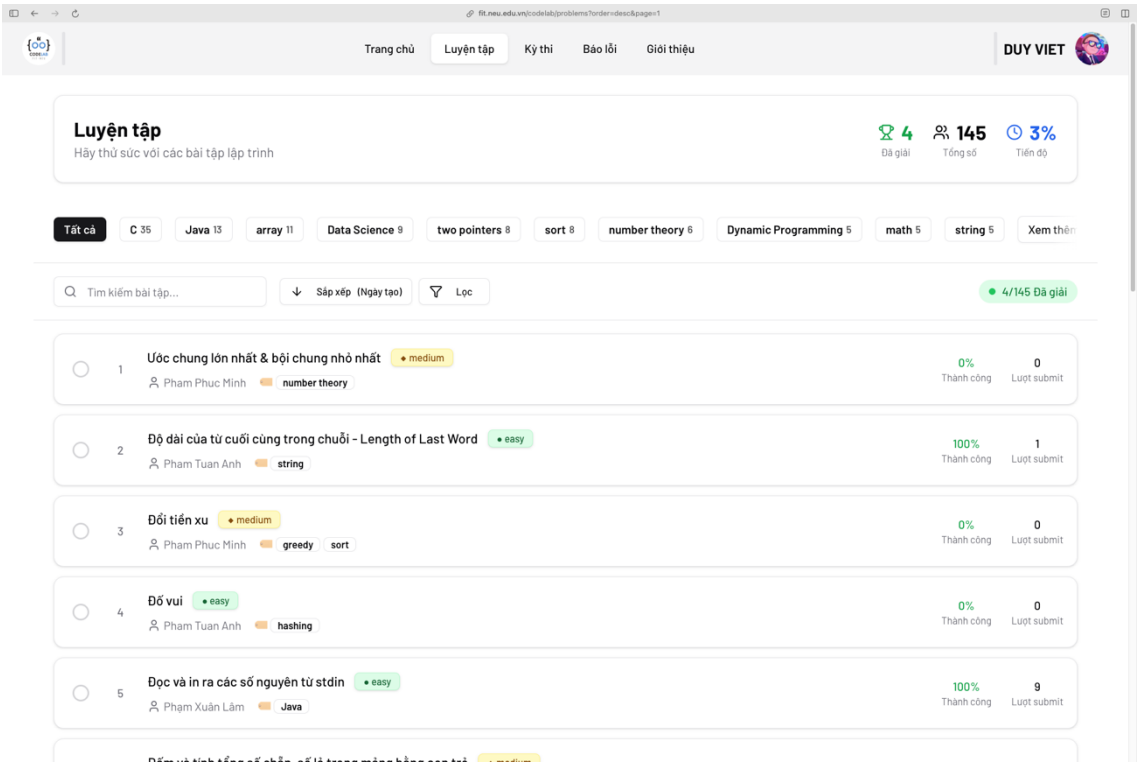


Hình 4.16. Giao diện màn hình chi tiết bài thi của người tham gia.



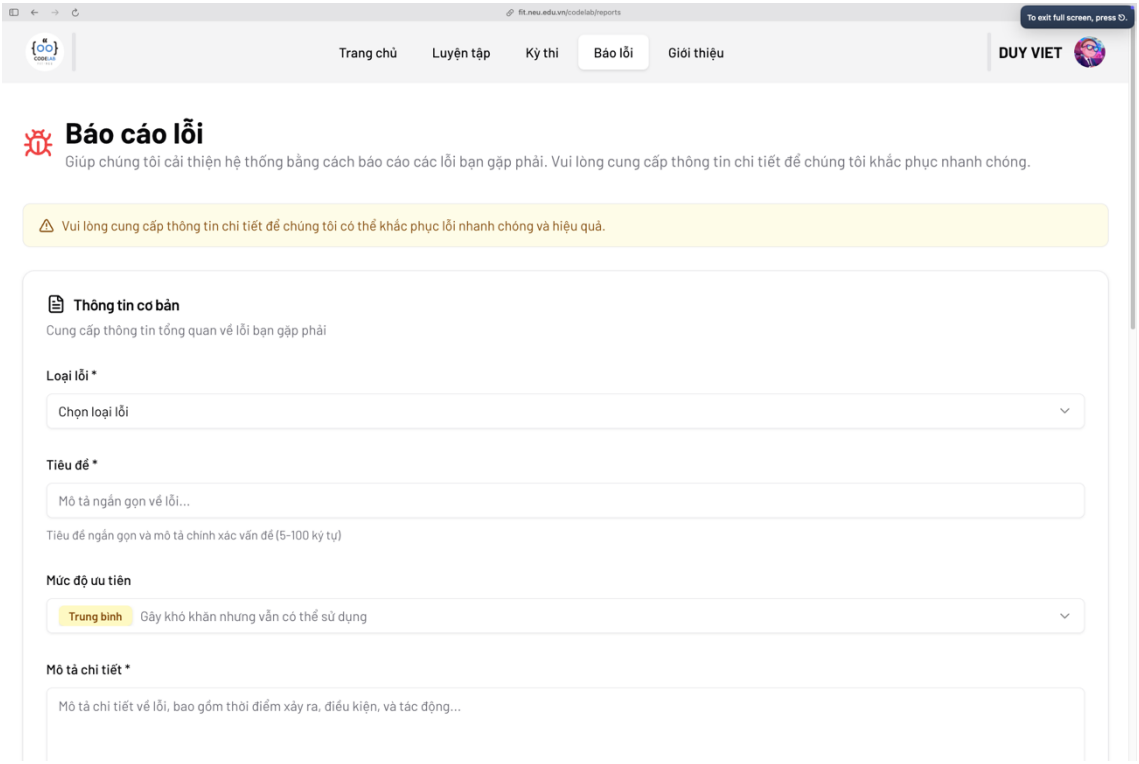
Hình 4.17. Giao diện màn hình chức năng giám sát kỳ thi.

Với nhu cầu luyện tập, hệ thống cũng đã cung cấp chức năng kho bài tập với nhiều chủ đề do cộng đồng đóng góp, với các thông số thống kê cơ bản cho cá nhân từng người dùng.



Hình 4.18. Giao diện chức năng luyện tập.

Hệ thống cũng triển khai module báo lỗi để người dùng có thể gửi thông tin khi gặp vấn đề trong quá trình sử dụng, điều này giúp đội ngũ phát triển kịp thời và nhanh chóng giải đáp, truy vết lỗi để sửa và cập nhật hệ thống.



Hình 4.19. Giao diện chức năng báo lỗi.

4.3. Đánh giá hiệu năng hệ thống

4.3.1. Phương pháp và công cụ đo lường

Hệ thống sẽ trải qua kiểm thử chịu tải bằng cách giả lập một số lượng yêu cầu liên tục thao tác thực thi một đoạn mã Python với một lượng người dùng đồng thời. Công cụ để triển khai kịch bản này là Loctus – một thư viện Python cho phép cấu hình và thực thi giả lập các yêu cầu HTTP đến máy chủ yêu cầu.

Đoạn mã python sẽ được dùng để chạy kiểm thử chịu tải như hình dưới. Lượng thời gian cần để chạy chương trình sẽ là trong khoản từ 0.6 – 0.8 giây/lượt.

```python
import time
from random import normalvariate
import numpy as np
N = 1_000_000

samples_py_rng = [normalvariate(0, 1) for _ in range(N)]
t1 = time.time()
print(f"Python random: {t1 - t0:.4f} s")

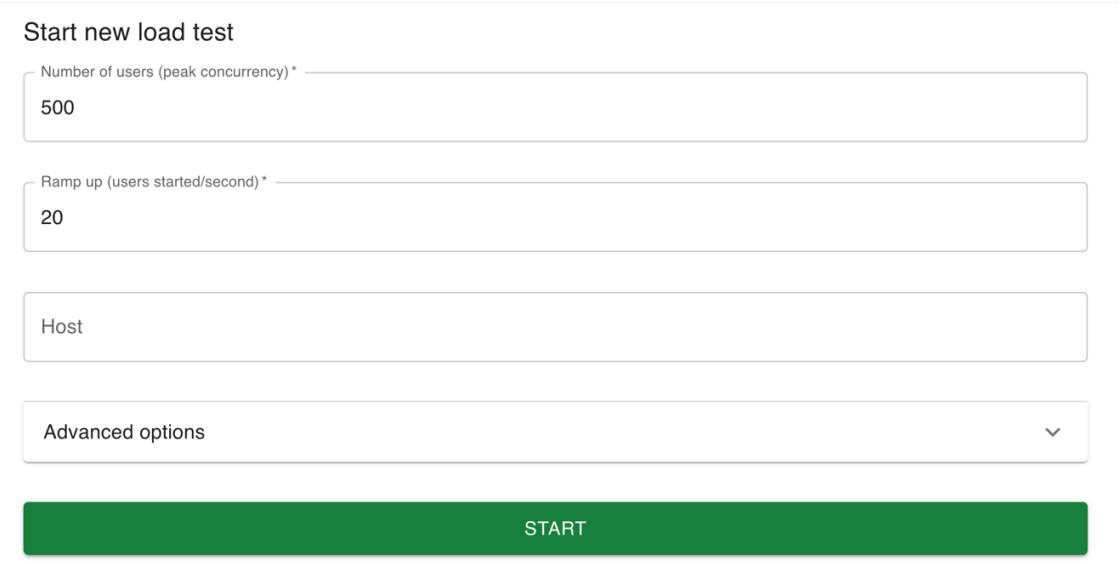
t0 = time.time()
samples_np_rng = np.random.standard_normal(N)
t1 = time.time()
print(f"NumPy random: {t1 - t0:.4f} s")
```

Hình 4.20. Mã Python sử dụng để chạy kiểm thử chịu tải trên NEU OJ.

```
Python random: 0.5563 s
NumPy random: 0.0627 s
```

Hình 4.21. Kết quả thực thi đoạn mã kiểm thử chịu tải.

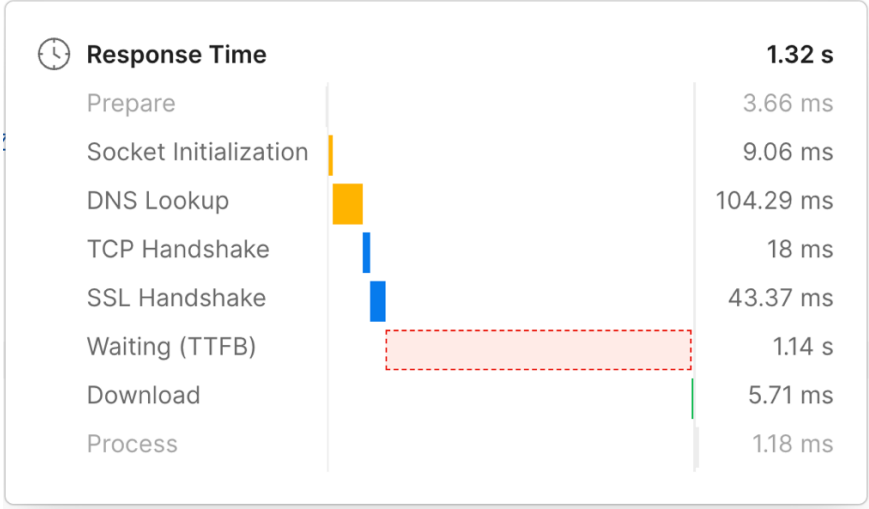
Cấu hình kịch bản kiểm thử cho Loctus với lần lượt 1000, 1500, 2000 người dùng đồng thời, bắt đầu từ 0 và mỗi giây tăng 100 người. Kịch bản được thực hiện với cấu trúc hệ thống phân tán đã triển khai với 02 máy chủ thực thi mã.



Hình 4.22. Cấu hình kịch bản kiểm thử với Loctus.

4.3.2. Đánh giá thời gian xử lý

Kết quả thống kê thực tế đối với một yêu cầu chấm bài đơn lẻ cho thấy tổng thời gian phản hồi trung bình là 1.32 giây. Trong đó, phần lớn thời gian (1.14 giây) được dành cho quy trình xử lý phía máy chủ, bao gồm việc tiếp nhận yêu cầu, khởi tạo môi trường cô lập, biên dịch và thực thi mã nguồn. Nếu loại trừ thời gian chạy thực tế của chương trình python nói trên (0.6 đến 0.8 giây), độ trễ hệ thống để vận hành quy trình chấm tự động chỉ dao động trong khoảng 0.3 đến 0.4 giây. Điều này chứng minh kiến trúc hệ thống hoạt động hiệu quả với độ trễ khởi động thấp, đảm bảo phản hồi nhanh chóng cho người dùng trong điều kiện vận hành thông thường.



Hình 4.23. Thời gian phản hồi với 1 yêu cầu.

4.3.3. Kiểm thử chịu tải

Sau khi thực hiện 3 thử nghiệm lần lượt với 1000, 1500 và 2000 người dùng đồng thời, ta có thể nhận thấy 1500 người dùng đồng thời là mức tải tối đa của hệ thống, trước khi xuất hiện lượng lớn kết quả thất bại tại mức 2000 người dùng.

Bảng 2. Kết quả kiểm thử chịu tải theo các mức tải.

Mức tải (Người)	Thông lượng (RPS)	Thời gian phản hồi	Tỷ lệ lỗi
1000	~16.9 req/s	~3,488 ms (3.4s)	0%
1500	~17.7 req/s	~3,670 ms (3.6s)	0%

2000	~58.9 req/s	~23,606 ms (23.6s)	~41.7% (696/1669)

Type	Name	# Requests	# Fails	Average (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/submissions? base64_encoded=true&wait=true	1747	0	3488.53	251.89	16.9	0
	Aggregated	1747	0	3488.53	251.89	16.9	0

Hình 4.24. Kết quả thống kê chịu tải tại 1000 người dùng đồng thời.

Type	Name	# Requests	# Fails	Average (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/submissions? base64_encoded=true&wait=true	1731	0	3670.14	251.89	17.7	0
	Aggregated	1731	0	3670.14	251.89	17.7	0

Hình 4.25. Kết quả thống kê chịu tải tại 1500 người dùng đồng thời.

Type	Name	# Requests	# Fails	Average (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/submissions? base64_encoded=true&wait=true	1669	696	23605.53	146.86	58.9	2.5
	Aggregated	1669	696	23605.53	146.86	58.9	2.5

Hình 4.26. Kết quả thống kê chịu tải tại 2000 người dùng đồng thời.

Khi tăng lượng người dùng lên 50% (từ 1000 lên 1500), thông lượng xử lý (RPS) hầu như đi ngang (tăng nhẹ từ 16.9 lên 17.7). Điều này cho thấy hệ thống đã đạt công suất xử lý tối đa của cụm máy chấm (Judge Cluster) ở mức ~18 bài nộp/giây. Thời gian phản hồi tăng nhẹ (từ 3.4s lên 3.6s) do các yêu cầu phải chờ lâu hơn trong hàng đợi, nhưng vẫn nằm trong ngưỡng chấp nhận được.

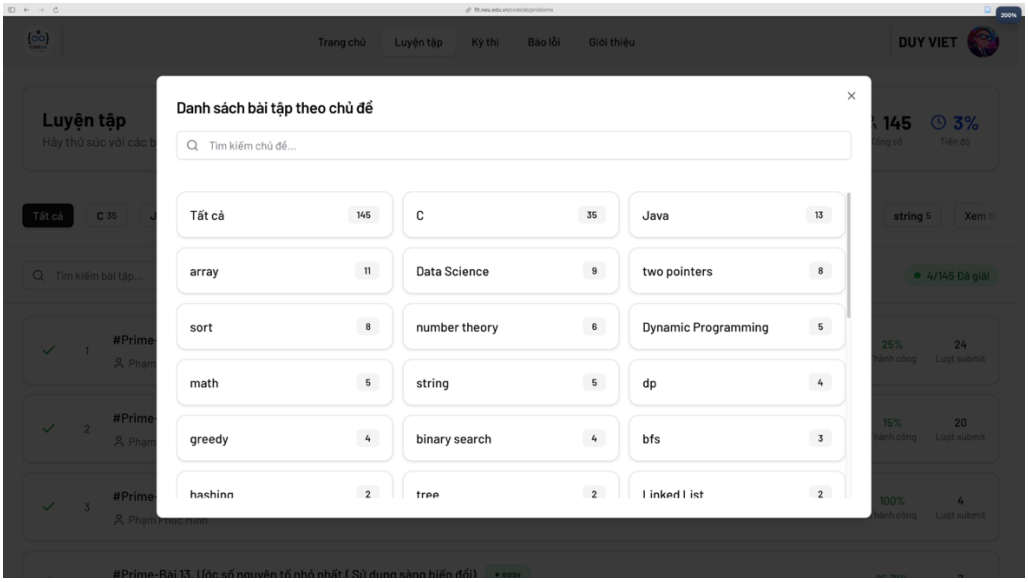
Khi mức tải chạm mốc 2000 người dùng, hệ thống vượt quá giới hạn chịu đựng, Thời gian phản hồi trung bình nhảy vọt lên 23.6 giây (tăng gấp gần 7 lần), cho thấy tình trạng ùn ứ nghiêm trọng tại hàng đợi xử lý. Hệ thống ghi nhận 41.7% yêu cầu thất bại. Các lỗi này chủ yếu do quá thời gian chờ hoặc từ chối kết nối từ phía máy chủ.

Như vậy, hệ thống NEU OJ trên hạ tầng thử nghiệm hiện tại vận hành tối ưu ở mức dưới 1500 người dùng đồng thời. Để phục vụ quy mô lớn hơn (trên 2000

người), cần mở rộng hệ thống theo chiều ngang bằng cách thêm số lượng workers (Judge0) chấm bài để giải tỏa nút thắt cổ chai tại khâu thực thi mã nguồn.

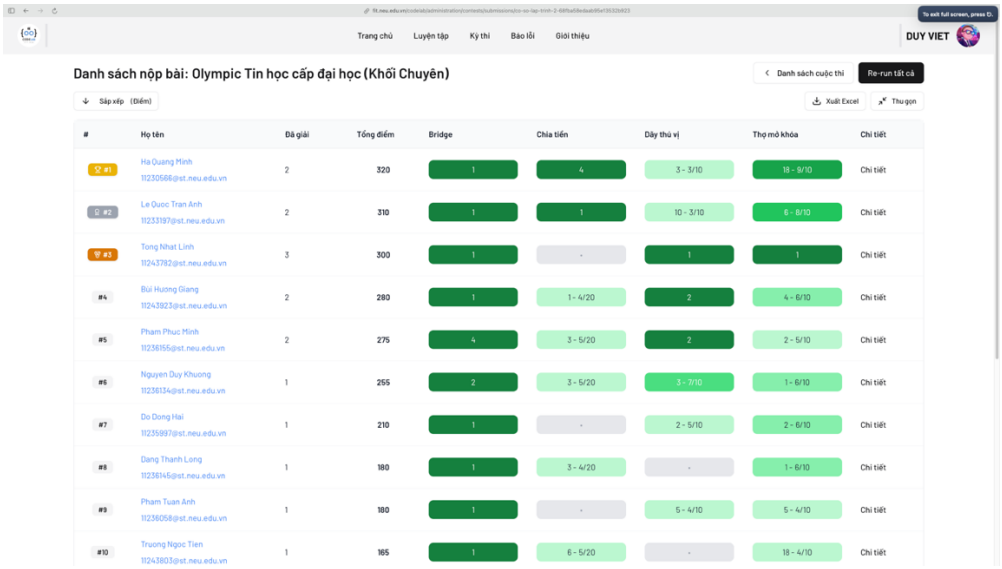
4.4. Kết quả triển khai thực tế

Hiện tại, hệ thống đã được triển khai phục vụ việc ôn luyện các môn học, kỳ thi liên quan đến lập trình của sinh viên như C, Java, kỳ thi ICPC, Olympic tin học,… thu hút được hơn 750 người dùng trong đó có gần 20 giảng viên. Khoảng 250 bài tập đa dạng đã được đóng góp bởi cả người dạy và người học.



Hình 4.27. Các chủ đề ôn luyện hiện có trên hệ thống.

Một số đợt kiểm tra giữa kỳ các môn học, kỳ thi cũng đã được tổ chức bằng NEU OJ, đặc biệt là kỳ thi Olympic tin học sinh viên cấp đại học.



Hình 4.28. Leaderboard cuộc thi Olympic Tin học cấp đại học khối chuyên.

Với tổng số sinh viên tham gia thực tế khoảng 30 người bao gồm hai khối chuyên và không chuyên tại cùng một thời điểm, hệ thống đã thực hiện tổng cộng 9.212 lượt thực thi mã nguồn trong trạng thái ổn định, phản hồi nhanh chóng.

4.5. Đánh giá tổng thể, hạn chế và rủi ro

4.5.1. So sánh tính năng giữa NEU OJ và các hệ thống phổ biến.

Để có cái nhìn khách quan về giá trị thực tiễn của hệ thống, Bảng 3 dưới đây thực hiện so sánh đối sánh các tính năng chính giữa nền tảng NEU OJ vừa xây dựng với các giải pháp đang phổ biến trên thị trường (nhóm thương mại và nhóm mã nguồn mở truyền thống). Kết quả cho thấy NEU OJ đã giải quyết tốt bài toán cân bằng giữa trải nghiệm người dùng hiện đại, khắc phục được các điểm yếu về giao diện và khả năng giám sát của các hệ thống cũ.

Bảng 3. Bảng so sánh tính nữa giữa NEU OJ với các hệ thống khác.

Tiêu chí / Chức năng	Hệ thống thương mại (Codeforces, LeetCode)	Hệ thống thi đấu cũ (CMS, DOMjudge)	NEU OJ (Hệ thống đề tài)
Mã nguồn mở & Tùy biến	×	✓	✓
Chi phí triển khai	Cao (Gói Enterprise)	Miễn phí	Thấp
Giao diện hiện đại	✓	×	✓
Giám sát thi cử trực quan (Real-time)	× (Hạn chế ở bản Free)	× (Yêu cầu mạng LAN)	✓ (Cảnh báo chuyển tab)
Chấm bài tự động đa ngôn ngữ	✓	✓	✓
Báo cáo lỗi tích hợp	✓	×	✓
Kiến trúc dễ mở rộng	✓	×	✓

4.5.2. Ưu điểm

- Đáp ứng đầy đủ các chức năng cốt lõi của một hệ thống OJ hiện đại như tổ chức và giám sát kỳ thi, quản lý bài tập, IDE...
- Giao diện thân thiện hơn các hệ thống cũ.
- Đã được triển khai thực tế thành công với độ ổn định cao.

4.5.3. Hạn chế

- Vẫn còn hạn chế khi lượng người dùng quá lớn (> 2000), sẽ cần thực thi thêm những giải pháp cân bằng tải nâng cao hơn và mở rộng hệ thống theo chiến thuật phù hợp.
- Chưa triển khai được các tính năng AI như CodeMentor – hỏi đáp kiến thức cơ bản, cú pháp code hay quy trình đánh giá, chấm bài kết hợp với mô hình ngôn ngữ lớn. Điều này có thể khắc phục nhanh chóng nếu sở hữu cơ sở vật chất có thể phục vụ các mô hình ngôn ngữ lớn mã nguồn mở, hay có chi phí để sử dụng các mô hình ngôn ngữ lớn bên thứ 3.
- Giao diện IDE chưa thể đáp ứng responsive trên các màn hình quá nhỏ như điện thoại, thiết bị cầm tay. Đội ngũ phát triển sẽ cần lắng nghe nhu cầu từ người dùng để cải thiện đem lại trải nghiệm tốt nhất.
- Gặp hạn chế về các bài tập thao tác với tệp tin, chưa thể đồng bộ quá trình sinh tệp với hiển thị trong trình quản lý tệp. Mặc dù là hạn chế của lõi chấm bài Judge0, nhưng vẫn có cách khắc phục như truyền tải nội dung tệp qua yêu cầu HTTP có thể được triển khai trong tương lai.

4.5.4. Rủi ro

4.5.4.1. Rủi ro về vận hành và tải của hệ thống

Trong các kỳ thi lớn hoặc khi xảy ra sự cố mạng khiến sinh viên nộp lại bài hàng loạt, số lượng yêu cầu chấm bài có thể tăng đột biến vượt quá khả năng xử lý của cụm máy chấm. Điều này dẫn đến hiện tượng ùn ứ hàng đợi, làm tăng độ trễ trả kết quả từ vài giây lên vài phút, gây tâm lý không tốt cho thí sinh.

Mặc dù hệ thống được thiết kế để có thể mở rộng bất cứ khi nào, nhưng quy trình mở rộng hệ thống hiện tại yêu cầu mất thời gian để cấu hình và cài đặt thủ công các máy chủ. Điều này khiến cho người vận hành cần ước lượng trước về nhu cầu sử dụng tại từng thời điểm để có thể chủ động thay đổi quy mô hệ thống để có

thể đáp ứng đúng thực tế, tránh việc thiếu hụt hoặc thừa thãi hiệu năng gây lãng phí tài nguyên máy chủ.

4.5.4.2. Rủi ro về nguy cơ bị tấn công trong môi trường thực thi mã (sandbox)

Kẻ tấn công có thể lợi dụng các lỗ hổng trong trình biên dịch hoặc hệ điều hành để viết các đoạn mã độc nhằm thoát khỏi môi trường cô lập, từ đó chiếm quyền điều khiển máy chủ, đọc trộm mã nguồn hệ thống hoặc tấn công vào mạng nội bộ của nhà trường. Hiện tại hệ thống đã tận dụng tối đa các cơ chế cô lập môi trường thực thi mã (Docker container, isolate…), nhưng không ngoại trừ rủi ro khi hệ thống sau này có thể lỗi thời hoặc lộ ra những "backdoor".

4.5.4.3. Rủi ro khi tích hợp AI trong tương lai

Mặc dù tính năng AI đang ở giai đoạn thiết kế, nhưng việc phân tích rủi ro là cần thiết để chuẩn bị cho việc triển khai sau này.

- Độ trễ phản hồi: Các mô hình ngôn ngữ lớn thường mất từ vài giây đến hàng chục giây để sinh câu trả lời hoàn chỉnh. Điều này có thể làm gián đoạn trải nghiệm học tập thời gian thực. Giải pháp là sử dụng cơ chế streaming để giảm độ trễ nhận thức của người dùng.
- Ảo giác của AI: AI có thể sinh ra các lời giải sai cú pháp, sai logic hoặc bịa đặt các kiến thức không có thật nhưng với văn phong rất thuyết phục. Điều này cực kỳ nguy hiểm trong môi trường giáo dục vì có thể làm sinh viên hiểu sai bản chất vấn đề. Giải pháp là giới hạn phạm vi kiến thức của AI bằng kỹ thuật RAG dựa trên giáo trình chuẩn của nhà trường thay vì để AI tự do lấy kiến thức từ internet.
- Chi phí vận hành: Việc sử dụng API của các mô hình thương mại hoặc tự vận hành mô hình mã nguồn mở đều tốn kém tài nguyên tính toán và chi phí tokens khi quy mô người dùng lớn. Cần có chiến lược cache câu trả lời hoặc sử dụng các mô hình nhỏ hơn được tinh chỉnh chuyên biệt cho lập trình để tối ưu chi phí.

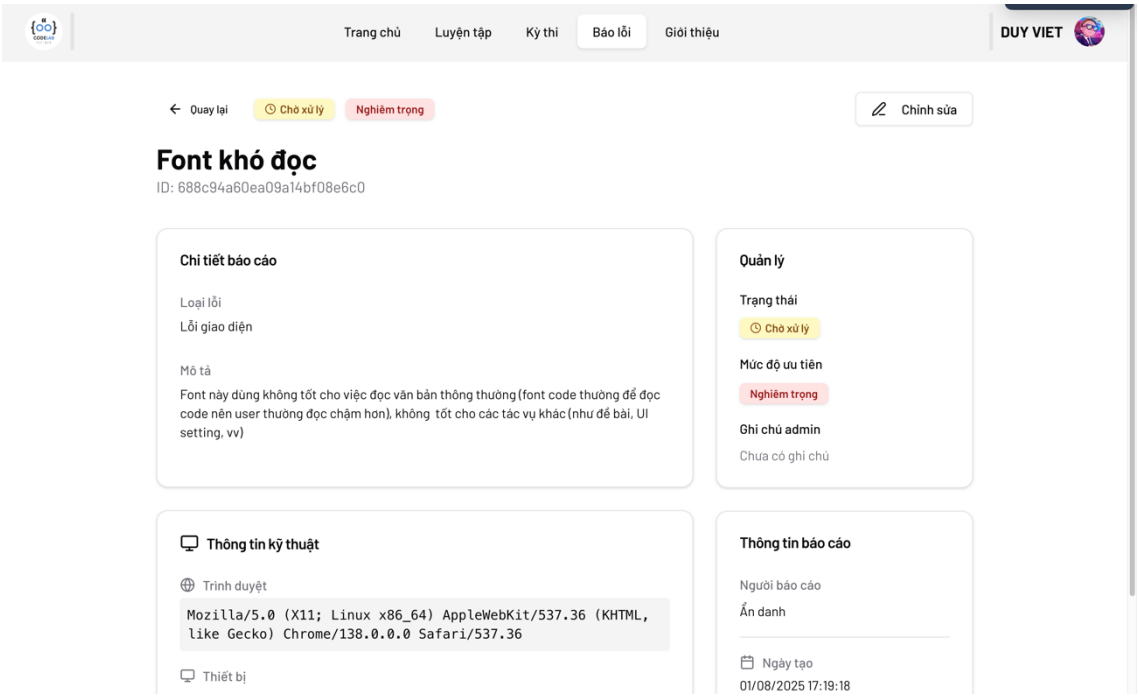
4.6. Đánh giá từ người dùng

Do giới hạn về thời gian thực hiện đề tài và phạm vi triển khai đang dừng ở mức thử nghiệm cục bộ, khóa luận chưa thực hiện các khảo sát diện rộng bằng bảng hỏi để đo lường chỉ số hài lòng của người dùng trên quy mô lớn. Thay vào đó, quá trình đánh giá tập trung vào việc thu thập phản hồi thông qua phỏng vấn

nhanh và quan sát trực tiếp trải nghiệm của nhóm người dùng chính bao gồm giảng viên và sinh viên tham gia các kỳ thi thử nghiệm.

Kết quả phỏng vấn nhanh cho thấy những tín hiệu rất tích cực từ phía người dùng đối với hệ thống NEU OJ. Đa số sinh viên đánh giá giao diện của nền tảng có thiết kế hiện đại, trực quan và thân thiện hơn đáng kể so với các hệ thống cũ, đồng thời trải nghiệm chuyển trang cũng mượt mà, giảm thiểu tối đa độ trễ tương tác. Bên cạnh đó, người dùng cũng bày tỏ sự hài lòng đặc biệt về hiệu năng hệ thống khi tốc độ chấm và trả kết quả đối với các bài tập cơ bản diễn ra gần như tức thì, giúp sinh viên duy trì được mạch tư duy làm bài liền mạch mà không bị gián đoạn bởi thời gian chờ đợi. Ở góc độ quản lý, giảng viên cũng đánh giá cao tính năng giám sát thi cử trực quan, cho phép phát hiện dễ dàng và chính xác các hành vi vi phạm quy chế như chuyển tab ngay trong thời gian thực.

Thông qua chức năng báo lỗi, hệ thống cũng đã ghi nhận được khoảng 40 ý kiến góp ý về giao diện, hệ thống và các báo cáo vấn đề gặp phải, từ đó giúp đội ngũ phát triển cũng dễ dàng, thuận tiện hơn trong việc đáp ứng nhu cầu của người dùng và cập nhật sửa lỗi còn tồn đọng.



Hình 4.29. Người dùng báo lỗi font web khó đọc.

4.7. Định hướng phát triển

4.7.1. Phát triển trợ lý CodeMentor

Một trọng tâm của định hướng phát triển là tính năng AI CodeMentor. AI CodeMentor trong bối cảnh giảng dạy đại học được thiết kế với mục tiêu cốt lõi là phát triển tư duy lập trình cho sinh viên. Điều này đòi hỏi sự chuyển dịch từ mô hình "trả lời câu hỏi" sang mô hình "đối thoại sư phạm", nghĩa là thay vì CodeMentor ngay lập tức cung cấp lời giải, đáp án của bài toán thì nó chỉ nên cung cấp cho người hỏi các kiến thức nền liên quan, dẫn dắt và khuyến khích người dùng tư duy để tự tìm ra lời giải. Những câu trả lời này cũng cần được đảm bảo độ chính xác tuyệt đối dựa trên các tài liệu mà người học đã được cung cấp từ giảng viên hay người hướng dẫn, do đó, yêu cầu sử dụng kỹ thuật RAG là bắt buộc.

Một số chức năng chính của CodeMentor có thể kể đến như:

- Tra cứu cú pháp và kiến thức nền tảng: CodeMentor sẽ đóng vai trò như một cuốn từ điển, cho phép sinh viên hỏi nhanh về cú pháp lệnh, các thư viện chuẩn hoặc các khái niệm thuật toán cơ bản mà không cần tự tìm kiếm.
- Giải thích lỗi biên dịch: Thay vì hệ thống sơ khai chỉ hiển thị các thông báo lỗi khô khan đến từ lõi chạy mã nguồn, CodeMentor sẽ phân tích log lỗi và hướng dẫn người dùng xác định nguyên nhân lỗi, gợi ý cách khắc phục.

4.7.2. Xây dựng quy trình đánh giá, chấm điểm kết hợp AI

Quy trình đánh giá, chấm điểm kết hợp AI cũng là một mục tiêu phát triển trong tương lai của NEU OJ. Mặc dù các mô hình ngôn ngữ lớn có khả năng đánh giá, đưa ra nhận xét, chúng vẫn tồn tại sai số, thiên kiến và đôi khi là sự không nhất quán. Do đó, một hệ thống đánh giá hiện đại phải là sự kết hợp giữa độ chính xác tuyệt đối của máy chấm dựa vào các đầu vào – đầu ra, khả năng phân tích sâu sắc của mô hình ngôn ngữ lớn và bao trùm lên chúng là một quy trình, quy chuẩn đánh giá do con người xây dựng.

Quy trình đánh giá có thể gồm một số lớp sau đây:

- Tầng 1: Đánh giá kết quả thực thi mã nguồn với OJ.
- Tầng 2: Đánh giá về phong cách trình bày như các quy ước được áp dụng hay không, có “code sạch” hay không và độ phức tạp của lời giải.
- Tầng 3: Dựa vào kết quả đánh giá của 2 tầng trên, kết hợp với chiến lược prompting và quy tắc chấm điểm được quy định từ trước, tất cả sẽ là đầu vào cho mô hình ngôn ngữ lớn để bắt đầu quá trình phân tích và nhận xét chi tiết.

KẾT LUẬN

Trong bối cảnh chuyển đổi số đang diễn ra mạnh mẽ tại các cơ sở giáo dục đại học, việc đổi mới phương pháp giảng dạy và đánh giá năng lực lập trình đã trở thành yêu cầu cấp thiết. Với định hướng nâng cao chất lượng đào tạo khối ngành lĩnh vực Công nghệ thông tin và Khoa học máy tính tại Đại học Kinh tế Quốc dân, nhu cầu về một môi trường thực hành hiện đại, tự động hóa và minh bạch là vô cùng thiết thực. Hệ thống NEU OJ được phát triển với hy vọng không chỉ giải quyết bài toán chấm điểm thủ công truyền thống mà còn tạo ra một không gian học tập năng động, góp phần vào công cuộc hiện đại hóa môi trường giáo dục của nhà trường.

Về mặt kỹ thuật, hệ thống được xây dựng dựa trên kiến trúc microservices hiện đại, ứng dụng các công nghệ tiên tiến như Next.js, NestJS và cơ sở dữ liệu NoSQL. Đặc biệt, việc tích hợp thành công lõi chấm bài Judge0 với cơ chế cô lập và cơ chế cân bằng tải đã đảm bảo hệ thống vận hành an toàn, bảo mật và ổn định ngay cả khi chịu tải cao. Hệ thống đã hiện thực hóa thành công các chức năng cốt lõi bao gồm: IDE trực tuyến hỗ trợ đa ngôn ngữ, cơ chế chấm điểm tự động trả về phản hồi tức thì, và module quản lý thi cử tích hợp các công cụ giám sát giúp phát hiện gian lận hiệu quả.

Kết quả triển khai thực tế cho thấy NEU OJ đã đáp ứng tốt nhu cầu sử dụng trong các học phần C, Java và đặc biệt là tổ chức thành công kỳ thi Olympic Tin học sinh viên cấp đại học với sự tham gia của cả khối Chuyên và Không chuyên.

Quá trình nghiên cứu và phát triển NEU OJ đã giúp tôi có cái nhìn tổng quan sâu sắc về quy trình xây dựng hệ thống phần mềm quy mô lớn, kỹ thuật ảo hóa và các phương pháp đánh giá năng lực lập trình tự động. Tôi xin gửi lời cảm ơn chân thành tới các thầy cô khoa Công nghệ thông tin, Trường Công nghệ; đặc biệt là TS. Phạm Xuân Lâm đã tận tình hướng dẫn và hỗ trợ tôi trong suốt quá trình thực hiện đề tài.

Tuy nhiên, do hạn chế về thời gian và nguồn lực, dự án vẫn còn một số điểm cần khắc phục. Từ những kinh nghiệm thu được, tôi cam kết sẽ tiếp tục nghiên cứu để tích hợp các chức năng, công nghệ mới nhằm cải thiện hệ thống, nhằm đưa NEU OJ trở thành một nền tảng toàn diện, đóng góp tích cực vào hệ sinh thái công nghệ của Đại học Kinh tế Quốc dân.

TÀI LIỆU THAM KHẢO

[1] S. Taxipulati and H.-D. Lu, "The Influence of Feedback Content and Feedback Time on Multimedia Learning Achievement of College Students and Its Mechanism," *Front. Psychol.*, vol. 12, p. 706821, Dec. 2021, doi: 10.3389/fpsyg.2021.706821.

[2] L. Y. Tan, S. Hu, D. J. Yeo, and K. H. Cheong, "A Comprehensive Review on Automated Grading Systems in STEM Using AI Techniques," *Mathematics*, vol. 13, no. 17, p. 2828, Sept. 2025, doi: 10.3390/math13172828.

[3] S. Wasik, M. Antczak, J. Badura, A. Laskowski, and T. Sternal, "A Survey on Online Judge Systems and Their Applications," *ACM Comput. Surv.*, vol. 51, no. 1, pp. 1–34, Jan. 2019, doi: 10.1145/3143560.

[4] J. C. Paiva, J. P. Leal, and Á. Figueira, "Automated Assessment in Computer Science Education: A State-of-the-Art Review," *ACM Trans. Comput. Educ.*, vol. 22, no. 3, pp. 1–40, Sept. 2022, doi: 10.1145/3513140.

[5] I. Mekterović, L. Brkić, and M. Horvat, "Scaling Automated Programming Assessment Systems," *Electronics*, vol. 12, no. 4, p. 942, Feb. 2023, doi: 10.3390/electronics12040942.

[6] S. Verma, "5 Reasons Why Immediate Feedback is Important for Effective Learning." Accessed: Nov. 25, 2025. [Online]. Available: https://www.blog.intedashboard.com/blogs/tbl-learning/immediate-feedback

[7] X. Liu, S. Wang, P. Wang, and D. Wu, "Automatic Grading of Programming Assignments: An Approach Based on Formal Semantics," in *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET)*, Montreal, QC, Canada: IEEE, May 2019, pp. 126–137. doi: 10.1109/ICSE-SEET.2019.00022.

[8] "PC2's Wiki." Accessed: Nov. 26, 2025. [Online]. Available: https://pc2ccs.github.io/

[9] M. A. Revilla, S. Manzoor, and R. Liu, "Competitive Learning in Informatics: The UVa Online Judge Experience," vol. 2, pp. 131–148, 2008.

[10] H. Z. Dosilovic and I. Mekterovic, "Robust and Scalable Online Code Execution System," in *2020 43rd International Convention on Information,*

Communication and Electronic Technology (MIPRO), Opatija, Croatia: IEEE, Sept. 2020, pp. 1627–1632. doi: 10.23919/MIPRO48935.2020.9245310.

[11] A. Bernik, D. Radošević, and A. Čep, "A Comparative Study of Large Language Models in Programming Education: Accuracy, Efficiency, and Feedback in Student Assignment Grading," *Appl. Sci.*, vol. 15, no. 18, p. 10055, Sept. 2025, doi: 10.3390/app151810055.

[12] M. Jukiewicz, "A systematic comparison of Large Language Models for automated assignment assessment in programming education: Exploring the importance of architecture and vendor," Sept. 30, 2025, *arXiv*: arXiv:2509.26483. doi: 10.48550/arXiv.2509.26483.

[13] A. Piscitelli, M. De Rosa, V. Fuccella, and G. Costagliola, "Large Language Models for Student Code Evaluation: Insights and Accuracy:," in *Proceedings of the 17th International Conference on Computer Supported Education*, Porto, Portugal: SCITEPRESS - Science and Technology Publications, 2025, pp. 534–544. doi: 10.5220/0013287500003932.

[14] P. Kehrer, K. Kelly, and N. Heffernan, "Does Immediate Feedback While Doing Homework Improve Learning?," *Proc. Twenty-Sixth Int. Fla. Artif. Intell. Res. Soc. Conf.*, pp. 542–545, 2013.

[15] Y. Attali and D. Powers, "Immediate Feedback and Opportunity to Revise Answers to Open-Ended Questions," *Educ. Psychol. Meas.*, vol. 70, no. 1, pp. 22–35, Feb. 2010, doi: 10.1177/0013164409332231.

[16] J.-Y. Kuo, H.-C. Lin, P.-F. Wang, and Z.-G. Nie, "A Feedback System Supporting Students Approaching a High-Level Programming Course," *Appl. Sci.*, vol. 12, no. 14, p. 7064, July 2022, doi: 10.3390/app12147064.

[17] N. Slijepcevic and A. Yaylali, "Leveraging 'Khanmigo' Generative AI-Powered Tool for Personalized Tutoring to Learn Scientific Concepts," *J. Teach. Learn.*, vol. 19, no. 4, Oct. 2025, doi: 10.22329/jtl.v19i4.10052.

[18] M. Mahdaoui, S. Nouh, M. S. El Kasmi Alaoui, and K. Kandali, "Automated Grading Method of Python Code Submissions Using Large Language Models and Machine Learning," *Information*, vol. 16, no. 8, p. 674, Aug. 2025, doi: 10.3390/info16080674.

[19] Z. Liu, P. Agrawal, S. Singhal, V. Madaan, M. Kumar, and P. K. Verma, "LPITutor: an LLM based personalized intelligent tutoring system using RAG

and prompt engineering," *PeerJ Comput. Sci.*, vol. 11, p. e2991, Aug. 2025, doi: 10.7717/peerj-cs.2991.

[20] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, Eds., *Compilers: principles, techniques, & tools*, 2. ed., Pearson internat. ed. Boston Munich: Pearson Addison-Wesley, 2007.

[21] J.-Y. Kuo, Z.-J. Wen, T.-F. Hsieh, and H.-X. Huang, "A Study on the Security of Online Judge System Applied Sandbox Technology," *Electronics*, vol. 12, no. 14, p. 3018, July 2023, doi: 10.3390/electronics12143018.

[22] S. Maggiolo and G. Mascellani, "Introducing CMS: A Contest Management System," *Olymp. Inform.*, vol. 6, pp. 86–99, 2012.

[23] G.-C. Pan, P. Liu, and J.-J. Wu, "A Cloud-Native Online Judge System," in *2022 IEEE 46th Annual Computers, Software, and Applications Conference (COMPSAC)*, Los Alamitos, CA, USA: IEEE, June 2022, pp. 1293–1298. doi: 10.1109/COMPSAC54236.2022.00204.

PHỤ LỤC

5.1. Phụ lục A: Xây dựng hệ thống

5.1.1. Xây dựng hệ thống back-end

5.1.1.1. Thiết lập NodeJS và NestJS

Sử dụng NestJS Framework trên nền tảng Node.js để xây dựng hệ thống backend cho NEU OJ. Với ưu điểm về kiến trúc module chặt chẽ, sử dụng TypeScript giúp đảm bảo tính định kiểu mạnh và khả năng mở rộng cao; NestJS cho phép triển khai nhanh chóng các API hiệu năng cao và dễ dàng bảo trì. Hệ thống backend này tuân theo kiến trúc Modular Architecture, bao gồm các thành phần chính:

- Entities & Schemas (Data Source): Thực hiện tương tác trực tiếp với cơ sở dữ liệu để thao tác CRUD. Sử dụng thư viện Mongoose (đối với CSDL MongoDB) mà NestJS hỗ trợ sẵn để tương tác thông qua ODM. Điều này cho phép thao tác với dữ liệu thông qua các phương thức được cung cấp sẵn mà không cần viết câu truy vấn thủ công, đồng thời hỗ trợ kết nối linh hoạt tới các cơ sở dữ liệu khác như Redis nếu cần mở rộng.
- Services & Repositories: Đóng vai trò là lớp trung gian xử lý logic nghiệp vụ. Service sẽ nhận yêu cầu từ Controller, thực hiện chuyển đổi, tính toán và gọi xuống tầng Data Source thông qua Repository pattern để đảm bảo tính nhất quán và tách biệt trách nhiệm của dữ liệu.
- Controllers (Endpoints): Danh sách các API được định nghĩa sẵn thông qua các Class được gắn Decorator. Mỗi API có một đường dẫn và phương thức HTTP riêng biệt, tuân theo quy tắc RESTful: GET để truy vấn, POST để tạo mới, PUT/PATCH để cập nhật. Các Controller này nhận dữ liệu từ người dùng (thông qua DTO - Data Transfer Object), gọi đến Service xử lý và trả về kết quả dưới dạng JSON.

Để đảm bảo tính đồng bộ và khả năng tương thích cao khi tích hợp với các hệ thống bên ngoài, định dạng dữ liệu trong thân yêu cầu và dữ liệu trả về được chuẩn hóa thống nhất theo quy tắc snake_case. Về mặt kỹ thuật, hệ thống tận dụng các Interceptor và cơ chế tuần tự hóa của NestJS để tự động chuyển đổi hai chiều: từ camelCase trong logic xử lý nội tại sang snake_case khi xuất dữ liệu JSON, đảm bảo cấu trúc trả về luôn tuân thủ quy định chung.

Về cơ chế bảo mật, hệ thống API được phân loại thành hai nhóm chính: API công khai và API riêng tư. Trong đó, các API riêng tư yêu cầu xác thực người dùng thông qua mã thông báo JWT (JSON Web Token) được cấp phát bởi Backend sau khi đăng nhập thành công, trong khi API công khai cho phép truy cập tự do.

5.1.1.2. Thiết lập cơ sở dữ liệu

Sử dụng Docker để thực hiện container hoá cơ sở dữ liệu MongoDB (và các dịch vụ phụ trợ như Redis, MinIO) nhằm đảm bảo tính đồng bộ giữa môi trường phát triển và triển khai, đồng thời giúp việc chuyển tiếp lên máy chủ (Application Server) diễn ra nhanh chóng, thuận tiện.

Đối với việc quản lý dữ liệu, thư viện Mongoose đóng vai trò là ODM trong kiến trúc NestJS. Khác với cơ chế tạo bảng cứng của cơ sở dữ liệu quan hệ:

- Schema & Model: Mongoose cho phép định nghĩa cấu trúc dữ liệu thông qua các lớp ngay trong mã nguồn. Mongoose sẽ tự động ánh xạ các lớp này thành các bộ sưu tập tương ứng trong MongoDB khi có dữ liệu phát sinh hoặc khi khởi tạo.
- Indexing: Các chỉ mục giúp tăng tốc độ truy vấn được khai báo trực tiếp trong cấu trúc dữ liệu. Khi ứng dụng khởi chạy, Mongoose sẽ tự động kiểm tra và tạo các chỉ mục này trong MongoDB nếu chúng chưa tồn tại.

Thực hiện cấu hình các thông số nhạy cảm của cơ sở dữ liệu như chuỗi kết nối, tên cơ sở dữ liệu, tài khoản và mật khẩu vào tệp biến môi trường **“.env”**. Hệ thống sử dụng module cấu hình hệ thống của NestJS để đọc các tham số này và tự động thiết lập kết nối thông qua module Mongoose ngay khi backend được khởi chạy.

5.1.1.3. Thiết lập dịch vụ biên dịch và thực thi mã nguồn - Judge0

Judge0 được thiết kế là một hệ thống thực thi mã trực tuyến độc lập, mã nguồn mở, tập trung vào tính mạnh mẽ và khả năng mở rộng.

Về kiến trúc, Judge0 tuân theo mô hình module hóa hiện đại giúp tách biệt việc thực thi mã khỏi logic nghiệp vụ của ứng dụng, bao gồm các thành phần chính sau:

- Web API: Cung cấp giao diện JSON đơn giản cho phép người dùng gửi mã nguồn và nhận kết quả thực thi thông qua cơ chế đồng bộ hoặc bất đồng bộ.
- Middleware: Sử dụng PostgreSQL làm cơ sở dữ liệu để lưu trữ thông tin

bài nộp và Redis đóng vai trò hàng đợi. Hàng đợi này giúp điều phối các tác vụ gửi đến các worker, đảm bảo hệ thống không bị quá tải.

- Code Execution Engine (CEE): Đây là các worker thực hiện việc biên dịch và chạy mã. CEE sử dụng isolate – một công nghệ sandbox đã được kiểm chứng tại các kỳ thi IOI – để cô lập tài nguyên (CPU, RAM) và bảo mật hệ thống trước các đoạn mã không tin cậy.

Về thiết lập triển khai, Judge0 tận dụng Docker và Docker Compose để đóng gói toàn bộ các thành phần (API, Database, Redis, Workers). Điều này cho phép triển khai hệ thống nhanh chóng trên bất kỳ nền tảng nào hỗ trợ Docker, đồng thời dễ dàng mở rộng theo chiều ngang bằng cách tăng số lượng container CEE xử lý trên nhiều máy chủ khác nhau. Đồng thời, với các yêu cầu nâng cao hơn, ví dụ như bổ sung thư viện liên quan đến thống kê hay học máy cho Python, đều có thể thực hiện được nhờ khả năng hỗ trợ cấu hình tuỳ biến của Judge0.

5.1.1.4. Thiết lập mở rộng và cân bằng tải cho Judge0

Là "trái tim" của hệ thống chấm bài trực tuyến, Judge0 chịu trách nhiệm tiếp nhận, biên dịch và thực thi mã nguồn trong môi trường Sandbox cô lập. Để đảm bảo hệ thống hoạt động ổn định dưới áp lực truy cập lớn và tránh điểm nghẽn cổ chai khi số lượng sinh viên nộp bài tăng đột biến, kiến trúc của Judge0 được thiết lập mở rộng theo chiều ngang.

Mô hình triển khai cụm: Thay vì triển khai trên một máy chủ vật lý duy nhất, hệ thống thực thi mã nguồn được nhân bản trên nhiều máy chủ khác nhau. Mỗi node chạy một bản cài đặt Judge0 đầy đủ (bao gồm API và worker) hoặc các worker chuyên biệt. Tất cả các node này kết nối chung tới một cơ sở dữ liệu PostgreSQL và chia sẻ chung một hàng đợi Redis để đảm bảo tính nhất quán dữ liệu và trạng thái bài nộp.

Nginx làm bộ cân bằng tải (Load Balancer): Để điều phối lưu lượng truy cập tới cụm máy chủ này, hệ thống sử dụng Nginx đóng vai trò là reverse proxy và cân bằng tải đặt tại lớp tiền xử lý.

- Cơ chế phân phối: Nginx được cấu hình sử dụng thuật toán Round-robin. Khi NestJS back-end gửi yêu cầu chấm bài – một yêu cầu HTTP, Nginx sẽ tự động điều hướng yêu cầu này tới một trong các máy chủ Judge0 đang hoạt động trong cụm. Việc này giúp chia đều tải trọng xử lý (CPU/RAM), ngăn chặn tình trạng một máy chủ bị quá tải trong khi các máy khác đang rảnh rỗi.

- Cấu hình upstream: Các địa chỉ IP của các máy chủ Judge0 thành viên được khai báo trong khối upstream của file cấu hình Nginx. Nginx cũng thực hiện cơ chế kiểm tra sức khỏe thụ động; nếu một node Judge0 gặp sự cố không phản hồi, Nginx sẽ tạm thời loại bỏ node đó khỏi danh sách điều hướng và chuyển tải sang các node còn lại, đảm bảo tính liên tục của dịch vụ.

Việc kết hợp Nginx và kiến trúc mở rộng của Judge0 cho phép hệ thống dễ dàng nâng cấp năng lực xử lý bằng cách bổ sung thêm máy chủ mới vào cụm mà không làm gián đoạn dịch vụ hiện tại.

5.1.2. Trực quan hoá dữ liệu bằng Grafana và Prometheus

Hệ thống backend NEU OJ thực hiện thu thập và giám sát toàn diện các thông tin bao gồm:

- Số liệu hiệu năng: Dữ liệu về tài nguyên hệ thống (CPU, RAM, Disk), độ trễ của Event Loop, và lưu lượng truy cập API được thu thập định kỳ.
- Nhật ký hoạt động: Các vấn đề phát sinh trong quá trình xử lý, chi tiết lỗi và thông tin luồng yêu cầu – phản hồi được ghi lại dưới chuẩn cấu trúc JSON.

Trong kiến trúc này, Grafana đóng vai trò là trung tâm điều khiển, tự động liên kết tới hai nguồn dữ liệu chính: Prometheus (lưu trữ dữ liệu chuỗi thời gian - time series) và Loki (lưu trữ dữ liệu log tập trung). Thay vì chỉ lưu trữ, hệ thống sử dụng Promtail để thu thập logs từ backend, phân tích cú pháp và chuyển tiếp về Loki.

Grafana lúc này đóng vai trò là công cụ mạnh mẽ để truy vấn dữ liệu thông qua ngôn ngữ PromQL và LogQL. Đồng thời, các bảng điều khiển chuyên biệt như *"Node.js Performance"* hay *"Backend Logs"* được thiết lập để trực quan hóa các thông số này theo thời gian thực. Từ đó, giúp quản trị viên dễ dàng truy vết nguyên nhân sự cố, nhận cảnh báo sớm và có cái nhìn tổng quan về sức khỏe của toàn bộ hệ thống hạ tầng.

5.1.3. Xây dựng hệ thống giao diện NEU OJ

5.1.3.1. Thiết lập Next.js Framework

Hệ thống giao diện người dùng của NEU OJ được xây dựng dựa trên Next.js, một framework mạnh mẽ được phát triển trên nền tảng React. Việc lựa chọn Next.js mang lại lợi thế vượt trội nhờ cơ chế render hỗn hợp, cho phép kết hợp linh hoạt giữa render phía máy chủ để tối ưu hóa SEO và tốc độ tải trang ban

đầu, với render phía máy khách để đảm bảo trải nghiệm tương tác mượt mà như một ứng dụng Single Page Application (SPA).

Mã nguồn front-end được viết hoàn toàn bằng TypeScript, đảm bảo tính đồng bộ về mặt định kiểu dữ liệu với hệ thống back-end NestJS, giúp giảm thiểu lỗi trong quá trình phát triển và dễ dàng bảo trì.

5.1.3.2. Kiến trúc thành phần và giao diện người dùng

Giao diện được tổ chức theo kiến trúc hướng thành phần, chia nhỏ giao diện thành các module tái sử dụng. Hệ thống sử dụng các thư viện hỗ trợ chuyên biệt:

- UI Framework: Sử dụng Tailwind CSS kết hợp với Shadcn để xây dựng hệ thống giao diện nhất quán, hiện đại và có khả năng thích ứng trên đa thiết bị từ máy tính đến điện thoại di động.
- Monaco Editor: Tích hợp trình soạn thảo mã nguồn Monaco Editor (lõi của VS Code) vào trang làm bài. Điều này cung cấp trải nghiệm lập trình chuyên nghiệp cho sinh viên với các tính năng như: tô màu cú pháp, gợi ý code và tự động định dạng cho nhiều ngôn ngữ lập trình khác nhau (C++, Java, Python...).
- Markdown Rendering: Sử dụng thư viện để hiển thị nội dung đề bài, hỗ trợ render các công thức toán học (KaTeX) phục vụ cho các bài tập thuật toán phức tạp.

5.1.4. Thực hiện triển khai lên môi trường Production

Quá trình triển khai hệ thống NEU OJ được thực hiện trên hạ tầng máy chủ vật lý do khoa Công nghệ thông tin cung cấp, đảm bảo tính ổn định và khả năng chịu tải phục vụ cho một số lượng lớn sinh viên sử dụng.

5.1.4.1. Kiến trúc hạ tầng máy chủ

Hệ thống áp dụng mô hình triển khai phân tán trên hạ tầng gồm 02 máy chủ vật lý vận hành hệ điều hành Ubuntu Server 24.04 LTS. Kiến trúc được phân chia vai trò cụ thể như sau:

- Máy chủ 1 (Primary Node): Đóng vai trò là máy chủ ứng dụng trung tâm tích hợp. Nơi đây vận hành toàn bộ các thành phần cốt lõi bao gồm: back-end (NestJS), front-end (Next.js), hệ quản trị cơ sở dữ liệu (MongoDB, Redis, MinIO) và bộ công cụ giám sát (Grafana Stack). Đồng thời, máy chủ

này cũng khởi chạy một worker của hệ thống chấm bài Judge0 để xử lý các yêu cầu tại chỗ.

- Máy chủ 2: Đóng vai trò là máy chủ thực thi mở rộng, vận hành song song một dịch vụ Judge0 thứ hai. Máy chủ này giúp cân bằng tải, chia sẻ khối lượng tính toán với máy chủ 1, từ đó nâng cao năng lực xử lý đồng thời của hệ thống.

Quá trình giao tiếp và đồng bộ dữ liệu giữa hai máy chủ được thực hiện hoàn toàn thông qua mạng nội bộ của trường nhằm tối ưu hóa băng thông và giảm thiểu độ trễ.

5.1.4.2. Khởi chạy hệ thống và thiết lập luồng triển khai tự động

Hệ thống được triển khai trên hạ tầng phân tán gồm 02 máy chủ Ubuntu Server 24.04 LTS. Để đảm bảo tính nhất quán và khả năng mở rộng, quy trình vận hành dựa trên ba trụ cột công nghệ chính:

1. Container hóa và điều phối hạ tầng: Toàn bộ các thành phần dịch vụ của hệ thống được đóng gói thành các Docker container độc lập và quản lý thống nhất thông qua Docker Compose. Tại máy chủ ứng dụng trung tâm, Nginx đóng vai trò là gateway và reverse proxy. Nginx không chỉ thực hiện mã hóa bảo mật SSL/TLS mà còn hoạt động như một bộ cân bằng tải, điều phối các yêu cầu thực thi mã nguồn phân tán đều về cả hai máy chủ, giúp tối ưu hóa tài nguyên và ngăn chặn quá tải cục bộ.

2. Quy trình tích hợp và triển khai liên tục (CI/CD): Hệ thống thiết lập quy trình tự động hóa khép kín sử dụng GitHub Actions, giúp giảm thiểu thao tác thủ công và rủi ro sai sót.

KẾT QUẢ TURNITIN

11226912_TranDuyViet_ThietKeVaPhatTrienNenTangChamB...

ORIGINALITY REPORT

14%	12%	12%	9%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to National Economics University Student Paper	4%
2	www.ctu.edu.vn Internet Source	3%
3	Submitted to Thai Nguyen University of Education Student Paper	1%
4	Phenikaa University Publication	1%
5	Submitted to University of Finance – Marketing Student Paper	1%
6	Hanoi National University of Education Publication	<1%
7	Submitted to Ho Chi Minh City Open University Student Paper	<1%
8	Submitted to Nha Trang University Student Paper	<1%
9	Submitted to Ton Duc Thang University Student Paper	<1%
10	files.qnamuni.edu.vn Internet Source	<1%